

EARLY REVIEW EDITION
WORK IN PROGRESS

Enterprise Mainframe Transformation

Evidence, Dependency Graphs, Bounded Inference,
and HLASM Comprehension

ARSHADUL SHAIKH

This is an early review edition for technical learning and practitioner feedback. It is not a final edition, official guide, production certification, or vendor training material.

Evidence Before Modernization

How a Mainframe Transformation Pipeline Reads, Groups, Translates and Refuses

Working title.

EARLY REVIEW EDITION — WORK IN PROGRESS

This is an early review edition of a book that is still being written. It is not a final book, not a certification, not official documentation and not vendor training.

Chapters 1 to 12 form the opening of the book. Chapters 21 to 24, on analysing assembler source, are released early as a special review section so that readers with assembler experience can challenge them. Chapters 13 to 20 and 25 to 33 are forthcoming and are listed by title only.

Text, figures and measurements may change before any later edition. Where this edition does not establish a fact, it says so in words.

Contents

[Evidence Before Modernization](#)

[EARLY REVIEW EDITION — WORK IN PROGRESS](#)

[Notices](#)

[Edition notice](#)

[Independence](#)

[Trademarks and non-endorsement](#)

[The case study is fictional](#)

[No certification](#)

[Public sample applications](#)

[Vendor documentation](#)

[How to Read the Evidence Labels](#)

[1. Three levels in every chapter](#)

[2. Three truths, kept apart](#)

[3. Three kinds of factual claim](#)

[4. Code labels](#)

[5. Figure labels](#)

[6. The case study and its three layers](#)

[7. Items not established in this edition](#)

[8. Current Support and Current Limitation](#)

[9. UNKNOWN, UNSUPPORTED, AMBIGUOUS](#)

[10. Why failures and refusals appear on purpose](#)

[11. Numbers, dates and baselines](#)

[12. Chapter furniture](#)

[Open-Source Attribution](#)

[Notes](#)

[Chapter 1 — Why Modernization Claims Fail](#)

[Level 1 — Intuition](#)

[A question with three questions inside](#)

[What Quillon's bundle contains](#)

[Seven confusions](#)

[Two Quillon scenes](#)

[Level 2 — Engineering mechanics](#)

[Three kinds of factual claim](#)

[Evidence](#)

[UNKNOWN is not NONE](#)

[Duplicate evidence is not corroboration](#)

[Refusal over guessing](#)

[Quillon, sorted](#)

[Level 3 — Architecture reasoning](#)

[The pattern behind the seven confusions](#)

[Evidence-bounded modernization](#)

[Architect's Warning](#)

[Claim Check](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 2 — Three Truths: Platform, Implementation, Recommendation](#)

[Level 1 — Intuition](#)

[The speed limit, the speedometer and the satnav](#)

[Where the Chapter 1 belief sits](#)

[Level 2 — Engineering mechanics](#)

[Authority depends on the question](#)

[How a sentence is built](#)

[Seven worked examples](#)

[Level 3 — Architecture reasoning](#)

[Three inferences that look valid and are not](#)

[Architect's Warning](#)

[Claim Check](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 3 — ETC in One Picture](#)

[Level 1 — Intuition](#)

[Read the estate before redesigning it](#)

[Two words to fix first](#)

[Three names to remember](#)

[Level 2 — Engineering mechanics](#)

[The stages, and what stands behind each](#)

[A worked reading of one row](#)

[Where the deterministic core stops](#)

[Two figures: understanding, then transforming](#)

[FIG-03-1A — How ETC understands a mainframe estate · AUTHOR-CONCEPTUAL](#)

[FIG-03-1B — How ETC transforms and verifies a program · AUTHOR-CONCEPTUAL](#)

[Level 3 — Architecture reasoning](#)

[Reading the picture as an architect](#)
[Claim Check](#)
[EvidenceGraph is not business architecture](#)
[ETC Lesson Learned](#)
[Current Support](#)
[Current Limitation](#)
[Quick Recall](#)
[Architecture Question](#)
[Key Takeaways](#)

[Chapter 4 — How a Batch Night Runs](#)

[Level 1 — Intuition](#)

[A night shift with a wall plan](#)
[Quillon's night](#)
[FIG-04-1 — A batch night · AUTHOR-CONCEPTUAL](#)

[Level 2 — Engineering mechanics](#)

[Jobs, steps and programs](#)
[Return codes](#)
[`COND`: a test that bypasses](#)
[`IF`/`THEN`/`ELSE`/`ENDIF`: the positive model](#)
[When no test applies](#)
[The scheduler: declared prerequisites](#)
[Hands-On Exercise — read the night](#)
[Quillon, sorted](#)

[Level 3 — Architecture reasoning](#)

[Scheduler order is not data dependency, and the reverse](#)
[A return code is not the business result](#)
[Architect's Warning](#)
[Claim Check](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 5 — JCL as Evidence](#)

[Level 1 — Intuition](#)

[A work order, not the work](#)
[FIG-05-1 — Anatomy of a JCL job · SOURCE-DERIVED](#)

[Level 2 — Engineering mechanics](#)

[Part A. What JCL means on z/OS](#)
[Part B. What ETC currently records](#)
[Current Support](#)

[Current Limitation](#)

[Code Trace Challenge — predict what is recorded](#)

[Quillon, sorted](#)

[Level 3 — Architecture reasoning](#)

[Platform coverage is not parser coverage](#)

[Failure Case — a clean read that says less than it seems](#)

[Declared text is evidence of a request](#)

[Architect's Warning](#)

[Claim Check](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 6 — Procedures: Declared vs Effective Execution](#)

[Level 1 — Intuition](#)

[A form letter and its blanks](#)

[Quillon's `QMABL020`](#)

[FIG-06-1 — PROC expansion · AUTHOR-CONCEPTUAL](#)

[Level 2 — Engineering mechanics](#)

[Part A. What procedures mean on z/OS](#)

[Part B. `QMABL020` expanded by hand](#)

[Part C. What ETC currently does](#)

[FIG-06-2 — Declared vs effective execution · AUTHOR-CONCEPTUAL](#)

[Current Support](#)

[Current Limitation](#)

[Code Trace Challenge — a cycle](#)

[Quillon, sorted](#)

[Level 3 — Architecture reasoning](#)

[Three views, one of them absent](#)

[ETC Lesson Learned — expansion must not erase the invocation](#)

[Failure Case — an unresolved procedure](#)

[Failure Case and Architect's Warning — the DD override](#)

[The nesting limit](#)

[Cycles are ETC's, not the platform's](#)

[Architect's Warning](#)

[Claim Check](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 7 — DD Statements and the Five Axes of a Data Store](#)

[Level 1 — Intuition](#)

[One label, five questions](#)

[FIG-07-1 — The five axes of a data store · AUTHOR-CONCEPTUAL](#)

[The axes vary independently](#)

[Level 2 — Engineering mechanics](#)

[Part A. What the platform defines](#)

[Part B. What ETC records, axis by axis](#)

[FIG-07-2 — `DISP` against `OPEN` · AUTHOR-CONCEPTUAL and SOURCE-DERIVED](#)

[FIG-07-3 — Three cases of a missing `DISP` · SOURCE-DERIVED and AUTHOR-CONCEPTUAL](#)

[FIG-07-4 — A bundle-wide join · AUTHOR-CONCEPTUAL](#)

[Quillon's data stores](#)

[Current Support](#)

[Current Limitation](#)

[Code Trace Challenge — the join](#)

[Level 3 — Architecture reasoning](#)

[Why five axes and not one "type"](#)

[Failure Case — a missing edge that looks like a missing dependency](#)

[Architect's Warning](#)

[Claim Check](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 8 — COBOL Programs, Copybooks and Artifact Classification](#)

[Level 1 — Intuition](#)

[A shelf label is not the book](#)

[A member that carries the wrong label](#)

[Level 2 — Engineering mechanics](#)

[Part A. The artifacts on the platform](#)

[Part B. Who decides what, at the baseline](#)

[Pseudocode — two mechanisms, two pieces of pseudocode](#)

[The three reference estates, with their denominators](#)

[The `BANKDATA` story](#)

[FIG-08-1 — Artifact classification: location, type, eligibility · AUTHOR-CONCEPTUAL](#)

[The six members of `asm/`](#)

[Quillon, sorted](#)

[Current Support](#)

[Current Limitation](#)

[Code Trace Challenge — a data name that looks like an opcode](#)

[Level 3 — Architecture reasoning](#)

[Three questions, three owners](#)

[Architecture Question](#)

[ETC Lesson Learned — a path is a claim by whoever placed the file](#)

[Architect's Warning](#)

[Claim Check](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 9 — The Evidence Corpus: Public Estates, Digests and Provenance](#)

[Level 1 — Intuition](#)

[A sealed box is not the warehouse](#)

[Six terms](#)

[Level 2 — Engineering mechanics](#)

[How an imported estate is built](#)

[What each record establishes](#)

[Pinned revisions](#)

[What the digest proves](#)

[Provenance is a record, not a guarantee](#)

[FIG-09-1 — The evidence corpus · AUTHOR-CONCEPTUAL, with CORPUS-DERIVED counts](#)

[The validation corpus is not an estate](#)

[Licence evidence is not clearance](#)

[Quillon, positioned](#)

[Current Support](#)

[Current Limitation](#)

[Level 3 — Architecture reasoning](#)

[Why the distinction pays](#)

[Failure Case — counting the record](#)

[Architect's Warning](#)

[Claim Check](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 10 — The Typed EvidenceGraph](#)

[Level 1 — Intuition](#)

[A map with pins and threads](#)

[One name, two things](#)

[FIG-10-2 — Typed identity versus name-only lookup · AUTHOR-CONCEPTUAL](#)

[Level 2 — Engineering mechanics](#)

[What the graph is made of](#)

[Node kinds](#)

[Identity: kind and name](#)

[Edges: nineteen kinds, each with a meaning](#)

[The schema: allowed relationships](#)

[FIG-10-1 — A typed evidence graph, conceptually · AUTHOR-CONCEPTUAL](#)

[Where validation happens](#)

[Pseudocode — building the graph](#)

[Pseudocode — the four checks](#)

[Code Trace Challenge — four faults](#)

[What goes in a support](#)

[From job to program: declared, effective, runtime](#)

[Calls, and what 442 means](#)

[Shared layouts are not runtime links](#)

[The size of the graph](#)

[Estate boundaries: where the evidence stops](#)

[Quillon, sorted — one step, four edges](#)

[Level 3 — Architecture reasoning](#)

[Why not one "depends on" edge](#)

[EvidenceGraph is not a migration plan](#)

[Shape is not truth](#)

[Failure Case — "442 calls"](#)

[Architect's Warning](#)

[Claim Check](#)

[Current Support](#)

[Current Limitation](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 11 — Scheduler Evidence](#)

[Level 1 — Intuition](#)

[A flag board](#)

[What ETC does with a plan on paper](#)

[Level 2 — Engineering mechanics](#)

[The formats ETC reads](#)

[The three states of scheduler evidence](#)

[Control-M: what is in an export](#)

[Condition identity](#)

[`AND OR`: all, or any](#)

[`SIGN`: add and delete](#)

[A condition nobody posts](#)

[CA-7: schedule IDs, triggers and requirements](#)

[Issues the readers raise](#)

[From declaration to edges](#)

[Pseudocode — export to ordering edges](#)

[Quillon's scheduler export — layer A](#)

[Quillon, sorted — layer B](#)

[Preview of panel A \(scheduler layer\) of FIG-12-2 — the full figure is in Chapter 12 · AUTHOR-CONCEPTUAL](#)

[The scheduler-only job](#)

[Code Trace Challenge — an OR](#)

[Level 3 — Architecture reasoning](#)

[Declared is not run](#)

[The shape of ETC's errors](#)

[Why the omission matters](#)

[Failure Case — a name dropped, a schedule not missing](#)

[Architect's Warning](#)

[Claim Check](#)

[Current Support](#)

[Current Limitation](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 12 — Producer/Consumer Evidence, Corroboration and Conflict](#)

[Level 1 — Intuition](#)

[A rota, a delivery note and a receipt](#)

[Three kinds of store](#)

[Level 2 — Engineering mechanics](#)

[Where producers and consumers come from](#)

[One job, one store, two directions](#)

[Three things called "corroboration"](#)

[FIG-12-1 — Duplicate evidence, multiple named supports, and agreement · AUTHOR-CONCEPTUAL, with case study layer B](#)

[The 5, 17 and 0](#)

[Order findings](#)

[FIG-12-2 — Scheduler vs producer/consumer evidence · AUTHOR-CONCEPTUAL \(case study\), layer B](#)

[Store Role](#)

[FIG-12-3 — Store roles and the seeded store · AUTHOR-CONCEPTUAL, with case study layer B](#)

[Quillon's `QMAPV001` and `RATE.TABLE`](#)

[Quillon, sorted — four stores](#)

[Pseudocode — three rules, one path](#)

[Code Trace Challenge — flows, findings and roles](#)

[Level 3 — Architecture reasoning](#)

[From principle to rule](#)

[Conflict is kept, not resolved](#)

[Failure Case — choosing a winner for `QMABL040`](#)

[Architect's Warning](#)

[Claim Check](#)

[Current Support](#)

[Current Limitation](#)

[Quick Recall](#)

[Key Takeaways](#)

Special Early Review Section — Chapters 21–24

[Chapter 21 — Reading Assembler as Evidence](#)

[Level 1 — Intuition](#)

[A ledger in a hand you half read](#)

[The evidence ladder](#)

[Level 2 — Engineering mechanics](#)

[Physical location is not language type](#)

[How a member becomes an assembler unit](#)

[Program units: two, not six](#)

[The standard source format](#)

[Three kinds of operation](#)

[Sections: `START`, `CSECT` and `DSECT`](#)

[`ENTRY`, `EXTRN` and address constants: declarations](#)

[Symbols, `EQU` and `USING`](#)

[`COPY`: recorded, not expanded](#)

[`MACRO` ... `MEND`: tracked, not expanded](#)

[The location counter, and where it gives up](#)

[FIG-21-1 — HLASM reading layers · AUTHOR-CONCEPTUAL](#)

[Data access and CICS: not examined](#)

[Pseudocode — from text to locations](#)

[Code Trace Challenge — what the reader records](#)

[Quillon's assembler member](#)

[Level 3 — Architecture reasoning](#)

[Why assembler needs evidence discipline](#)

[Comprehension, not transformation](#)

[Failure Case — the directory inventory](#)

[Architect's Warning](#)

[Claim Check](#)

[Current Support](#)

[Current Limitation](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 22 — Bounded Control Flow: What Syntax Can Name](#)

[Level 1 — Intuition](#)

[Signposts and phone numbers](#)

[Three separate questions](#)

[Level 2 — Engineering mechanics](#)

[The raw control-flow graph](#)

[Where the raw CFG sits](#)

[The control-transfer vocabulary](#)

[`BR`: a branch through a register](#)

[`BAL` and `BALR`: branch and link](#)

[`BAS` and `BASR`: branch and save](#)

[Register zero: when branch-and-link does not branch](#)

[`BCTR`: decrement, then perhaps branch](#)

[Label targets: static, and not calls between programs](#)

[Branch through a register: why it stays unnamed](#)

[Refusal reasons](#)

[What stays out of the EvidenceGraph](#)

[FIG-22-1 — Raw CFG vs semantic transfer · AUTHOR-CONCEPTUAL, with case study layer B](#)

[Pseudocode — one section's raw CFG](#)

[Code Trace Challenge — name what you can](#)

[Level 3 — Architecture reasoning](#)

[Unknown is a better answer than invented](#)

[What a reviewer does with an unresolved transfer](#)

[Failure Case — the base register that became a call](#)

[Architect's Warning](#)

[Claim Check](#)

[Current Support](#)

[Current Limitation](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 23 — Linkage: External Calls and Intra-Assembly Section Calls](#)

[Level 1 — Intuition](#)

[A directory entry is not a phone call](#)

[Two kinds of linkage](#)

[Level 2 — Engineering mechanics](#)

[Address constants are evidence, not execution](#)

[What the binder does, and what it does not](#)

[FIG-23-1 — V-type vs A-type constants · SOURCE-DERIVED + AUTHOR-CONCEPTUAL](#)

[ETC's external-call rule](#)

[FIG-23-2 — The five facts of a proven external call · AUTHOR-CONCEPTUAL](#)

[Logical, not physical](#)

[When a fact is missing](#)

[The labelled V-type constant](#)

[Intra-assembly section calls](#)

[Why section calls stay out of `CALLS`](#)

[FIG-23-3 — BALR/BASR vs BR · SOURCE-DERIVED](#)

[What reaches the EvidenceGraph](#)

[`ENTRY` and `EXTRN` again](#)

[Quillon's `QMADTCV`: the linkage](#)

[Pseudocode — proving an external call](#)

[Code Trace Challenge — which are calls?](#)

[Level 3 — Architecture reasoning](#)

[Narrow rules, honest gaps](#)

[Failure Case — the V-con inventory](#)

[Architect's Warning](#)

[Claim Check](#)

[Current Support](#)

[Current Limitation](#)

[Quick Recall](#)

[Key Takeaways](#)

[Chapter 24 — Proving Returns and Refusing the Rest](#)

[Level 1 — Intuition](#)

[A key card in the right shape](#)

[Three truths about a return](#)

[Level 2 — Engineering mechanics](#)

[The standard linkage convention](#)

[Where a link register comes from](#)

[The chain ETC needs](#)

[FIG-24-1 — Convention vs proof · SOURCE-DERIVED + AUTHOR-CONCEPTUAL](#)

[What the validation corpus shows](#)

[The CardDemo exits: the textbook shape, refused](#)

[Quillon's `QMADTCV`: one proof path, three refusal paths](#)

[FIG-24-2 — A proof-path pattern and a refusal-path pattern · AUTHOR-CONCEPTUAL \(case study\),
with separate measured panels](#)

[Pseudocode — proving and refusing returns](#)

[Code Trace Challenge — which exits are proven?](#)

[Level 3 — Architecture reasoning](#)

[Why refusal matters](#)

[Why ETC stops at analysis](#)

[Failure Case — the return assumed](#)

[Architect's Warning](#)

[Claim Check](#)

[Current Support](#)

[Current Limitation](#)

[Quick Recall](#)

[Key Takeaways](#)

[Closing Part V — what the special review section has shown](#)

[Forthcoming Chapters](#)

[Part III — Evidence Graph and Dependency Reasoning \(continued\)](#)

[Part IV — COBOL Transformation and the Evidence of Correctness](#)

[Part VI — LLM Interpretation and Agentic Architecture Boundaries](#)

[Part VII — Target Architecture](#)

[Part VIII — End-to-End Modernization Methodology](#)

[References](#)

[IBM documentation](#)

[Cloud and scheduler documentation](#)

[Language and compiler documentation](#)

[Public sample applications measured in this edition](#)

Notices

Edition notice

This is an EARLY REVIEW EDITION and a WORK IN PROGRESS. It is not a final book, a certification, official documentation or vendor training.

Independence

This book is independent, educational and engineering-focused. It is not an official publication of IBM, AWS or any other vendor.

Trademarks and non-endorsement

Product and company names are used only for identification and remain the property of their respective owners. This publication is independent and is not endorsed by IBM, AWS, or other referenced vendors.

The case study is fictional

Quillon Mutual Assurance is an invented company. Its people, jobs, programs, data sets, files and scenarios exist only in this book. They are not intended to represent any real organization or system. Statements about Quillon describe what the invented artifacts contain or what a scenario is designed to exercise. They are not results of running any tool on a real estate.

No certification

The analyses in this book describe what a prototype tool observed in public sample applications, and how that evidence can be reasoned about. They are not a certification of any production migration, and they do not establish behavioural equivalence between any mainframe program and any other program.

Public sample applications

Measurements in this edition were taken over public sample applications at pinned commits. They describe those samples only. Attribution for each is in the Open-Source Attribution section.

Vendor documentation

Statements about platform behaviour are written in the author's own words and cite public vendor documentation in the References. This edition reproduces no vendor tables, figures, screenshots or long quotations. Readers should consult the cited documents for the authoritative wording.

How to Read the Evidence Labels

This book makes claims about a prototype tool, and every claim wears a label. The labels tell you what kind of claim you are reading and how far to trust it. Read this page once, then come back when a label puzzles you.

1. Three levels in every chapter

Level	The reader asks	What you will find
Level 1 — Intuition	What is this, and why does it matter?	A mental model, a scene, one figure
Level 2 — Engineering mechanics	How does it work, exactly?	Rules, artifacts, tool behavior, small traces
Level 3 — Architecture reasoning	What can go wrong, and how do I defend a decision?	Failure modes, trade-offs, limits

Each level builds on the one before it. You can read Level 1 alone for the idea, or skip to Level 3 if you already know the mechanics.

2. Three truths, kept apart

PLATFORM TRUTH	what IBM, AWS or a vendor documents
≠	
ETC IMPLEMENTATION TRUTH	what the prototype's behavior is, shown by tests or measurement
≠	
MIGRATION RECOMMENDATION	what someone should do, justified separately

A sentence carries one of these at a time. When it must carry two, it says how they relate. Chapter 2 develops the idea.

3. Three kinds of factual claim

Every factual statement about ETC is one of **observed fact**, **deterministic derivation** or **interpretation**. Chapter 1 defines them. Where a chapter advances the case study, a small panel sorts its statements into the three columns.

4. Code labels

Every code block carries exactly one label.

Label	Meaning
[PSEUDOCODE – PUBLIC EDITION]	A rule described in generic terms, written for this book. It is not an excerpt of ETC source and not a file
[PSEUDOCODE]	A short generic fragment, such as generic JCL, written for this book
[CASE-STUDY ARTIFACT]	A member of the invented case study
[AUTHOR EXAMPLE]	A small fragment created by the author for an exercise

This edition prints no excerpts of ETC source. Pseudocode shows the conceptual algorithm, the relation between input and output, the order of evidence and the refusal behaviour. It does not show how the prototype is written.

5. Figure labels

Every figure has an ID (FIG-<chapter>-<n>), a title and one provenance label.

Provenance	Meaning
SOURCE-DERIVED	Drawn from a cited external source
AUTHOR-CONCEPTUAL	An explanatory model drawn by the author. It is not a picture of the prototype's internals unless it says which part it simplifies
CORPUS-DERIVED	A count or measurement taken over a named public sample application, with its date

Figures read in grayscale. A solid line means present. A dashed outline or line means absent. A dotted line means withheld or context-only. Every caption ends by saying what the figure does **not** show.

6. The case study and its three layers

Quillon Mutual Assurance is invented. Its people, systems, data sets and programs exist only in this book. Quillon has not been run through ETC, so each statement about it belongs to exactly one layer.

Layer	Meaning	Status
A. Fictional artifact truth	What the invented members contain	Exists
B. Expected pedagogical outcome	What the scenario is designed to exercise	Authored as expectation, always labelled
C. Actual ETC result	What ETC produced when run on the bundle	UNKNOWN

Layer C is marked "Layer C: no case-study run is recorded in this edition". A layer-B statement is never a measured result. It never shares a table, total or figure panel with measurements from the public sample applications, which are the only source of measured counts in this book.

7. Items not established in this edition

Where the evidence a sentence needs is missing, the text says so in words, and the nearby sentence is not established. The gap stays visible rather than being filled with plausible text.

8. Current Support and Current Limitation

A chapter that describes a capability ends with two boxes. **Current Support** lists what the tool does at the baseline, with its status and evidence. **Current Limitation** lists what it does not do or cannot establish. Limitations are stated plainly. They carry no promise of a later fix, and no "not yet".

9. UNKNOWN, UNSUPPORTED, AMBIGUOUS

- **UNKNOWN** means the evidence did not establish the value. It is not NONE, which means the evidence established that nothing is there. UNKNOWN is a result, not an error.
- **UNSUPPORTED** names a construct ETC recognizes but has no rule for.
- **AMBIGUOUS** names a construct whose meaning the evidence cannot fix.

Chapter 17 defines the last two. In every case the book reports the state and its reason. It does not round UNKNOWN up to "probably yes" or down to "none".

10. Why failures and refusals appear on purpose

A tool that reports only success cannot be trusted. This book treats a comparison that fails, a program ETC declines to translate and a value that stays UNKNOWN as first-class results, taught in the chapter where they arise.

11. Numbers, dates and baselines

Every count names its baseline or its measurement date. Historical records keep their original date and say whether they were re-run. Where this book writes "equivalence" or "confidence", it names which kind.

12. Chapter furniture

Element	Purpose
Quick Recall	Three to five questions for self-check
Architecture Question	An open prompt for Level 3
Code Trace Challenge	Follow a small trace and predict its output
Hands-On Exercise	Work on the case study, never on private data
Failure Case	A refusal, UNKNOWN or FAIL, taught as a result
Architect's Warning	The most likely misreading of the chapter
ETC Lesson Learned	An engineering lesson, labelled as such
Claim Check	A sentence you might be tempted to write, and why it fails
Key Takeaways	The chapter in a few lines

Open-Source Attribution

This edition measures, and in places describes, six public repositories. It reproduces no corpus source files. It uses counts, structural descriptions, short identifiers and examples written by the author. Each repository is identified at the commit that was measured.

Corpus	Upstream repository	Attribution	Pinned commit
AWS Mainframe Modernization CardDemo	aws-samples/aws-mainframe-modernization-carddemo	© Amazon Web Services, Inc., repository licensed under Apache License 2.0.	59cc6c2fd7ebd7ef7925cad552a01a4b8b6e4d5e (main)
CICS Banking Sample Application (CBSA)	cicsdev/cics-banking-sample-application-cbsa	© IBM Corporation. This book reproduces no CBSA source. It reports names, counts and structure only.	417334533178ab6e753cc64b0e0e5cf0b4952704 (July2024Refresh)
CICS GenApp	cicsdev/cics-genapp	© IBM Corporation. EPL-2.0, as stated by the project README.	f6f3f4b2580d31b7d8dcc31ce3e3676f4cceaaaa (main)
zblas	Zaneham/zblas	© Zane Hambly, licensed under Apache License 2.0.	9ee3044b48b4c2d26b0793da67aa6acc9c823d06
zkvs	Zaneham/zkvs	© Zane Hambly, licensed under Apache License 2.0.	0c47e5284e395fdb072be39d9d9e9be322047ea1
zos-hlasm-kvstore	netmaster296/zos-hlasm-kvstore	© 2024 netmaster296, licensed under the MIT License.	0b34698b4a018454096e5f117aba987bfc3f8eab

The three assembler validation repositories (zblas, zkvs and zos-hlasm-kvstore) were measured at the pinned commits on their default branch.

Notes

- **AWS Mainframe Modernization CardDemo.** © Amazon Web Services, Inc. The repository is licensed under Apache License 2.0.

- **CICS Banking Sample Application (CBSA).** © IBM Corporation. Repository `cicsdev/cics-banking-sample-application-cbsa`, commit `417334533178ab6e753cc64b0e0e5cf0b4952704` (branch `July2024Refresh`). This book reproduces no CBSA source. It reports names, counts and structure only.
- **CICS GenApp.** © IBM Corporation. Repository `cicsdev/cics-genapp`, commit `f6f3f4b2580d31b7d8dcc31ce3e3676f4cceaaaa`. The EPL-2.0 licence is stated by the project README.
- Product and company names in these projects remain the property of their respective owners. Inclusion here does not imply endorsement by any project, author or vendor.
- This section does not give legal advice and does not assert that any use is cleared.

Chapter 1 — Why Modernization Claims Fail

After this chapter you can:

- name seven ways a modernization claim substitutes something easy to get for something hard to establish;
- sort any factual sentence into observed fact, deterministic derivation or interpretation;
- explain why "UNKNOWN" and "none" are different answers.

Level 1 — Intuition

A question with three questions inside

Quillon Mutual Assurance is an invented property insurer. Its people, systems, data sets and programs exist only in this book. Every night, a billing cycle extracts policies, rates them, posts premiums, prints a report and feeds a summary to Finance. Quillon's CIO asks:

"Which parts of billing can move to AWS first, and how sure are we?"

That sentence hides three separate questions.

1. **What is there?** Which programs, jobs and data sets exist, and how do they connect?
2. **What does it do?** What behavior would a replacement have to preserve?
3. **What should we do about it?** Which pieces move, in what order, onto what?

Answering the second takes evidence the first cannot supply. Answering the third takes judgement the second cannot supply. Most modernization claims fail because one of these questions is answered with the answer to an easier one.

A note on the case study. Quillon has not been run through ETC, the tool this book examines. Each statement about Quillon belongs to one of three layers. This chapter uses the first throughout, and the second once, in FIG-01-1, where the example is labelled as an expected derivation and not an ETC result.

Layer	Meaning	Status
A. Fictional artifact truth	What the invented members contain	Exists
B. Expected pedagogical outcome	What the scenario is designed to exercise	Authored as expectation only
C. Actual ETC result	What ETC produced on the bundle	UNKNOWN (Layer C: no case-study run is recorded in this edition)

What Quillon's bundle contains

[CASE-STUDY ARTIFACT] — layer A: the invented members, by kind.

Kind	Members
Scheduler export	QMANIGHT.xml
Batch jobs	QMABL010, QMABL020, QMABL030, QMABL040, QMAFN100, QMAPV001
Cataloged procedures	QMARATE, QMASORT
COBOL programs	QMAEXT01, QMARTE01, QMAPST01, QMARPT01, QMAFN101, QMAINQ01, QMAVALID
Copybook	QMAPOLCP
Screen map	QMAMAP1
Assembler module	QMADTCV
Named but not in the bundle	QMACL900 (a claims job), QMALOG (a logging routine)

This list is an inventory. It is useful, and it is also where understanding tends to stop. Later chapters ask what it leaves out.

Seven confusions

Each confusion below swaps an artifact that is cheap to obtain for the thing it stands for. Plain examples come first, and none involves ETC.

#	The confusion	A plain example	What is actually established
1	Inventory mistaken for understanding	A parts list names every bolt in an engine. It does not say which parts move together.	That the parts exist
2	Syntax mistaken for semantics	A field named <code>AMOUNT</code> is stored as a packed number with two decimals. The layout is syntax. Whether the amount is before or after tax is meaning.	The format of the data
3	Generated code mistaken for transformed behavior	A machine translation of a recipe yields fluent sentences. Whether the cake rises is another question.	That text was produced
4	Compile success mistaken for equivalence	A spelling checker passes your letter. It says nothing about whether the letter says what you meant.	That the text is well formed under the language's rules
5	Technical grouping mistaken for application architecture	Two shops that share a loading dock are coupled in operations. They are not one business.	A coupling
6	Cloud artifact generation mistaken for deployment	An architect's drawing of a house is not a house.	A description of a target
7	Model-generated prose mistaken for evidence	A fluent summary of a contract is not a reading of the contract. It may cite clauses nobody checked.	That a model produced text

Later chapters return to each: classification in Chapter 8, grouping in Chapters 13 and 16, generated code and compilation in Chapters 19 and 20, models in Chapters 25 and 26, and targets in Chapter 27.

Two Quillon scenes

Scene 1: a name that misleads. The directory listing places `QMAVALID.asm` among the assembler members.

```
[CASE-STUDY ARTIFACT]  asm/QMAVALID.asm  (opening lines)
    IDENTIFICATION DIVISION.
    PROGRAM-ID. QMAVALID.
```

Those two lines open a COBOL program, not an assembler module. A reader who trusts the inventory files it under the wrong language. A reader who opens the member sees the difference. The directory and the first lines are two separate sources, and they disagree. Chapter 8 returns to this.

Scene 2: a belief. Quillon's operations team calls the nightly cycle "the billing application", and the cycle ends with a feed to Finance, which owns the last job and its program. Whether billing and finance form one application or two is a business question. No artifact in the bundle answers it. This book records the belief now, as an interpretation, and tests it in Chapter 16.

Level 2 — Engineering mechanics

Three kinds of factual claim

Every factual sentence about a tool belongs to one of three classes.

Class	Meaning	What you should see next to it
Observed fact	Read directly from an artifact, with a location	The artifact, test or measurement
Deterministic derivation	Produced from observed facts by a named, repeatable rule	The rule and its inputs
Interpretation	A judgement beyond what a rule derives: author reasoning, a reviewer's call or a model's proposal	Words such as "suggests" or "a reasonable reading is"

An **observed fact** answers "what does the artifact say?" A derivation answers "what follows, by a rule we can name and repeat?" An interpretation answers "what do we think it means?" The three are all useful. They are not interchangeable.

FIG-01-1 — Observed, derived, interpreted · AUTHOR-CONCEPTUAL

<pre> +-----+ OBSERVED read directly from an artifact QMARATE, DD RATED: DISP=(NEW,CATLG,DELETE) cite: the artifact and the location +-----+ named rule: status NEW -> WRITE v +-----+ DERIVED computed by a named, repeatable rule EXPECTED DERIVATION (not an ETC result): access to RATED = WRITE cite: the rule and its inputs +-----+ the rule alone is not enough v +-----+ INTERPRETED a judgement beyond the rule "The rating program writes this file." needs separate evidence: the program's OPEN mode +-----+ </pre>	
--	--

Caption. A DD statement is a job-control line that names a data set a step uses; Chapter 5 teaches JCL. The observed band is layer A. The derived band applies a documented ETC rule by hand to a fictional statement; it is an expected derivation, not an ETC result (Layer C: no case-study run is recorded in this edition). The figure does not imply that a derived fact is less true than an observed one, or that interpretation is worthless; it shows that each needs its own support.

The status **NEW** inside the disposition is observed text. "Access mode WRITE" is derived, because a rule maps the status to a direction. "The program writes this file" is an interpretation, because a disposition governs how a data set is allocated, not what a program does with it. Chapter 2 develops that distinction from IBM's documentation.

Evidence

In this book, **evidence** is an observed fact or a deterministic derivation that carries its provenance: which artifact, which reader, which location, and on what basis. A model's output is not evidence. A test that has not been run is not evidence. Interpretation can be well reasoned and still not be evidence.

UNKNOWN is not NONE

UNKNOWN means the available evidence did not establish the value. **NONE** means the evidence established that nothing is there. They look alike on a report and mean opposite things.

FIG-01-2 — NONE and UNKNOWN · AUTHOR-CONCEPTUAL

NONE known absence	UNKNOWN not established
+-----+	+-----+
The whole step card was	Hypothetical: the bundle
read. Step REPORT in	shipped without a
QMABL040 has no COND	scheduler export. Each
parameter.	job's predecessors are
	not established.
+-----+	+-----+
Report it as: none	Report it as: UNKNOWN
+-----+	+-----+

Caption. **COND** is a job-control parameter that tests earlier steps' return codes to decide whether a step is skipped; Chapter 4 explains it. Both examples are layer A about invented artifacts, and the right-hand one is a hypothetical. The figure does not imply that UNKNOWN is an error state. It is a correct answer when the evidence is silent. Missing evidence is not evidence of absence.

ETC provides one example of the same distinction. When a data set statement carries no disposition at all, ETC records the status as UNSPECIFIED and the access mode as UNKNOWN, issues a warning, and excludes the store from data flows. Chapter 7 shows exactly what it does and what it does not.

Duplicate evidence is not corroboration

Suppose two tools read the same file. You now hold two readings of **one** artifact. If they agree, you have learned that the tools agree. You have not learned that the file is right. **Corroboration** needs more than one distinct artifact, such as a scheduler declaration and a job's data hand-off that point to the same ordering. Two readings of one source are **duplicate evidence**, and they do not strengthen a claim. Chapter 12 turns this principle into a rule.

Refusal over guessing

When the evidence cannot support an output, an evidence-bounded tool says so and stops. That is a **refusal**. A refusal is a result, not a defect. It tells you exactly where human judgement is still required. Chapter 17 defines the kinds of refusal ETC records.

Quillon, sorted

The panel below sorts the statements Chapter 1 has made about Quillon. Nothing has been run through ETC, so the middle column holds the one expected derivation from FIG-01-1 and no ETC result.

Observed (layer A)	Expected derivation (not an ETC result)	Interpreted
The CIO's question exists. The bundle contains the members listed above. <code>asm/QMAVALID.asm</code> opens with <code>IDENTIFICATION DIVISION</code> .	Access to <code>RATED</code> = WRITE, from the status <code>NEW</code> by the disposition rule, applied by hand by the author (layer B, FIG-01-1). Not an ETC result.	"Billing is one application."

Level 3 — Architecture reasoning

The pattern behind the seven confusions

Each confusion has the same shape: a deliverable that is cheap to produce is reported in place of the thing it represents. Lists are cheap, and understanding is not. Generated text is cheap, and preserved behavior is not. A successful build is cheap, and a verified migration is not.

The cost moves later. A wrong grouping appears at cut-over planning. A false sense of equivalence appears in production. An unchecked model summary appears when someone relies on it in a design review. The earlier the substitution, the more decisions are built on it.

Evidence-bounded modernization

This book uses three working rules, and tests one tool against them.

1. **Every claim says what it rests on:** observation, rule, or interpretation.
2. **Missing evidence is reported as UNKNOWN**, never rounded to yes or none.
3. **When evidence cannot support an output, the tool refuses and says why.**

The price is slower progress and more answers of the form "this is not established". The return is that each decision can be defended by pointing at the evidence it stands on.

Architect's Warning

Be suspicious of progress measured in artifacts: files read, programs generated, templates produced, diagrams drawn. Each count can rise while understanding stays flat. Ask what each artifact lets you conclude, and stop at that conclusion.

Claim Check

"The tool listed every member of the estate, so it understands the estate."

This fails for two reasons. A list of members is an inventory, and it records nothing about how members connect (confusion 1). A list built from file locations can also misfile a member, as `QMAVALID.asm` shows. Understanding needs relationships, which are derived, and meaning, which is interpreted.

Quick Recall

1. Name the three classes of factual claim and give one example of each from a data set card.
2. What is the difference between UNKNOWN and NONE?
3. Why are two readings of one file not corroboration?
4. Which of the seven confusions is "it compiles, so it works"?
5. Is "billing is one application" observed, derived or interpreted? Why?

Key Takeaways

- A modernization claim usually substitutes something easy to get for something hard to establish.
- Observed fact, deterministic derivation and interpretation are different classes. Do not merge them.
- UNKNOWN is a result. It is not NONE, and it is not an error.
- Duplicate evidence is not corroboration.
- When evidence cannot support an output, refusal beats guessing.
- Quillon is invented, and nothing in this chapter is a Quillon result from ETC.

Chapter 2 — Three Truths: Platform, Implementation, Recommendation

After this chapter you can:

- say which of three kinds of truth a sentence carries, and which source can support it;
- split a mixed sentence into its platform, implementation and recommendation parts;
- apply the split to seven worked examples, from JCL to AWS.

Level 1 — Intuition

The speed limit, the speedometer and the satnav

A road has a legal speed limit. Your car has a speedometer. Your satnav suggests a route. Each can be right while the others are wrong. The law tells you what the road requires, and says nothing about how fast you are going. The speedometer tells you your speed, and says nothing about the law. The satnav recommends an exit, and neither the law nor the speedometer justifies it.

Modernization has the same three sources of truth.

FIG-02-1 — Three truths · AUTHOR-CONCEPTUAL

+-----+		+-----+		+-----+
PLATFORM TRUTH		ETC IMPLEMENTATION		MIGRATION
	≠	TRUTH	≠	RECOMMENDATION
what IBM, AWS or a		what ETC's code does		what someone
vendor documents		on stated inputs		should do
authority:		authority:		authority:
cited public source		code + tests +		a separate
		measurement		justification
+-----+		+-----+		+-----+

Caption. Each box has its own authority. This figure does not depict a pipeline in which one box feeds the next: knowing what a platform documents does not tell you what ETC does, and knowing what ETC does does not tell you what to migrate.

Two terms we will use throughout:

- A **result** is an output ETC produced, together with its evidence and limitations. A result is not a recommendation.
- A **recommendation** is a proposal for what to do, with its own justification. A recommendation is not a capability of ETC.

Where the Chapter 1 belief sits

In Chapter 1, the operations team of Quillon, the book's invented insurer, called the nightly cycle "the billing application". Sort that sentence. No platform source says it, and no ETC result supports it, because nothing has been run. It is an **interpretation**, and it stays one until evidence and an owner's decision say otherwise.

Level 2 — Engineering mechanics

Authority depends on the question

No single ladder of authority covers all three questions. Each truth has its own authority.

The question	What governs the answer	What does not
What does the platform mean? (JCL, COBOL, HLASM, a scheduler, Java, an AWS service)	The cited platform or vendor documentation	ETC's code, tests and measurements; ETC's behavior establishes nothing about the platform
What does ETC currently do?	ETC's code at the baseline, its tests and reproducible measurements, and the author's reviews made for this book	External documentation; it establishes nothing about ETC
What should this workload migrate to?	A separate engineering recommendation, justified from evidence and constraints, and labelled as interpretation	Platform documentation alone, or ETC's behavior alone

Only the second question has an internal order. For claims about what ETC does, a lower rank never overrules a higher one. It can only add context.

Rank	Source	What it can establish
1	ETC's code at the baseline	What the tool does
2	ETC's tests and reproducible measurements	That behavior holds on stated inputs
3	The author's reviews made for this book	The reviewed reading of ranks 1 and 2, with a stated level of verification

A code comment, a design note or a product document is evidence of what someone wrote. Where it conflicts with ETC's behavior, behavior wins. External documentation never establishes ETC's behavior, and ETC's behavior never establishes that a migration is suitable. Interpretation, whether an explanation, an analogy or a recommendation, establishes nothing on its own and must be labelled.

How a sentence is built

Each sentence carries one truth. When a point needs all three, write three sentences:

Platform: the source documents X. *ETC*: the tool does Y on these inputs. *Recommendation*: we would do Z, because W.

Seven worked examples

Each example states what the platform or source establishes, what ETC currently establishes, and what remains interpretation or recommendation.

1. JCL DISP

```
[CASE-STUDY ARTIFACT] QMARATE (two DD statements)
//POLMAST DD DSN=&HLQ..POLICY.MASTER,DISP=SHR
//AUDIT   DD DSN=&HLQ..BILL.AUDIT
```

Truth	Statement	Basis
Platform	DISP states a data set's status and what happens to it when the step ends. If the whole DISP is omitted, the system uses (NEW,DELETE,DELETE) . DISP=SHR does not guarantee that the program only reads.	[B1]
ETC	Access direction in the graph evidence comes from the disposition and documented utility and role rules, for example SHR gives READ and NEW gives WRITE. When DISP is absent, ETC records the status as UNSPECIFIED and the access mode as UNKNOWN, warns, and keeps the store out of data flows. IBM documents a platform default, and ETC's behavior is more conservative than applying it. The reason for this behavior is not recorded.	Reviewed at the baseline
Recommendation	AUDIT has no DISP (layer A). On the platform, the default would delete the file when the step ends. A reasonable reading is a latent defect for the owner to confirm.	Interpretation

2. COBOL OPEN

```
[CASE-STUDY ARTIFACT] QMARTE01 (two statements)
  OPEN I-O POLICY-FILE.
  OPEN EXTEND AUDIT-FILE.
```

Truth	Statement	Basis
Platform	The <code>OPEN</code> mode (<code>INPUT</code> , <code>OUTPUT</code> , <code>I-O</code> , <code>EXTEND</code>) decides which I/O statements the program may use. The file's <code>ASSIGN</code> name corresponds to a DD name. The documentation does not make <code>DISP</code> control the permitted <code>OPEN</code> mode.	[B3]
ETC	The direction in the graph evidence is derived from the disposition and is not merged with the <code>OPEN</code> mode. It is therefore derived from JCL, and is not proven program I/O. Another consumer in ETC reads <code>OPEN</code> modes separately, and this book does not describe it.	Reviewed at the baseline
Recommendation	<code>POLMAST</code> is <code>SHR</code> in the JCL, while <code>QMARTE01</code> opens it <code>I-O</code> (layer A). The <code>SHR</code> may be a leftover. Which side to trust when designing a target store is a decision that needs both.	Interpretation

3. ExecutionSlice

Truth	Statement	Basis
Platform	The platform documentation this book cites describes jobs, schedules and data sets. It defines no grouping called a slice. Chapter 4 covers schedulers and data sets.	Chapter 4
ETC	An ExecutionSlice is a technical grouping of jobs connected by scheduled or data orderings. It is not an application, deployment unit or migration wave. Over CardDemo at the baseline, ETC's slice discovery yields 26 slices, matching the pinned regression fixture.	the rule has stated exceptions, and Chapter 13 gives it
Recommendation	Whether a slice is a unit of migration, ownership or deployment is a separate decision, with its own justification. A slice is a candidate for review by the people who own the jobs.	Interpretation

Quillon. "Billing is one application" is the belief from Chapter 1 and stays an interpretation.

4. Java compilation

Truth	Statement	Basis
Platform	The Java SE 21 specification defines which programs are valid Java. Compilation under a conforming compiler shows that the source is syntactically valid and type correct. It does not show that the program matches any COBOL source.	[B14]
ETC	At the baseline, ETC emitted Java for 36 of the 105 COBOL translation members in the three reference estates and refused the other 69 whole (Chapter 8 defines the term). All 36 emitted programs compiled in a local run under JDK 21 against the runtime built from source (2026-10-04). That is compile evidence only.	Reviewed at the baseline
Recommendation	Whether the Java is correct, and whether it is fit for production, needs evidence beyond compilation. Chapter 20 shows what exists.	Interpretation

5. GnuCOBOL-based historical comparison

Truth	Statement	Basis
Platform	GnuCOBOL is an independent open-source COBOL compiler. It is not an official or reference implementation of IBM Enterprise COBOL. Its <code>-std=ibm</code> mode establishes dialect-support behavior only, not IBM runtime identity. Documented differences include character encoding, floating-point representation, and the CICS and VSAM runtimes.	[B13]
ETC	In a run recorded on 2026-09-20, five of CardDemo's 44 COBOL programs produced byte-identical stdout, condition code and written files compared with a GnuCOBOL 3.1.2 build of the unmodified source, configured for IBM-oriented dialect compatibility. These are historical PASS records, not re-run at this baseline and not automated.	Reviewed at the baseline
Recommendation	Five historical agreements say nothing about the programs not tested, and production z/OS equivalence is absent from this edition's evidence. Whether they support any cut-over decision is a judgement, not a result.	Interpretation

CBEXPORT is a separate case. Compared field by field with a committed file, and not with GnuCOBOL output, its verdict is **FAIL**: 446 of 500 records equal. That record dates from 2026-09-19 and was not re-run at this baseline. It is never counted with the five. Chapter 20 takes it apart.

6. HLASM linkage convention

Truth	Statement	Basis
Platform	By the standard OS linkage convention, register 13 points to the caller's save area, register 14 holds the return address, and register 15 holds the entry address and then the return code. The architecture defines what BR 14 does: it branches to the address in register 14. It has no concept of a caller or a return. A convention does not prove that every BR 14 is a return.	[B6], [B7]
ETC	ETC does not conclude a return from the convention alone. It proves a return only through a deterministic chain: register provenance from the call, a save and restore interval, and the reaching definition. On the independent validation corpus at the baseline, it proves four returns, all BR reached by LM, and no register branches. No estate return evidence is observed. Assembler is read and analyzed, not translated.	Reviewed at the baseline
Recommendation	Whether an assembler module should be kept, wrapped or rewritten is a decision ETC does not make.	Interpretation

7. AWS service capability

Truth	Statement	Basis
Platform	An AWS Batch job definition for Fargate must specify the Fargate platform capability, the container type, and a valid vCPU and memory combination. Multi-node parallel jobs, GPU resources and privileged mode are not supported on Fargate. The documentation does not say a mainframe job stream should move to AWS Batch, and it provides no condition-code evaluation or data-set disposition handling.	[B10]
ETC	The target output this edition supports is translated Java (above). Production AWS deployment is outside this edition's evidence. One hand-run AWS Batch demonstration is recorded as a transcription, not a live-run record. Other target outputs are not described here.	Reviewed at the baseline
Recommendation	Which target suits a Quillon job is a recommendation of the form "for this job, a reasonable target is ..., because ...". The AWS documentation does not supply the reason.	Interpretation; Chapter 27

Level 3 — Architecture reasoning

Three inferences that look valid and are not

Inference	Tempting sentence	Why it fails
Platform to implementation	"IBM documents a default for a missing <code>DISP</code> , so ETC applies it."	ETC records UNKNOWN and keeps the store out of data flows. Read the tool, not the manual.
Implementation to recommendation	"ETC compiled 36 programs, so they are ready to migrate."	Compilation is neither behavior nor suitability, and production evidence is absent.
Implementation to platform	"ETC derives READ from <code>SHR</code> , so IBM defines <code>SHR</code> as read-only."	ETC applies its own rule. IBM's documentation does not make <code>SHR</code> read-only for the program.

A reader who spots these three can audit most modernization material.

Architect's Warning

The commonest slip is citing a source for the wrong truth: an IBM manual as proof of what ETC does, or ETC's output as proof that a migration is sound. When a sentence names a source, ask which of the three truths that source can support, and stop there.

Claim Check

"ETC marks the store READ, so the program never writes it."

This fails. The READ is derived from the disposition by ETC's rule. It is not program I/O, and IBM's documentation does not make `SHR` read-only for the program. In Quillon, `QMARTE01` opens `POLMAST` I-O (layer A).

Quick Recall

1. What can an IBM or AWS manual establish about ETC's behavior?
2. A design note says a component works one way, but the code works another way. Which wins, and why?
3. Write the three-part statement for a data set card with no `DISP`.
4. Why is a recommendation not a capability of the tool?
5. Which two qualifiers must accompany the five GnuCOBOL-based PASS records?

Key Takeaways

- Platform truth, implementation truth and recommendation have different authorities. None substitutes for another.
- A sentence carries one kind of truth. Split mixed sentences.
- ETC's `DISP`-derived READ and WRITE are derivations, not proven program I/O.
- Compilation, historical comparison and AWS documentation each establish less than they appear to.
- Quillon's "billing is one application" is an interpretation.

Chapter 3 — ETC in One Picture

After this chapter you can:

- name the stages between a legacy mainframe artifact and the evidence ETC can offer;
- say what is implemented, partial or absent at the baseline;
- explain how reading, grouping, transforming and verifying are kept apart, and why code generation does not use the EvidenceGraph.

This chapter names things so later chapters can open them. It gives no mechanics.

Level 1 — Intuition

Read the estate before redesigning it

Picture a library that has received a crate of old ledgers.

1. A team **reads** the ledgers and writes careful notes, each note pointing back to the page it came from.
2. The notes are **structured** into an index of *what ran with what*, so that every entry can be traced to the notes behind it.
3. The index is used to **group** the ledgers that belong together for examination.
4. Elsewhere in the building, a translator works on one ledger at a time, from the notes and not from the index. This is the **transform** step.
5. Every translation is then **verified**, in separate steps: does it compile, and does it behave the same as the original?

ETC has this shape: READ, then STRUCTURE EVIDENCE, then GROUP, then TRANSFORM, then VERIFY. A reader who pictures it as one pipeline in which a single graph feeds everything will misjudge what each part can claim.

Two words to fix first

- An **artifact** is a file in an evidence bundle: a JCL member, a procedure, COBOL source, a copybook, assembler source, a scheduler export or a screen map. An artifact may contain several programs or jobs, or none, and its directory does not decide what it is.
- A **capability** is something ETC's code can do at the baseline, with a status. It is not a result on a particular estate, and it is not something an AWS service can do.

Status words in this book are IMPLEMENTED, PARTIAL, DOCUMENTED-ONLY, UNKNOWN and ABSENT.

Three names to remember

- The **EvidenceGraph** is the typed graph of evidence-backed relationships that ETC builds from the supplied artifacts.
- An **ExecutionSlice** is a technical grouping of jobs. It is not an application, deployment unit or migration wave.
- An **estate boundary** marks the point where the supplied evidence stops. Here the *estate* is the supplied evidence set. It is not a business domain, an application boundary or a deployment boundary.

Chapters 10 and 13 explain these. For now, treat the names as labels.

Quillon. The fictional estate is designed so that later chapters can exercise each stage in this chapter. No ETC run on it is recorded (Layer C: no case-study run is recorded in this edition).

Level 2 — Engineering mechanics

The stages, and what stands behind each

The chain below is a reading order, not a program. Each row has its own evidence and its own limits.

Stage	What exists at the baseline	Source
1. Legacy artifacts	JCL, cataloged procedures, COBOL, copybooks, scheduler exports and assembler source, supplied as an evidence bundle.	Reviewed at the baseline
2. Deterministic parsing	ETC reads native JSON, Control-M XML and CA-7 LJOB schedule exports, from exported definitions. Measured over the three reference estates (CardDemo, CBSA, GenApp) at the baseline: 105 COBOL translation members (Chapter 8 defines the term).	Reviewed at the baseline
3. Normalized evidence and reading quality	The readers produce typed, normalized evidence together with a reading-quality record. The record carries the absences that leave no edge behind (Chapters 10 and 11).	Reviewed at the baseline
4. Typed EvidenceGraph	IMPLEMENTED: the EvidenceGraph is built by scripts only. It holds scheduler order, data order, calls, copybook and layout evidence and estate boundaries, each with its supports (Chapters 10 to 12).	Reviewed at the baseline
5. Slices and technical-boundary confidence	An execution slice is a technical grouping of jobs connected by scheduled or data orderings. It is not an application, deployment unit or migration wave. Confidence here means <i>technical-boundary</i> confidence, with a written basis. This chapter states no score.	Reviewed at the baseline
6. Optional bounded interpretation	Optional and off by default. Model output is a separately typed, citation-checked proposal: citations and figures are checked, and semantic entailment is not. Deterministic results are computed first and cannot be written by the model.	Reviewed at the baseline
7. Transformation eligibility and refusal	Each COBOL translation member is judged as a whole: eligible for emission, or refused whole with a stated reason.	Reviewed at the baseline
8. COBOL to Java emission	At the baseline the emitter translated 36 of the 105 COBOL translation members in the three reference estates and refused the other 69 whole. Each refusal is recorded with a stated reason.	Reviewed at the baseline
9. Compile evidence	All 36 emitted programs compiled in a local run at the baseline, against the runtime built from source. That is compile evidence only.	Reviewed at the baseline
10. Behavioral and equivalence evidence	Five CardDemo PASS records dated 2026-09-20, not re-run at this baseline and not automated. CBEXPORT is a separate comparison with verdict FAIL (446 of 500 records equal), recorded 2026-09-19 and not re-run at this baseline. Production z/OS equivalence is outside this edition's evidence.	Reviewed at the baseline
11. Bounded target artifacts	Production AWS deployment is outside this edition's evidence. One hand-run AWS Batch demonstration is recorded as a transcription, not a live-run record. Other target outputs are not described here.	Reviewed at the baseline

Three points protect the reading.

Stage 6 is a separate quantity. A model-assisted interpretation of what a slice means to the business is not the same measurement as technical-boundary confidence. It enters only after the deterministic evidence exists. It cannot create graph truth or change deterministic confidence. Later chapters keep the two apart.

Stages 8 to 10 are three different claims. Emitting Java is not compiling it, and compiling it is not comparing its behavior. Chapter 19 and Chapter 20 open them.

Absence is stated, not implied. Assembler is read and analyzed, not translated. Production AWS deployment is outside this edition's evidence, and one hand-run AWS Batch demonstration is recorded.

A worked reading of one row

Take stage 8: "36 of the 105 translation members emitted, 69 refused whole."

- *Population.* The 105 are COBOL translation members in the three reference estates (Chapter 8 defines the translation population). They say nothing about the programs in any other portfolio.
- *Unit.* Each member is emitted or refused as a whole. The row says nothing about partial translation.
- *Silence.* The row does not say the 36 behave like their sources. Stages 9 and 10 are the places to look, and they carry their own qualifiers.
- *Reason.* Each refusal is recorded with a stated reason, so a refusal is a recorded result and not a gap in the data.

A sentence that cannot be read this way is carrying more than the evidence holds.

Where the deterministic core stops

The project describes a layered layout with a deterministic domain core, in which model-assisted interpretation is kept apart from the components that compute facts. Chapter 25 returns to the model boundary.

Two figures: understanding, then transforming

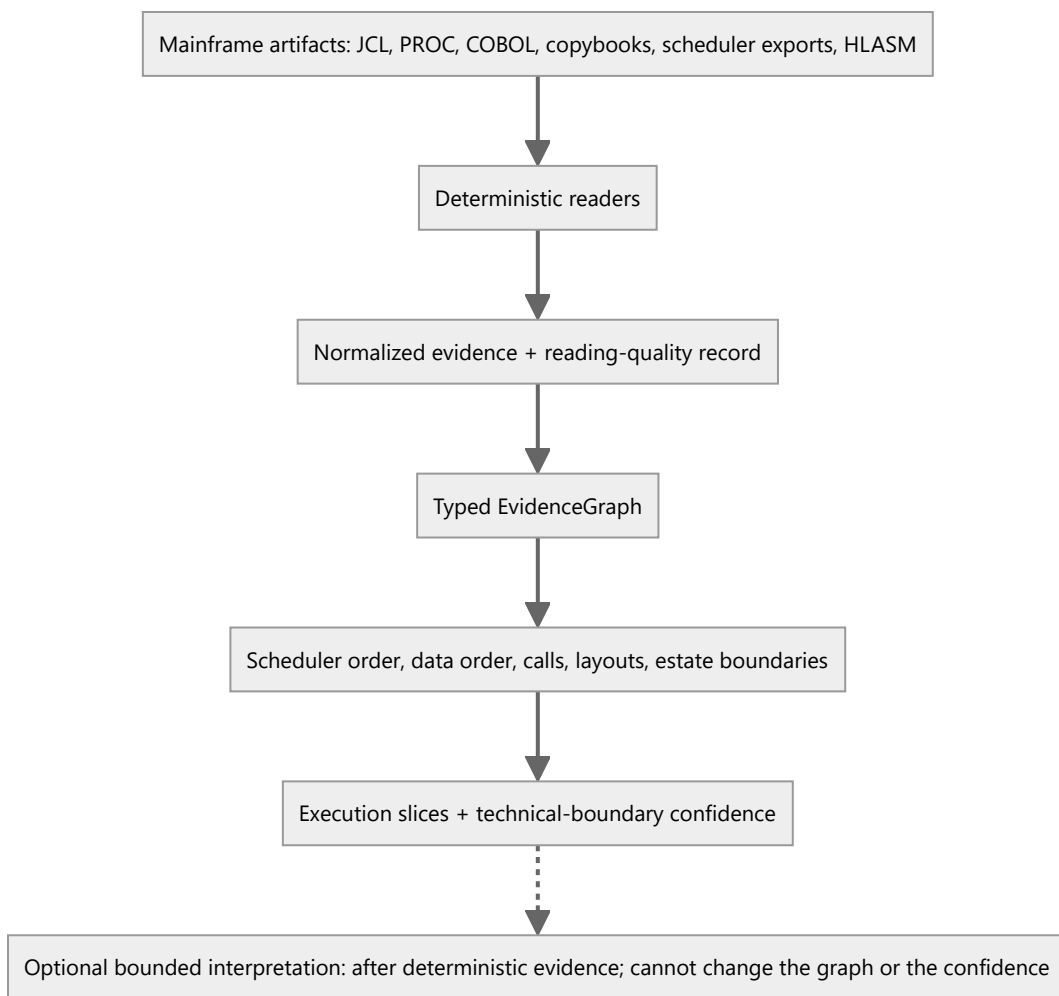
The pipeline is drawn as two figures so each can be read at print size. The first covers how ETC understands a mainframe estate. The second covers how it transforms and verifies one program. Arrows show reading order. They do not mean that each step proves the next.

Here is the conceptual shape of both, in generic names:

```
[PSEUDOCODE] understand, then transform (conceptual)
reading  = read_artifacts(bundle)
evidence = build_evidence_view(reading)
slices   = group_jobs(evidence)
# code generation takes the COBOL reading, not the 'evidence' graph
java     = emit_or_refuse(cobol_reading_of(reading))
```

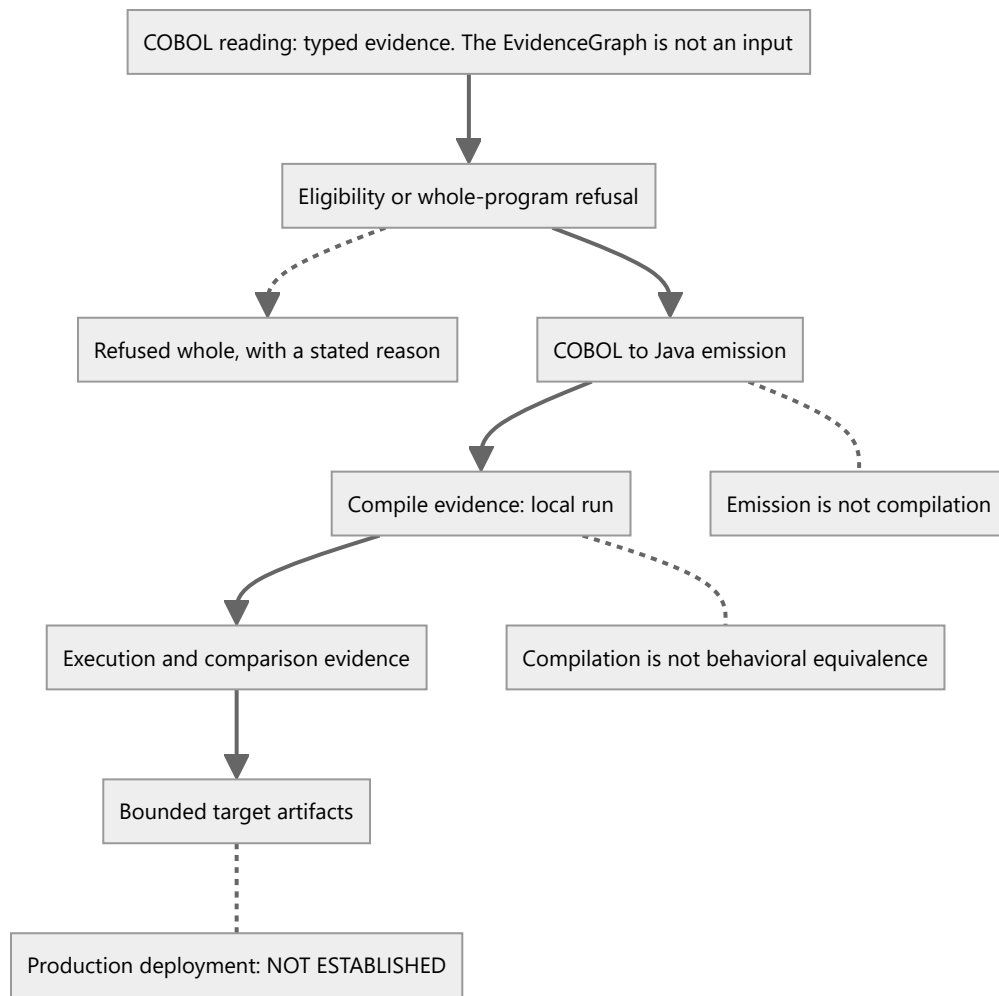
The block restates this design in generic names and is not an excerpt of ETC source.

FIG-03-1A — How ETC understands a mainframe estate · AUTHOR-CONCEPTUAL

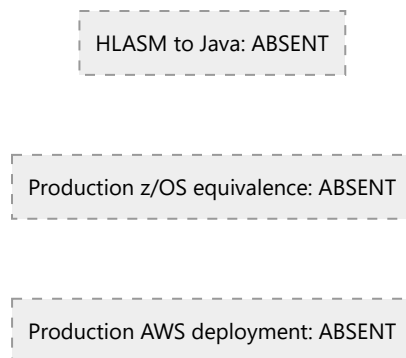


Caption. Solid arrows are deterministic steps. The dotted arrow is an optional side path that begins only after the deterministic evidence exists. It adds a separately typed proposal and never writes the graph. The figure does not show any step in the transformation path (FIG-03-1B). Provenance is author-conceptual.

FIG-03-1B — How ETC transforms and verifies a program · AUTHOR-CONCEPTUAL



Panel B: what is absent from this edition's evidence. Dashed means not present.



Caption. Solid boxes are present at the baseline, and the comparison step holds only historical records: a GnuCOBOL-based comparison with 5 PASS records dated 2026-09-20 and a committed-file comparison, CBEXPORT, with FAIL (446 of 500 records equal) dated 2026-09-19, neither re-run at this baseline. The two comparison tiers are separate and are never counted together. Dotted lines mark refusals and claims that do not follow. The emission box takes its input from the COBOL reading, not from the raw artifacts and not from the EvidenceGraph. Compile evidence follows emission. Comparison evidence does not: it needs a separate comparison between an execution of an emitted program and a reference, and it does not follow from generating or compiling the Java. Generation, compilation and comparison are three separate claims. The figure does not show the form that passes between the reading and the emitter (Chapter 19), does not show how any comparison was run, and does not show the existence of any target output beyond the one hand-run demonstration recorded in stage 11. Provenance is author-conceptual.

Level 3 — Architecture reasoning

Reading the picture as an architect

The picture separates questions that modernization material usually blends.

- *What is in the estate?* Parsing, normalized evidence and the reading-quality record.
- *How do the pieces relate?* The EvidenceGraph: typed, evidence-backed relationships, each with its supports. It records shape and provenance. It does not prove business meaning.
- *What should be grouped?* Slices: a technical grouping, nothing more.
- *Can the code be translated?* Emission or refusal, per program.
- *Does it compile? Does it behave the same?* Two further, separate pieces of evidence.
- *What should we do next?* Any decision about target architecture or order of work needs its own justification. The EvidenceGraph and the slices are inputs to that judgement. They are not the judgement.

A project that answers one question and quietly claims the next has crossed a boundary the architecture itself keeps.

Claim Check

"ETC builds a dependency graph and uses it to group jobs and generate code."

This fails three ways. "The dependency graph" is unqualified: an EvidenceGraph edge records an evidence-backed relationship, and it is not automatically a business dependency. Grouping jobs produces slices, which are technical groupings and not applications. Code generation does not

read the EvidenceGraph. It takes the COBOL reading.

EvidenceGraph is not business architecture

The EvidenceGraph records what the supplied evidence says. It does not state the application structure, the business domains or the deployment units of the estate. Chapter 10 teaches what an edge does and does not mean.

ETC Lesson Learned

Keeping the evidence graph, the code-generation path and the verification evidence separate makes each one's claims narrower and easier to check. The cost is that every statement must name which of them it means.

Current Support

- Reading native JSON, Control-M XML and CA-7 LJOB schedule exports, from exported definitions.
- Slices and technical-boundary confidence, on the EvidenceGraph.
- Emission or whole-program refusal of COBOL as Java: 36 of the 105 COBOL translation members in the three reference estates emitted at the baseline, and the other 69 refused whole.

Current Limitation

- No HLASM-to-Java translation.
- No production z/OS equivalence and no production AWS deployment in this edition's evidence.
- This edition describes no assignment of slices to migration waves.
- The reading at the baseline records no CICS transaction graph.

Quick Recall

1. What does the EvidenceGraph establish, and what does it not?
2. What is an ExecutionSlice?
3. Does ETC's code generation consume the EvidenceGraph?
4. Why are emission, compilation and behavioral comparison three different claims?
5. Where may bounded model interpretation enter, and what can it not do?

Architecture Question

A colleague says, "Our plan is to take ETC's slices and make them our migration waves." What evidence would you ask for before accepting the plan, and what does the EvidenceGraph say, and not say, about it?

Key Takeaways

- ETC reads, structures the evidence, groups, transforms and verifies. Each step has its own claim.
- The EvidenceGraph is evidence-backed structure. It is not a business architecture.
- Code generation does not use the EvidenceGraph.
- Emission, compilation and behavior comparison are three claims, each with its own limits.
- Absent capabilities are drawn as absent.
- Every graph and every confidence figure in this book names which kind it is.

Chapter 4 — How a Batch Night Runs

After this chapter you can:

- narrate a batch night as jobs, steps and data sets, and say which order comes from a scheduler and which from the data;
- read a `COND` parameter as a test that bypasses a step, and say what the system does when no test applies;
- explain why a return code is a program's own signal and not a verdict on the business outcome.

This chapter teaches the platform and narrates Quillon's artifacts. It describes no ETC behavior; Chapter 11 shows what ETC reads from a scheduler export.

Level 1 — Intuition

A night shift with a wall plan

Picture a bakery's night shift. A plan on the wall says pastry starts after dough. The crews also pass trays: dough to pastry, pastry to packing. A finished crew writes a number on the tray slip and decides what it means.

The wall plan and the trays are two different things. The plan may make packing wait for dough, though packing never touches dough, and trays may flow from pastry to packing though the plan never says so. A batch night has the same parts.

Bakery	Mainframe term	What it is
The plan on the wall	Scheduler definition	What must happen before a job is submitted
A crew	Job	A unit of batch work submitted to the system
One checklist item	Step	One program run inside a job, ordinarily in the order coded
The trays	Data sets	Files a step reads or writes; what jobs hand to each other
The number on the slip	Return code	A number a step leaves behind; the program decides what it means
A crew that is stopped mid-task	Abend	Abnormal termination of a step

Quillon's night

Quillon Mutual Assurance is the book's invented insurer (Chapter 1). Its nightly cycle begins when the claims system finishes. `QMACL900`, the claims job, is named by the scheduler but is not in the bundle, so what it does is unknown. The billing jobs then extract, rate and post policies, print a report and hand a summary to Finance. A sixth loads a rate table on request.

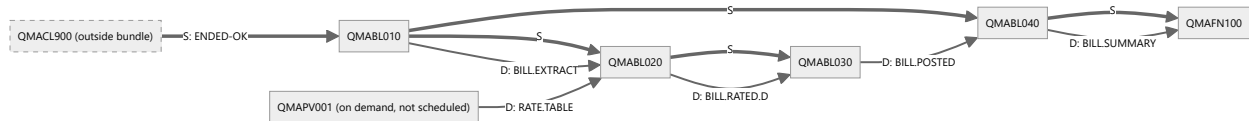
Everything about Quillon here is layer A: what the invented members say. "The night" is the designed scenario. No Quillon run log exists, so no return code, time or duration below happened.

Job	Designed role	Scheduler declares it after	Data set it hands on
<code>QMACL900</code>	Claims cycle (outside bundle)	—	—
<code>QMABL010</code>	Extract in-force policies	<code>QMACL900</code> ended OK	<code>BILL.EXTRACT</code>
<code>QMABL020</code>	Rate policies	<code>QMABL010</code>	<code>BILL.RATED.D</code>
<code>QMABL030</code>	Post premiums to the DB2 ledger	<code>QMABL020</code>	<code>BILL.POSTED</code>
<code>QMABL040</code>	Print report, write summary	<code>QMABL010</code>	<code>BILL.SUMMARY</code>
<code>QMAFN100</code>	Feed summary to Finance	<code>QMABL040</code>	—
<code>QMAPV001</code>	Load rate table, on demand	Not scheduled	<code>RATE.TABLE</code>

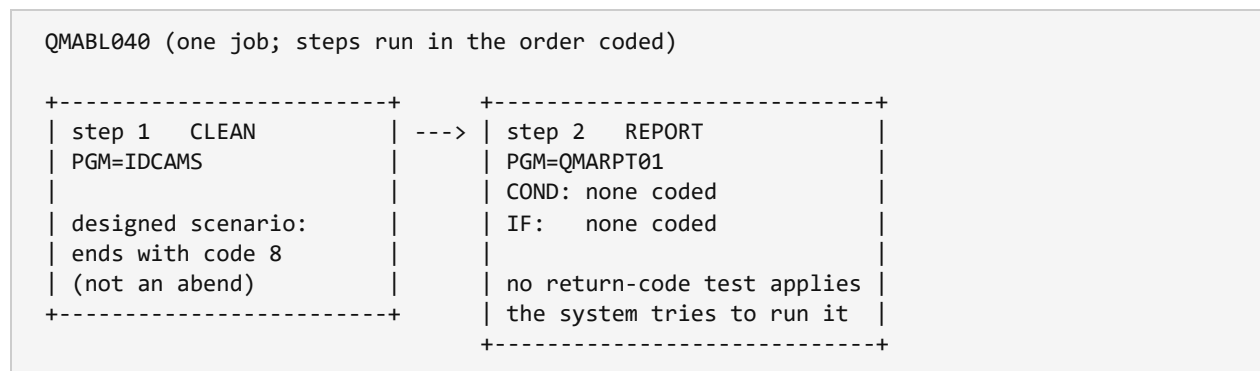
The data set names are short forms; the full names carry the qualifier `QMA`.

FIG-04-1 — A batch night · AUTHOR-CONCEPTUAL

Panel A: two kinds of arrow between jobs. The declared order of the designed scenario, not an observed run; there is no time axis.



Panel B: step order inside one job, with a condition-code callout for the designed scenario.



Legend. ==> with **S**: a scheduler-declared prerequisite. --> with **D**: a data set handed from one job to another. Dashed box: outside the bundle. No **S** arrow into a box: not scheduled.

Caption. The two arrow kinds are different evidence. QMABL040 has two predecessors: QMABL010 by the scheduler and QMABL030 by the data. The book does not choose between them; Level 3 returns to this. The figure does not show an observed run, a time of day, a duration or a return code from any real night; it does not imply that a scheduler folder is an application, that an **S** arrow means a data dependency, or that a **D** arrow creates a scheduler order.

Level 2 — Engineering mechanics

Jobs, steps and programs

A **job** is written in Job Control Language (JCL), which Chapter 5 reads closely: one **JOB** statement, then one or more steps. A **step** starts with an **EXEC** statement naming the program to run, or a procedure (Chapter 6). **DD** statements after it describe the data sets the step uses ([B2]).

Three facts matter for the rest of the chapter.

1. **Step order is order inside a job.** The steps of a job ordinarily run in the order coded. That says nothing about the order of jobs.

2. **A step hands something to a later step through a data set.** A temporary data set can be passed to the next step; a catalogued one persists, so a later job can find it by name. The other signal a later step can read is an earlier step's return code.
3. **Job classes, queues and how the system starts jobs are outside this book's cited sources.**

Return codes

When a step's program ends, it leaves a **return code**, a number that later tests can read. The system does not give the number a universal meaning. The program that set it does. A non-zero return code from a step that ended normally does not, by itself, stop the job or bypass a later step, and no platform rule makes 0 success and everything else failure.

QMABL040's first step runs the IDCAMS utility, so IDCAMS is the worked example. Its manual calls the number a condition code and gives these meanings ([B9], paraphrased):

Code	IDCAMS meaning
0	The function ran as directed and expected
4	A problem occurred, but the function continued
8	A requested function was completed, but major specifications were unavoidably bypassed; for example, an entry to be deleted could not be found in the catalog
12	The program could not perform the requested function
16	A severe error erased the remainder of the command stream

Three cautions.

- These meanings belong to IDCAMS. Another program may use 4 differently, or never set it.
- IDCAMS passes the highest code it set to the system when it returns.
- A condition code of 8 is **not** an abend.

An **abend** is different in kind. The system ends the step abnormally and records a system or user completion code. After an abend, later steps are normally bypassed. The exceptions are in the next two sections.

COND: a test that bypasses

COND on an EXEC statement is a **bypass test**. Written as COND=(code,operator,stepname), it reads "code operator return-code-of-step". If any test is true, the system bypasses the step. If every test is false, these tests do not bypass it; the abend rule above still applies ([B1]).

```
[PSEUDOCODE] generic JCL, not a Quillon member
//JOBX      JOB ...
//STEP1     EXEC PGM=ONE
//STEP2     EXEC PGM=TWO,COND=(4,LT,STEP1)
//STEP3     EXEC PGM=THREE
```

Read `COND=(4,LT,STEP1)` as "skip `STEP2` if 4 is less than the return code of `STEP1`". The number you write is on the **left** of the comparison, and the return code is on the right. To a programmer's eye the test reads backward, because it is the mirror of "bypass if the return code is greater than 4".

STEP1 return code	Is 4 LT that code?	STEP2
0	false	not bypassed by this test
4	false	not bypassed by this test
5	true	bypassed
8	true	bypassed

More rules from the same source:

- With several tests, one true test is enough to bypass the step.
- A test **without** a step name reads all previous steps' return codes.
- A test that names a step that was itself bypassed is false.
- A `COND` on the first `EXEC` of a job is evaluated as false.
- `COND=EVEN` lets a step run even after an earlier abend. `COND=ONLY` lets it run only after one. The system interprets these first, then applies any return-code tests.
- A `COND` on the `JOB` statement is a separate control. If one of its tests is satisfied, the remaining steps are bypassed.

IF / THEN / ELSE / ENDIF : the positive model

The `IF` statement is a second, separate mechanism. It speaks in positive terms: the steps in the `THEN` branch run when the condition is true, and the `ELSE` branch runs when it is false ([B1]).

```
[PSEUDOCODE] generic JCL, not a Quillon member
//CHK1      IF (STEP1.RC > 4) THEN
//STEP2     EXEC PGM=CLEANUP
//          ELSE
//STEP3     EXEC PGM=NORMAL
//CHK1      ENDIF
```

The condition can test a return code (`RC`, `stepname.RC`), an abend (`ABEND`, `-ABEND`, `ABENDCC`) or whether a step started (`RUN`, `-RUN`). The system evaluates it at execution time. `IF` constructs nest up to 15 levels, a limit unrelated to the procedure nesting limit in Chapter 6. When both an `IF`

and a `COND` apply to a step, the step runs only if both allow it. An `IF` that tests for an abend is one way a later step can run after one.

`COND` says "skip if". `IF` says "run if". Same question, opposite polarity, different statements.

When no test applies

IBM's JCL User's Guide states that when neither the `JOB` nor the `EXEC` statement carries `COND`, the system "tries to execute" each step ([B2]). Read that with its limits. Where no applicable `COND` causes bypass, and no surrounding `IF` rule or abnormal-termination rule excludes the step, the system performs no applicable return-code test and tries to execute the next step in normal sequence. "Tries to execute" is IBM's wording; it does not promise that the step completes.

Apply that to `QMABL040`.

```
[CASE-STUDY ARTIFACT] QMABL040 (jcl/QMABL040.jcl)
//QMABL040 JOB (ACCT),'BILLING REPORT',CLASS=A
//CLEAN     EXEC PGM=IDCAMS
//SYSIN     DD *
    DELETE QMA.BILL.WORK
/*
//REPORT    EXEC PGM=QMARPT01
//POSTED    DD DSN=QMA.BILL.POSTED,DISP=SHR
//SUMMARY   DD DSN=QMA.BILL.SUMMARY(+1),DISP=(NEW,CATLG,DELETE)
//REPORT    DD SYSOUT=*
```

In the designed scenario the work data set is absent on a clean night, so the `DELETE` ends with code 8: an entry to be deleted could not be found in the catalog, IDCAMS's own meaning for that case. This chapter does not claim every `DELETE` returns 8. `REPORT` has no `COND`, there is no `IF`, and 8 is not an abend, so the platform rules say the system performs no return-code test for `REPORT` and tries to execute it. That is a consequence of platform rules applied to layer-A text, not an observation.

The scheduler: declared prerequisites

A **scheduler** is a separate product that decides when jobs are submitted; its definition states what must be true first. This book's sources cover two, Control-M and CA-7, and this chapter describes only what they record. (the completeness of the scheduler export is not established)

- **Control-M.** The `exportdefjob` utility writes job definitions to XML. A `JOB` element describes a job, and sits inside a `SMART_FOLDER` container. `INCOND` lists input conditions that must be satisfied before submission. `OUTCOND` names conditions posted when the job completes ([B11]).
- **CA-7.** The `LJOB` command lists a job's definition. With the right options its output shows predecessor job requirements, data set requirements and network triggers ([B12]).

Quillon's export is a fictional Control-M-style file, `QMANIGHT.xml`. It declares the order shown in the earlier table, including the condition that makes `QMABL010` wait for `QMACL900`. What "ended OK" means is the scheduler's own definition; the cited sources do not state the rule, so this book does not equate it with return code 0.

Two limits on what a scheduler export can tell you:

- An export of definitions says what was defined. The Control-M source does not establish that an export preserves live schedule adjustments or manual overrides, so it does not say what ran on a given night.
- The CA-7 source does not establish that schedules reduce to a simple acyclic chain; requirements, triggers and manual release all exist.

A scheduler folder groups definitions. It is not a statement about application boundaries (Chapter 16).

Hands-On Exercise — read the night

Use the `QMABL040` text above and the scheduler order in the earlier table.

1. Name the steps of `QMABL040` in order, and the program each runs.
2. In the designed scenario `CLEAN` ends with code 8. List each rule that could cause `REPORT` to be bypassed. Does any apply?
3. *Hypothetical.* Someone adds `COND=(4,LT,CLEAN)` to `REPORT`. What happens when `CLEAN` ends with 8?
4. *Hypothetical.* Instead they add `COND=(8,LT,CLEAN)`. What happens?
5. Write each predecessor of `QMABL040` and the kind of artifact that evidences it.

Check your work. (1) `CLEAN` runs `IDCAMS`, then `REPORT` runs `QMARPT01`. (2) None applies. The rules are in "COND", "IF" and "When no test applies" above; `REPORT` has no `COND`, the `JOB` statement has none, there is no `IF`, and code 8 is not an abend. (3) 4 is less than 8, so the test is true and `REPORT` is bypassed; the step is not abended, it does not run. (4) 8 is not less than 8, so the test is false and does not bypass `REPORT`. (5) `QMABL010`, evidenced by the scheduler export; `QMABL030`, evidenced by the data set `QMA.BILL.POSTED`, which `QMABL030`'s JCL names with `DISP=(NEW,CATLG,DELETE)` and `QMABL040`'s JCL names with `DISP=SHR`.

Quillon, sorted

Observed (layer A)	Derived (platform rule applied by hand)	Interpreted
The export declares <code>QMABL040</code> after <code>QMABL010</code> . <code>QMABL030</code> 's JCL names <code>QMA.BILL.POSTED</code> with <code>DISP=(NEW,CATLG,DELETE)</code> ; <code>QMABL040</code> 's JCL names it with <code>DISP=SHR</code> . <code>REPORT</code> has no <code>COND</code> .	If <code>CLEAN</code> ends with 8, no return-code test applies to <code>REPORT</code> ; with no <code>IF</code> and no abend, the system tries to execute it.	" <code>QMABL040</code> should wait for <code>QMABL030</code> ." "Billing is one application."

No ETC output appears in the panel, so there is no layer-B column.

Level 3 — Architecture reasoning

Scheduler order is not data dependency, and the reverse

Two jobs can be ordered by a scheduler, tied by a data set, both, or neither. Each combination means something different.

Scheduler declares A before B	Data set handed from A to B	What the artifacts support	Quillon
yes	yes	Two distinct artifacts point the same way. Chapter 1 called this corroboration	QMABL020 then QMABL030
yes	no	An order with no data reason in these artifacts. A resource limit, a business rule or a leftover would fit; the artifacts do not say	QMABL010 then QMABL040
no	yes	A data need that nothing in the export enforces. Whether anything else does is not in the artifacts	QMABL030 to QMABL040 ; QMAPV001 to QMABL020
no	no	Independent, as far as these artifacts show	—

The third row is the sharp one. QMABL040 consumes what QMABL030 posts, yet the export makes it wait for QMABL010. If nothing else enforces the data order, the report could run before posting completes. Whether that happens is a question for the people who run the night; no artifact answers it. The book picks no winner.

Read from JCL alone, a data handoff rests on disposition text, which states allocation, not what a program does with the file (Chapters 2 and 7).

A return code is not the business result

A code of 0 says the program ended the way it defines 0. It says nothing about whether the premiums were right, so zero is not a business success.

Interpretation. A design that replaces step sequencing with an orchestrator that treats any non-zero code as a stop would change what the platform does on this night. That is a design consideration for any re-platforming, not a statement about any tool.

Architect's Warning

"The order of the batch" names three things: the order a scheduler declares, the order the data requires, and the order in which jobs ran. Only the third is run-time, and it is not in the bundle. A declared order must not stand in for another.

Claim Check

"`CLEAN` returned 8, so the job failed and `REPORT` did not run."

This fails. Code 8 is IDCAMS's own signal, not an abend. `REPORT` has no `COND` and sits in no `IF`, so the system tries to execute it. The sentence also treats a designed scenario as an observed run.

Quick Recall

1. Is the order of steps inside a job the order of jobs? Why?
2. `COND=(4,LT,STEP1)` : for which return codes of `STEP1` is the step bypassed?
3. Name two ways a step can run after an earlier abend.
4. How does an IDCAMS code of 8 differ from an abend?
5. A scheduler orders `A` before `B`. Does that show `B` reads `A`'s output?

Key Takeaways

- Jobs are ordered by a scheduler, tied by data sets, both or neither.
- `COND` is a bypass test: a true test skips the step. `IF` is a positive test: a true condition runs the `THEN` steps.
- With no applicable `COND`, `IF` or abend rule, the system performs no return-code test and tries to execute the step.
- A return code means what its program says: non-zero is not a platform failure, and zero is not a business success.
- Declared scheduler order, data order and run-time order are three different things. Quillon's night shows only the first two, as designed.

Chapter 5 — JCL as Evidence

After this chapter you can:

- read a `JOB / EXEC / DD` listing and say which facts it states, which the platform's rules fill in, and which it does not contain;

- explain `DISP`, `(+1)` generations, `DD *` and `SYSOUT` as platform concepts without reading program input or output into them;
- say what ETC currently records for a job, and list five things the reviewed reading does not record and does not report.

The chapter keeps two questions apart. *What does JCL mean on z/OS?* is platform truth, from IBM's documentation. *What does ETC currently record?* is implementation truth, from the code and its tests. Neither answers the other.

Level 1 — Intuition

A work order, not the work

A work order tells a factory which machine to start, which materials to load and what to do with the leftovers. It does not contain the machine's engineering, and it does not say what the machine did on a given night.

Job Control Language (JCL) is a work order. It tells the system which program to run, which data sets to connect to it, and what to do with each data set afterward. It does not contain the program's logic. It does not say what the program does with a file once the file is connected.

That makes a JCL member a precise but limited piece of evidence. Every item in a member falls into one of three groups.

Group	Meaning	Example from <code>QMABL040</code>
Stated	The member says it in so many words	<code>PGM=QMARPT01 ; DISP=SHR</code> on <code>POSTED</code>
Left to the platform's rules	The member is silent, and IBM documents what the system does	No <code>COND</code> on <code>REPORT</code> : the system performs no return-code test for it and tries to execute it. An earlier abend, an <code>IF</code> or a <code>JOB</code> -level <code>COND</code> could still exclude a step
Not in the member	Only another artifact or a run could say	What <code>QMARPT01</code> does with <code>POSTED</code> ; what code <code>CLEAN</code> ends with on a given night

FIG-05-1 — Anatomy of a JCL job · SOURCE-DERIVED

```
[CASE-STUDY ARTIFACT] QMABL040, annotated
//QMABL040 JOB (ACCT),'BILLING REPORT',CLASS=A           [1]
//CLEAN      EXEC PGM=IDCAMS                             [2]
//SYSIN      DD *                                       [3]
    DELETE QMA.BILL.WORK
/*
//REPORT     EXEC PGM=QMARPT01                           [4]
//POSTED     DD DSN=QMA.BILL.POSTED,DISP=SHR             [5]
//SUMMARY    DD DSN=QMA.BILL.SUMMARY(+1),DISP=(NEW,CATLG,DELETE) [6]
//REPORT     DD SYSOUT=*                                 [7]
```

1. **Stated:** the job name, an accounting field and `CLASS=A`. **Not stated:** any `MSGCLASS`, and any `COND` on the `JOB` statement. This book interprets no job class.
2. **Stated:** the program. **Not in the member:** the code `CLEAN` ends with on any night.
3. **Stated:** the lines up to `/*` are the step's input. A `DD *` ends at `/*` or at a line that starts `//`.
4. **Stated:** nothing about conditions. No `COND` and no `IF`: the system performs no return-code test for `REPORT` and tries to execute it, unless an earlier step abended and nothing (`EVEN`, `ONLY`, an abend-testing `IF`) lets it run.
5. **Stated:** a data set name and a disposition. A `DISP` states allocation and disposition, not program I/O. `SHR` does not make the data set read-only for the program.
6. **Stated:** `(+1)` names a new generation that does not exist yet. `DISP=(NEW,CATLG,DELETE)` is what requests its allocation and cataloging.
7. **Stated:** output goes to the spool. `SYSOUT=*` takes its class from the job's `MSGCLASS`. This member codes none, so the class is not stated here.

Caption. The listing is a fictional member, annotated from IBM's z/OS JCL Reference and User's Guide ([B1], [B2]). The figure does not show any access-mode arrow, does not imply that JCL states what a program reads or writes, does not imply that this declared text is a view after procedure expansion (Chapter 6), and does not imply that any tool evaluates the `COND` callout.

Level 2 — Engineering mechanics

Part A. What JCL means on z/OS

Three statements

A `JOB` statement begins a job, names it and carries job-wide parameters. An `EXEC` statement begins a step and names the program (`PGM=`) or a procedure to run. A `DD` statement describes one data set or data source for the step. Its name, the **ddname**, is the name the program uses to

reach the data, and it is local to the step. Two steps may use the same ddname for different data sets.

A statement starts with `/**`. An operand field that ends with a comma continues on the next line. A line starting `/**` is a comment ([B1]).

The constructs this chapter needs

Construct	Platform meaning	What it leaves unstated
<code>EXEC PGM=name</code>	The step runs this program	What the program does; what it returns
<code>EXEC name</code> or <code>PROC=name</code>	The step invokes a procedure (Chapter 6)	The steps inside the procedure
<code>PARM=</code>	A string passed to the program	What the program does with it
<code>COND=</code>	A bypass test (Chapter 4)	Whether any test is true on a given night
<code>DD DSN=</code>	Names the data set the ddname refers to	Whether it exists; what the program does with it
<code>DISP=</code>	Status and end-of-step handling (below)	Program input or output
<code>DD DUMMY</code>	No data set is allocated and no disposition is processed; for BSAM and QSAM no input or output is performed, and a read reaches end-of-data at once	Chapter 6 uses it in an override
<code>DD *</code>	Input records follow in the job stream	What the records mean to the program
<code>SYSOUT=class</code>	Output goes to the spool; <code>*</code> takes the class from <code>MSGCLASS</code>	What happens to the output next
<code>&&NAME</code>	A temporary data set that exists for the job and can be passed between steps	—
<code>NAME(0)</code> , <code>(-1)</code> , <code>(+1)</code>	Relative generations of a generation data group (GDG)	—

`DISP` in one page

`DISP=(status,normal,abnormal)` has three positions. The **status** says what the system should expect of the data set: `NEW`, `OLD`, `SHR` or `MOD`. The **normal** and **abnormal** dispositions say what to do with it at the end of the step: for example `KEEP`, `DELETE`, `PASS` or `CATLG`. IBM documents these defaults ([B1]):

- an omitted status means `NEW`;
- an omitted normal disposition means `DELETE` for a `NEW` data set and `KEEP` for an existing one;
- an omitted `DISP` means `(NEW,DELETE,DELETE)`, the case Chapter 2 used.

`DISP` governs allocation. `SHR` allows sharing and does not promise that the program only reads. `OLD` asks for exclusive access and does not imply that the program modifies anything. What a program does with the file is decided inside the program (Chapter 2, Chapter 7).

Generations

A generation data group is a set of related data sets reached by relative number. `(0)` is the current generation and `(-1)` the one before it. `(+1)` names a new generation that does not yet exist ([B2]). The number is a **name**, and naming a generation creates nothing. On `SUMMARY`, `DISP=(NEW,CATLG,DELETE)` requests that the generation be allocated and cataloged. Whether one comes to exist depends on the allocation succeeding and the step running. Chapter 7 shows how ETC names such a store.

Reading `QMABL040` as evidence

Question	Stated in the member	Platform rules say	Not available
Which program does step 2 run?	<code>QMARPT01</code>	—	—
Is <code>REPORT</code> conditioned on <code>CLEAN</code> ?	No <code>COND</code> , no <code>IF</code>	No return-code test; the system tries to execute it unless an earlier abend excludes it	What happened on any night
What happens to <code>SUMMARY(+1)</code> at the end?	<code>(NEW,CATLG,DELETE)</code>	Status <code>NEW</code> ; cataloged at normal end; deleted at abnormal end	Whether the allocation succeeds
Does <code>QMARPT01</code> read <code>POSTED</code> ?	<code>DISP=SHR</code>	<code>SHR</code> is about sharing	The program's <code>OPEN</code> mode
Where does the <code>REPORT</code> output go?	<code>SYSOUT=*</code>	Spool, class from <code>MSGCLASS</code>	A <code>MSGCLASS</code> , which this member does not code

`QMABL030` shows the last column too. It runs `IKJEFT01` and names the program `QMAPST01` only inside its in-stream control text. The book's sources do not explain `IKJEFT01`, so it records that observation and nothing more.

Part B. What ETC currently records

Everything above is platform truth. From here the chapter describes ETC at the baseline, as reviewed (re-read and checked against its tests on 2026-10-04).

Current behavior

ETC reads one job member into an ordered list of steps, each with the data definitions coded under it.

- A member with no `JOB` statement is refused. One member is one job.
- Steps are recorded in the order coded and numbered from 1.
- `PGM=` records a program. `PROC=` or a positional name records a procedure invocation; Chapter 6 follows it.
- `PARM` is kept as text. A single-test `COND` is kept as text. Neither is evaluated.
- Continuation lines are joined, comment cards are skipped, and a `DD` with a blank name field inherits the name above it within the step.
- An in-stream procedure definition is separated from the steps (Chapter 6).
- Each `DD` card under a recorded step becomes one recorded access. A candidate store name is recorded where the card names a data set; a card with no `DSN` (such as `SYSOUT` or in-stream data), a `DUMMY`, or a reserved no-store name carries none. A `DD` before the first `EXEC` is not recorded (limitation 4). What a store name means is Chapter 7.

A preview: a store's access direction "comes from the DD disposition and documented utility/role rules, not from the COBOL `OPEN` mode." Chapter 7 covers this.

Pseudocode — the statement loop

```
[PSEUDOCODE – PUBLIC EDITION]  reading one member's statements, in order
for each statement in the member, in order:
  if the statement is a JOB, PROC or PEND statement:
    skip it
  if the statement is an EXEC statement:
    close the step being built
    start a new step with the next ordinal
    capture, from the operand text, the FIRST match of each of:
      the program name, the procedure name,
      the parameter text, the condition text
  if the statement is a DD statement and a step has been started:
    record one data-store access on that step:
      logical name = the DD name, or "UNNAMED"
      store name   = the DSN text, normalised
      store kind   = from the JCL evidence
      access mode  = from the DISP status           # Chapter 7
      lifecycle    = from the DISP subparameters    # Chapter 7
    (a DD statement before any EXEC statement is not recorded)
```

Simplified. Continuation joining and comment skipping (done earlier, by a statement scanner); the positional-name and unreadable-`EXEC` handling; `STEPnnn` naming of unnamed steps; symbol overrides; the derivation of the data set name and the disposition text from the `DSN` and `DISP`

operands; `DUMMY` and no-store-name handling; inline data; record length; the temporary-name flag; the absent-disposition warning; and the work done when a step is closed. The pseudocode adds no behavior. The access-mode and lifecycle mappings are Chapter 7's.

Read it twice. First, the `EXEC` branch captures four operands, taking the first match of each. Second, a `DD` is recorded only when a step has already been started, and no branch handles any other statement.

`QMABL040` as ETC is expected to record it

`QMABL040` is designed to be read into two steps and four data definitions. The table below is an **expected pedagogical outcome, not an ETC result** (Layer C: no case-study run is recorded in this edition). It lists only fields whose values come straight from the card text.

Step	Program	PARM	COND	Data definitions (ddname: operand text)
1 CLEAN	IDCAMS	none	none	SYSIN: in-stream text DELETE QMA.BILL.WORK
2 REPORT	QMARPT01	none	none	POSTED: QMA.BILL.POSTED, SHR. SUMMARY: QMA.BILL.SUMMARY(+1), (NEW,CATLG,DELETE). REPORT: SYSOUT=*

Store kind, access mode, lifecycle and normalized store name are left blank on purpose. They are Chapter 7's questions.

Current Support

Capability	Status	Evidence
Read one member as one ordered job with its data definitions	IMPLEMENTED	tests re-run at the baseline
Keep <code>PARM</code> and single-test <code>COND</code> as text	IMPLEMENTED, text only	Reviewed at the baseline
Refuse a member with no <code>JOB</code> statement	IMPLEMENTED	Reviewed at the baseline

Current Limitation

CURRENT IMPLEMENTATION LIMITATION. The following five statements describe the reviewed reading at the baseline. They are properties of the code, found by reading it and confirmed by a probe. They are not a statement about what JCL permits, and the review records no intent behind them.

1. **A multi-test `COND` is truncated.** The pattern stops at the first `)`. `COND=((4,LT),(8,GT,S1))` is recorded as `((4,LT)`. The recorded text is reliable only for the single-test form.
2. **IF / THEN / ELSE / ENDIF structure is not preserved.** Steps inside the construct are read as plain sequential steps with no condition.
3. **A `JOB` -level `COND` is not read.** The pattern is applied only on an `EXEC` statement.

4. **DD statements before the first EXEC are not recorded.** A `JOBLIB` is an example.
5. **None of these four cases raises a diagnostic** in the reviewed reading.

Nothing in this chapter says ETC records each step's conditions or reads conditional execution. It does not.

A consequence follows from items 1 to 3. An empty `COND` on a recorded step means only that no single-test `COND` text was captured on that `EXEC`. It does not show that nothing governs the step. Chapter 1's distinction applies: the record is silent, and silence is UNKNOWN, not NONE.

Code Trace Challenge — predict what is recorded

The input is a fictional job, used as a probe.

```
[PSEUDOCODE] generic JCL, not a Quillon member
//PROBE      JOB (X), 'P', CLASS=A, COND=(8,LT)
//JOBLIB     DD DSN=MY.LOAD, DISP=SHR
//S1         EXEC PGM=ONE
//S2         EXEC PGM=TWO, COND=((4,LT),(8,GT,S1))
//CHK       IF (S1.RC > 4) THEN
//S3         EXEC PGM=THREE
//          ELSE
//S4         EXEC PGM=FOUR
//CHK       ENDIF
//          EXEC PGM=FIVE
```

First say what the platform means: `S3` runs only when `S1`'s return code is above 4, and `S4` runs otherwise. Then predict what the reviewed reading records:

a. How many steps, under what names? b. What `COND` text does it record for `S2`? c. Which steps carry the `IF` condition? d. Is `JOBLIB` recorded? e. How many ingestion issues are raised?

Answer (a probe run at the baseline on 2026-10-04, not a committed test). (a) Five: `S1`, `S2`, `S3`, `S4`, and `STEP005`, the name an unnamed step gets from its position. (b) `((4,LT),(8,GT,S1))`. (c) None. `S3` and `S4` are recorded as plain sequential steps with no condition, so the record no longer shows that they are alternatives. (d) No, and no data definition is recorded at all. (e) None. The job's own `COND=(8,LT)` is not read either.

Quillon, sorted

Observed (layer A)	Expected derivation (not an ETC result)	Interpreted
<code>QMABL040</code> has two steps and four <code>DD</code> statements. <code>REPORT</code> has no <code>COND</code> . <code>SUMMARY</code> names a (+1) generation with <code>DISP=(NEW,CATLG,DELETE)</code> .	The record shape in the table above, read from the card text (layer B; Layer C: no case-study run is recorded in this edition).	"The report is meant to run whatever <code>CLEAN</code> returns."

The last cell is interpretation. The absence of a `COND` is observed. Whether it was a decision is not in the member.

Level 3 — Architecture reasoning

Platform coverage is not parser coverage

The platform defines `COND` on the `JOB` statement, `IF` constructs and `JOBLIB`. ETC's reading covers some of that, as the limitation box shows. Teaching a construct in this chapter never implies that ETC reads it. A tool and a language have separate boundaries. Ask of any sentence whether it is about the language or the tool.

Failure Case — a clean read that says less than it seems

In the Code Trace Challenge the reviewed reading raised no issue. Five steps came back and nothing was flagged. The clean result is true as far as it goes: it describes the cards the reader looked at. It is not evidence that the job has no conditional structure, because the job did, and the reader never looked at it.

The lesson is general. A tool that reports no problem has told you about the constructs it covers. It has not told you about the ones it does not. Coverage has to be stated beside the result, which is why the limitation box is part of the chapter and not an appendix. Anything built on these records inherits their coverage.

Declared text is evidence of a request

A JCL member is evidence of what was requested of the system. It is not a record of what the system did. The chapter title says *evidence* for that reason. The next chapter makes the point sharper: even when procedures are expanded, the expanded text is still text.

Architect's Warning

The most likely misreading is to take recorded step data as the whole of a job's control flow. The reviewed reading preserves order, programs, `PARM` text, a single-test `COND` text and the data definitions under a step. It does not preserve conditional structure. Treat step order in the record as "coded order", not "order of execution".

Claim Check

"ETC reads the job's conditions, so it knows which steps run."

This fails three ways. It records only the first test of a multi-test `COND`. It does not preserve `IF` structure or read a `JOB`-level `COND`. And what runs on a given night is run-time evidence that no JCL member holds.

Quick Recall

1. Name the three groups into which every item in a JCL member falls.
2. What does `(+1)` name, and what requests the allocation?
3. Does `DISP=SHR` mean the program only reads? Why?
4. `REPORT` has no `COND` and no `IF`. What does the system do?
5. List the five limitations of the reviewed reading from the box.

Key Takeaways

- JCL is a work order. It states requests; it does not contain the program or the night's outcome.
- `DISP` is allocation and disposition, not program I/O. `(+1)` is a name, and the `DISP` is what requests the allocation.
- ETC reads steps, programs, `PARM`, single-test `COND` text and data definitions. It does not preserve `IF` structure, read a `JOB`-level `COND`, keep a multi-test `COND` whole, or record a `DD` before the first `EXEC`.
- None of those four cases raises a diagnostic, so a clean read describes coverage and nothing else.
- Quillon's expected record shape is a layer-B expectation until the bundle is run.

Chapter 6 — Procedures: Declared vs Effective Execution

After this chapter you can:

- explain how a procedure is found, given values and overridden on z/OS;
- tell a Declared Step from an Effective Step, and say why neither is a record of what ran;
- state what ETC's expansion does on the conformance fixture cases, and name the three things the evidence leaves open: the DD override, the nesting boundary and the cycle rule.

The chapter keeps three kinds of statement apart. *Platform* statements come from IBM's documentation. *ETC* statements describe the reviewed code and its tests at the baseline. *Quillon* statements are about invented members, and any ETC outcome for Quillon is an expectation, because the bundle has not been run.

Level 1 — Intuition

A form letter and its blanks

A procedure is a form letter with blanks: "Dear &NAME, your account is &ACCOUNT." The job says only "send form QMARATE, with HLQ=QMA". A margin note may amend one line of the form for this mailing only. Reading the job alone, you see "send the form". Reading with the blanks filled in and the margin notes applied, you see everything the letter will say.

Both readings are text. Neither is a record of what happened when the letter went out.

Two terms follow, and this chapter is where the book defines them.

- A **Declared Step** is a step as written in the job, including a step that invokes a procedure. It is not what executes.
- An **Effective Step** is a step after procedure expansion, symbol substitution and overrides. It is derived from text. It is not a step observed at run time.

The title says "effective execution". Read "effective" as "after expansion". It never means "as it ran". z/OS itself can print an expanded listing of a job, with procedures expanded, overrides applied and symbols resolved ([B2]). That listing is the platform's own expanded view of the job *text*. It is not evidence of what ran either. No run-time evidence for Quillon exists in this book.

Quillon's QMABL020

QMABL020 rates policies. Its entire JCL is three statements.

```
[CASE-STUDY ARTIFACT] QMABL020 (jcl/QMABL020.jcl)
//QMABL020 JOB (ACCT),'RATE POLICIES',CLASS=A
//RATE      EXEC QMARATE,HLQ=QMA,CYCLE=D
//RATE.SYSOUT DD DUMMY
```

Behind the one EXEC stand two cataloged procedures, QMARATE and QMASORT. FIG-06-1 puts the declared texts beside the expanded view. Skim it now. Level 2 explains each rule.

FIG-06-1 — PROC expansion · AUTHOR-CONCEPTUAL

DECLARED TEXT (as written)	EXPANDED VIEW (author's reading)
-----	-----
QMABL020 (job) RATE EXEC QMARATE, HLQ=QMA,CYCLE=D RATE.SYSOUT DD DUMMY	QMABL020 (job) RATE -> QMARATE SORTP -> QMASORT SORT PGM=SORT SORTIN QMA.BILL.EXTRACT(0)
QMARATE (PROC HLQ=QMA,CYCLE=D) SORTP EXEC QMASORT, IN=&HLQ..BILL.EXTRACT(0) RATE EXEC PGM=QMARTE01 RATED DD DSN=&HLQ..BILL. RATED.&CYCLE,... SYSOUT DD SYSOUT=*	RATE PGM=QMARTE01 RATED QMA.BILL.RATED.D SYSOUT DUMMY <- override
	procedure path: QMARATE > QMASORT (two procedures on the path)
QMASORT (PROC IN=) SORT EXEC PGM=SORT SORTIN DD DSN=&IN,DISP=SHR	

Caption. The left column is layer A: the three members as written. The right column is the author's reading of them by IBM's rules ([B1]), applied by hand. It is not an ETC-produced view. An ETC-produced view would be an expected pedagogical outcome, not an ETC result (Layer C: no case-study run is recorded in this edition). The figure shows only some data definitions. It does not show a run-time result, does not imply that ETC or any tool produces a view identical to the right column, and does not imply an expansion equivalent to the platform's own listing.

Level 2 — Engineering mechanics

Part A. What procedures mean on z/OS

Two kinds, one search order

An **in-stream procedure** is defined inside the job, between a `PROC` and a `PEND` statement, and must appear before the `EXEC` that calls it. A **cataloged procedure** is a member of a procedure library, called by its member name or alias, and needs no `PEND` ([B1], [B2]).

When the system looks for a procedure it searches the input stream first, then a private library named on a `JCLLIB` statement, then the system procedure library. A job that defines its own procedure therefore gets its own.

A job invokes a procedure with `EXEC name` or `EXEC PROC=name`, as `QMABL020` does.

Symbols and defaults

A procedure is written against **symbols**, names that start with `&`. Defaults are coded on the `PROC` statement (`PROC HLQ=QMA,CYCLE=D`). The `EXEC` that invokes the procedure may supply values, and a supplied value beats the default. Coding a symbol with nothing after the equals sign gives it a null value.

A period ends a symbol's name. When the fixed text after a symbol begins with a period, the symbol needs a period of its own, and that period does not appear after substitution. With `HLQ=QMA`:

Written	After substitution
<code>&HLQ..BILL.EXTRACT(0)</code>	<code>QMA.BILL.EXTRACT(0)</code>
<code>&HLQ..BILL.RATED.&CYCLE</code> with <code>CYCLE=D</code>	<code>QMA.BILL.RATED.D</code>

`QMARATE` also passes a value down: its `SORTP` step supplies `IN=&HLQ..BILL.EXTRACT(0)` to `QMASORT`, so inside `QMASORT` the symbol `&IN` stands for `QMA.BILL.EXTRACT(0)`. The result is the author's platform reading.

Overrides

A job changes a procedure's data definition for one execution by coding `//procstep.ddname DD ...` after the `EXEC`. IBM merges the overriding parameters into the procedure's statement:

- a parameter coded on the override replaces the matching parameter;
- a parameter not on the procedure's statement is added;
- a parameter not coded on the override remains;
- `DUMMY` on an override nullifies all parameters except `DCB`.

Overrides that name the step explicitly may appear in any order, and the procedure itself is not changed ([B1]).

`QMABL020` codes `//RATE.SYSOUT DD DUMMY`. It replaces `SYSOUT=*` on the procedure's step called `RATE`. There are two steps named `RATE` here: the job's declared step, and the step inside the procedure. The qualifier names the second.

Nesting

A procedure can invoke another. IBM states that cataloged and in-stream procedures can invoke other procedures, "up to 15 levels of nesting" ([B1]). The recorded text does not say how levels are counted or what happens at a sixteenth, and this chapter says nothing about either.

Three separate limits of 15 appear in IBM's documentation. Do not mix them: procedure nesting levels; `IF / THEN / ELSE` nesting levels (Chapter 4); and the number of in-stream procedures in one job ([B2]). The last is a count, not a depth.

Cycles

Neither IBM manual cited here says what the system does when a procedure invokes itself, directly or indirectly. Cycles therefore appear below as ETC behavior only.

Part B. QMABL020 expanded by hand

Applying those rules to the three members gives two effective steps from one declared step. This is the author's platform reading, not an ETC result.

Effective step	Reached through	Program	Data definitions after substitution
<code>SORT</code>	<code>QMARATE</code> , then <code>QMASORT</code>	<code>SORT</code>	<code>SORTIN: QMA.BILL.EXTRACT(0) , SHR . SORTOUT: &&SORTED , (NEW,PASS) . SYSIN: QMA.CNTL(SORTPOL) , SHR</code>
<code>RATE</code>	<code>QMARATE</code>	<code>QMARTE01</code>	<code>SORTED: &&SORTED , (OLD,DELETE) . POLMAST: QMA.POLICY.MASTER , SHR . RATETAB: QMA.RATE.TABLE , SHR . AUDIT: QMA.BILL.AUDIT , no DISP . RATED: QMA.BILL.RATED.D , (NEW,CATLG,DELETE) . SYSOUT: DUMMY</code>

The temporary data set `&&SORTED` is created by one procedure step and passed to the next. Chapter 7 returns to `AUDIT` 's missing `DISP` .

Part C. What ETC currently does

From here the chapter describes ETC at the baseline (re-read and checked against its tests on 2026-10-04). It speaks of the conformance fixture cases, the set of small procedure cases ETC is tested against. The statement this chapter supports is: ETC expands cataloged and in-stream procedures, including nesting, symbolic substitution, DD overrides, and reports cycles and unresolved procedures distinctly. ETC does not claim complete JCL semantics, and nothing here says ETC's expansion matches IBM's own listing.

Behavior the tests establish

- **Reading.** Cataloged members are read as procedures. An in-stream `PROC ... PEND` block is separated from the job's steps, and an unterminated block is reported.
- **Search.** An in-stream procedure is preferred over a cataloged one of the same name. ETC separates in-stream from cataloged and makes no claim about the order of private libraries.
- **Substitution.** Each invocation is expanded with its own symbols. A supplied value beats a default. A symbol nobody supplies stays a symbol. Each substitution records the symbol and whether its value came from the procedure's default or the invocation.
- **Nesting.** Both levels of a nest are expanded, and a symbol survives being renamed on the way down.

- **Overrides.** An override replaces the data definition of the same name. One naming a definition the procedure step lacks is added. One naming no step of the procedure is kept as *unplaced*, is not silently attached to a step, and is still counted.
- **Outcomes.** The result of each invocation is recorded as one of six kinds: resolved, not supplied (a determinate name the bundle does not hold), reference unresolved (the name is still a symbol), platform-declared (a curated catalogue names it), cycle, or too deep.
- **Repetition and cycles.** Recursion is guarded per invocation path, so a procedure invoked twice in one job is not a cycle. A procedure that reaches itself is. Expansion stops there, the steps read before it are kept, and the cycle is reported as the path of procedure names.
- **Depth.** A refusal at the nesting limit stops expansion and says so.

Pseudocode — the recursion and the override

```
[PSEUDOCODE – PUBLIC EDITION]  expanding one procedure invocation (messages and severities
omitted)
procedure = the invoked name, looked up among the available procedures
if no procedure is found:
    record an issue saying why it is unresolved
    mark the step with that unresolved state, and stop
if the name is already on the path of procedures being expanded:
    record a cycle issue naming the path
    mark the step CYCLE, and stop
if the path has reached the nesting limit:
    record a too-deep issue
    mark the step TOO_DEEP, and stop
extend the path with this name
expanded steps = for each statement-level step of the procedure,
                  with the invocation's operands applied,
                  expand it again by this same rule, along the extended path
attach the expanded steps and the procedure's origin to the step
mark the step RESOLVED
```

Simplified. The guard for a step that invokes no procedure, and the warning text and severities, are left out. The pseudocode adds no behavior. The path holds the procedures already being expanded above this step, which is why a repeat invocation in a different branch is not a cycle.

```
[PSEUDOCODE – PUBLIC EDITION] applying a job's DD overrides to one procedure step
if the job supplies no overrides for this step:
    keep the step as it is
remaining = the supplied overrides, keyed by upper-cased DD name
kept = for each data definition of the step, in order:
    the override of the same name, if there is one
        (and take it out of remaining),
    otherwise the procedure's own definition
result = kept, followed by whatever is left in remaining
```

The overrides come from the job's `//procstep.ddname DD` cards. For each data definition of the step, the job's card of that name replaces the procedure's own, if there is one. The whole recorded access is swapped. Names left over are appended at the end. Section "The DD override" in Level 3 returns to this.

Pseudocode — the two views

```
[PSEUDOCODE – PUBLIC EDITION] the declared view and the expanded view of a job
a step has: its declared data definitions, and a list of expanded steps
expanded_view(step):
    if the step has no expanded steps:
        return [step]
    return the concatenation of expanded_view(inner)
        for each inner step in the step's expanded steps
the job's expanded steps =
    the concatenation of expanded_view(step) for each declared step of the job
```

The job's declared steps are the declared view. The invocation step is kept and carries its expansion as a list of expanded steps. The expanded view returns that expansion, recursively, in place of the invocation. This book calls it the "expanded steps" view, and the result is derived from text. It is not run-time evidence.

An invocation also states things that belong to no step of its expansion, such as an unplaced definition. Two further accessors keep such definitions counted once and keep the invocation visible where it states something itself.

FIG-06-2 — Declared vs effective execution · AUTHOR-CONCEPTUAL

DECLARED VIEW steps, as written				EXPANDED / EFFECTIVE VIEW expanded steps, derived from text	
-----				-----	
1	INVOKE-A	EXEC PROCA	--->	A1	from PROCA
				B1	from PROCB, invoked by PROCA
2	STEP-X	EXEC PGM=X	--->	STEP-X	unchanged
The invocation stays in the declared view beside its expansion.					
Consumers the tests check, reading the expanded view:					
[data-flow dependencies] [store questions and coverage]					
[job-to-program CALLS projection]					

Caption. Generic names; the structure and the three consumers are checked against the project's tests (rechecked 2026-10-04). The right-hand view is derived from text and is not run-time-observation evidence. The figure does not show the declared view being discarded, does not imply that the expanded view is what ran, does not imply that any consumer beyond the three shown reads it, and shows no Quillon output produced by ETC.

For `QMABL020`, the declared view is expected to hold one step (`RATE`) and the expanded view two (`SORT`, then `RATE`), with the substituted data set names (`QMA.BILL.EXTRACT`, `QMA.BILL.RATED.D`) and the `SYSOUT` override applied. This is an expected pedagogical outcome, not an ETC result (Layer C: no case-study run is recorded in this edition).

Which consumers read the expanded view

ETC's tests check these consumers against the expanded view:

- data-flow dependencies, including one declared only inside a procedure;
- store questions and coverage, where a store named by an unsupplied symbol inside a procedure is reported, and the two coverage views agree;
- the job-to-program `CALLS` projection, which links a job to the program its procedure names.

This chapter claims no others.

Current Support

Capability	Status	Evidence
Expand cataloged and in-stream procedures, with nesting and symbol substitution with provenance, on the conformance fixture cases	IMPLEMENTED, scoped to those cases	Reviewed at the baseline
Replace a procedure data definition by a job's override of the same name; add or keep unplaced the ones that match nothing	IMPLEMENTED, scoped	Reviewed at the baseline
Report cycles, depth refusals and unresolved procedures as distinct outcomes	IMPLEMENTED	Reviewed at the baseline
Keep the declared invocation beside its expansion	IMPLEMENTED	Reviewed at the baseline

Current Limitation

1. **The DD override is whole-card replacement.** See Level 3.
2. **The nesting boundary at 15 is untested.** ETC refuses a procedure when 15 are already on the invocation path, a number that matches the IBM figure. The test substitutes a limit of 1, so the boundary at 15 is never exercised. Whether ETC counts levels the way IBM does is not established.
3. **ETC's diagnostics for cycles and depth carry phrasing about platform behavior.** That phrasing is not a source and this chapter does not rely on it.
4. **ETC distinguishes in-stream from cataloged procedures only.** It makes no claim about private-library order.
5. **The expanded view is text-derived.** Nothing here observes a run.

Code Trace Challenge — a cycle

```
[PSEUDOCODE] generic names, not a Quillon member
JOB1      //S1 EXEC PROCA
PROCA     //A1 EXEC PGM=PROG1
          //A2 EXEC PROCB
PROCB     //B1 EXEC PGM=PROG2
          //B2 EXEC PROCA
```

Trace `_resolved` on `S1` with `path = ()`.

a. Which step receives the `cycle` outcome? b. What path is reported? c. Which steps are kept? d. Remove `B2` and let the job invoke `PROCA` from two steps. Is that a cycle?

Answer. (a) `B2`: when it is reached, `path` is `(PROCA, PROCB)` and `PROCA` is already in it. (b) `PROCA -> PROCB -> PROCA`. (c) `A1`, `A2` and `B1`, the steps read before the repeat, and `B2`, which stays as an unexpanded step carrying the cycle outcome. (d) No. Each invocation starts with an empty path,

so repetition across steps is not recursion.

Quillon, sorted

Observed (layer A)	Expected derivation (not an ETC result)	Interpreted
QMABL020 has one EXEC of QMARATE with HLQ=QMA,CYCLE=D and an override RATE.SYSOUT DD DUMMY . QMARATE 's SORTP step invokes QMASORT . RATED is &HLQ . .BILL .RATED .&CYCLE .	By IBM's rules, applied by hand: QMA.BILL.RATED.D ; QMA.BILL.EXTRACT(0) ; two procedures on the path; SYSOUT is DUMMY . ETC's expanded view is expected to show one declared step; effective steps SORT then RATE ; the substituted data set names (QMA.BILL.EXTRACT , QMA.BILL.RATED.D); and the SYSOUT override applied (layer B; Layer C: no case-study run is recorded in this edition).	"The override silences a listing nobody wants." " QMARATE is the rating application."

Level 3 — Architecture reasoning

Three views, one of them absent

View	What it is	Evidence kind	In this edition
Declared text	The steps as the member wrote them	Observed text	Present
Expanded / effective analysis view	The same text with procedures expanded, symbols substituted and overrides applied	Deterministic derivation from text	Present, from the author's rules and from ETC's tests
Run-time	What the system did on a given night	Observation	Absent. No log or listing of an actual night exists

Moving from the first row to the second adds derivation. Moving to the third adds observation, which a text can never supply. The platform's own expanded listing is a second-row artifact.

ETC Lesson Learned — expansion must not erase the invocation

ETC keeps the invocation next to its expansion rather than replacing it. The reason shows in the override case. An override card that names no step of the procedure cannot be attached to any expanded step. If the invocation were replaced by its expansion, that card would appear in neither view. So the invocation stays, and it accounts for what the expansion could not place. The lesson is an engineering one: a derived view should lose nothing the source stated.

Failure Case — an unresolved procedure

Hypothetical. Suppose a bundle holds `QMABL020` but not the `QMARATE` member. ETC keeps the declared step, records that the procedure was not supplied, and issues a warning. It does not guess a procedure, and it does not drop the step. What the readings of the expanded view then do with such a step is not established by the tests.

The six outcomes are different answers and call for different actions.

Outcome	Meaning	Action
not supplied	A determinate name the bundle does not hold	Ask for the member
reference unresolved	The reference is still a symbol, so there is no name to look for. The bundle may hold the procedure	Find what the symbol stands for
platform-declared	A curated catalogue names this exact name	Nothing to send; a boundary of the estate
cycle	The procedure is already being expanded above this step	Fix the library
too deep	The path reached ETC's limit	See the nesting limit below
resolved	The procedure was found and expanded	—

Reporting them as one "missing" outcome would hide which of these a person must act on. That is why ETC reports them distinctly.

Failure Case and Architect's Warning — the DD override

Overrides decide which data a job touches. Procedures are written against ddnames and the job says which data sets they mean this time. The warning is about how ETC replaces a card and how IBM merges one.

- **PLATFORM RULE.** IBM merges the overriding parameters into the procedure's `DD` statement. Parameters coded on the override replace matching ones. The rest remain ([B1]).
- **CURRENT ETC BEHAVIOR.** The reviewed path replaces the whole `DD` card of the same name with the card read from the job. Whatever the procedure card said and the override did not repeat is not carried over.
- **TEST COVERAGE LIMITATION.** The conformance fixture for overrides codes full replacement cards, naming the data set and the disposition. For a full card, merging and replacing give the same result, so the existing test cannot tell them apart.

```
[PSEUDOCODE] generic names, not a Quillon member, not ETC code
procedure card:  //OUT      DD DSN=APP.RESULT,DISP=(NEW,CATLG,DELETE)
override card:   //S1.OUT   DD DSN=APP.OTHER

IBM merge:       DSN=APP.OTHER,DISP=(NEW,CATLG,DELETE)
ETC replacement: DSN=APP.OTHER                      (the card as written)
```

The two results differ the moment an override codes less than the full card. No test exercises that case. In Quillon's `QMABL020`, the override is `DUMMY`, which nullifies the procedure card on the platform and replaces it in ETC, so the two are expected to agree there (an expectation, not an ETC result). That agreement would be a property of that card, not of ETC's override logic.

ETC does not merge parameters as IBM does. The *warning* is for the reader: do not infer that an override-bearing job has been read correctly because the fixture passes.

The nesting limit

Statement	Status
IBM: procedures can invoke procedures, up to 15 levels ([B1])	Platform, sourced
How IBM counts levels; what happens at a sixteenth	Not stated in the recorded text
ETC refuses a procedure when 15 are on the path	Code, read directly
ETC's refusal works	Tested, with the limit set to 1
ETC's refusal falls exactly at 15	Not tested
ETC counts levels as IBM does	Not established

The number 15 appearing on both sides is a coincidence of values until someone tests the boundary or settles the counting. Quillon's `QMABL020` has two procedures on its path. It says nothing about either boundary.

Cycles are ETC's, not the platform's

ETC detects a procedure already on its own invocation path, stops there, keeps what it read and reports the path. That is the whole claim. Whether and how the platform rejects a cycle is a source gap, and the book leaves it open.

Architect's Warning

The most tempting misreading of this chapter is that the expanded view is "what runs". It is derived from text, using rules that ETC implements for the conformance cases and not for all of JCL. A fixture that passes shows the rule works on the fixture.

Claim Check

"ETC expands procedures the way the system does, so every override is read correctly."

This fails. ETC replaces a whole card where IBM merges parameters, and the fixture does not tell the two apart. The expansion is also text-derived and checked only on the conformance cases.

Quick Recall

1. Define Declared Step and Effective Step. Which one is observed at run time?
2. What is the search order for a procedure on z/OS?
3. What do the three limits of 15 in IBM's documentation count?
4. `//RATE.SYSOUT DD DUMMY`: what does IBM do with it, and what does ETC do?
5. Name the three consumers ETC's tests check against the expanded view.

Key Takeaways

- A procedure is a form with blanks. Its text, its expansion and a run are three different things.
- An Effective Step is derived from text. It is not observed.
- IBM merges override parameters. ETC replaces the whole card, and the existing test cannot tell the two apart.
- IBM's 15-level figure is sourced. ETC's boundary at 15 is untested, and ETC's level counting is not established as IBM's.
- Cycles are taught as ETC behavior only.
- ETC's tests check three consumers of the expanded view. Claim no others.
- Quillon's expected expansion is a layer-B expectation until the bundle is run.

Chapter 7 — DD Statements and the Five Axes of a Data Store

After this chapter you can:

- split one `DD` statement into five independent questions: identity, kind, DD semantic role, lifecycle and access mode;
- explain why `DISP` and the COBOL `OPEN` mode answer different questions, and what ETC does with each;

- say what the platform does with an omitted `DISP`, what ETC records instead, and what neither establishes;
- name the bundle-wide rule ETC uses to fill in a store's kind, and say where it can pick the wrong answer.

Chapter 5 read `DD` cards as evidence of a request. Chapter 6 showed that the cards ETC reads come from the effective steps. This chapter asks what one recorded `DD` card says about a data store. Two questions stay apart. *What does the platform do?* is platform truth, from IBM's documentation. *What does ETC record?* is implementation truth, from the code and its tests at the baseline.

Level 1 — Intuition

One label, five questions

A parcel at a depot carries a label. Five different questions can be asked about it, and the answer to one tells you little about the others.

Question about the parcel	The same question about a <code>DD</code>	Axis
What is it called?	Which data store does the card refer to?	Store Identity
What kind of thing is it?	Is it a sequential file, a keyed file, a generation group, a temporary file?	Store Kind
What is it doing in this delivery?	What does this step use the <code>DD</code> for: input, output, work, allocation?	DD semantic role / direction
What happens to it over time?	Is it created, kept, catalogued, deleted, passed on?	Lifecycle
How is it being handled right now?	Which way does the EvidenceGraph say data moves?	Access Mode

A `DD` statement carries evidence for all five at once, in different parts of the card and, for one of them, outside the card. ETC answers each question from its own source and records the answers separately. A reader who merges them draws conclusions that none of the five supports.

FIG-07-1 — The five axes of a data store · AUTHOR-CONCEPTUAL

#	Axis	Question	Where ETC reads the answer	Values the chapter names
1	Store Identity	Which store does the DD refer to?	The <code>DSN=</code> string, with a trailing numeric generation offset removed	The name; an identity gap where a name is unresolved
2	Store Kind	What sort of store is it?	JCL evidence first. Where that is silent, the COBOL file-control organisation declared for the same DD name anywhere in the bundle	Sequential, keyed, generation group, temporary file, <code>UNKNOWN</code>
3	DD semantic role / direction	What does the step use this DD for?	The program the step runs, its documented DD table, then reserved DD names. It never reads <code>DISP</code>	INPUT, OUTPUT, WORK, CONTROL, MESSAGE, ALLOCATION, LIBRARY, <code>UNKNOWN</code>
4	Lifecycle	Created, kept, catalogued, deleted, passed?	The three <code>DISP</code> subparameters	The coded values, <code>UNSPECIFIED</code> , <code>UNKNOWN</code>
5	Access Mode	Which way does the EvidenceGraph say data moves?	The <code>DISP</code> status, withdrawn to <code>NONE</code> for an allocation-only DD. The <code>DISP</code> mapping is shown below. The withdrawal is applied by the DD - semantics rule	READ, WRITE, UPDATE, APPEND, NONE, <code>UNKNOWN</code>

What FIG-07-1 does not imply. ETC does not resolve identity through a catalog. `DSNAME=` is not read. `DISP` and the COBOL `OPEN` mode are not merged; the `OPEN` mode is not an axis and is recorded on the program. The value lists are not exhaustive. The role a store plays across the whole estate is not an axis here: that is Store Role, and it belongs to Chapter 12.

Row 3 rests on a rule the author found by reading the code, not on documentation. The chapter states the axis's values and shows its mechanism only as pseudocode.

The axes vary independently

```
[PSEUDOCODE] generic JCL, not a Quillon member
//TMPA DD DSN=&&WORK1,DISP=(NEW,PASS)
//MASTER DD DSN=PROD.CUST.MASTER,DISP=SHR
//DAILY DD DSN=PROD.CUST.DAILY(+1),
// DISP=(NEW,CATLG,DELETE)
```

Read by the rules this chapter describes (the author's reading, not an ETC run), with the role axis left out because it depends on the program:

DD	Identity	Kind from the card	Lifecycle	Access Mode
TMPA	&&WORK1	Temporary file	NEW , PASS , abnormal end UNSPECIFIED	WRITE
MASTER	PROD.CUST.MASTER	UNKNOWN until a program declares it	SHR , both ends UNSPECIFIED	READ
DAILY	PROD.CUST.DAILY	Generation group	NEW , CATLG , DELETE	WRITE

TMPA and DAILY are both written and differ on every other axis. MASTER is read tonight and could be updated tomorrow by another job without any axis but the third and fifth changing. No axis predicts another.

Level 2 — Engineering mechanics

Part A. What the platform defines

Everything in this part is platform truth ([B1], [B2], [B3], [B4], [B8]). Chapter 5 taught DISP and generations. This table adds only what the chapter needs.

Topic	What the platform says
Scope of a DD	A DD statement belongs to the job step it follows. Nothing in IBM's documentation makes a DD name identify one data set across steps or jobs.
DISP against OPEN	DISP concerns allocation, sharing and disposition at the step boundary. OPEN expresses the program's access intent at run time. DISP=SHR does not prove read-only intent.
OPEN modes	INPUT , OUTPUT , I-O and EXTEND decide which I/O statements the program may use.
ASSIGN TO	The assignment name in SELECT ... ASSIGN TO may correspond to a DD name, or to another run-time assignment mechanism such as an environment variable.
File organisation	INDEXED corresponds to a VSAM key-sequenced data set, RELATIVE to a relative-record data set, SEQUENTIAL to a QSAM data set or an entry-sequenced data set.
Temporary data sets	A temporary data set has a && name or no DSN. If allocation succeeds and the disposition is DELETE , the disposition applies when the step or job terminates. Retention and lifecycle conditions may matter.

Part B. What ETC records, axis by axis

From here the chapter describes ETC at the baseline (code and tests rechecked on 2026-10-05).

Axis 1. Store Identity

The identity of a store is the `DSN=` operand as written, with a trailing numeric generation offset `((0), (+1), (-1))` removed. A job that writes `(+1)` and a job that reads `(0)` therefore name one store. Nothing else is normalised. ETC does not consult a catalog, does not canonicalise a PDS member name (`QMA.CNTL(SORTPOL)` keeps `(SORTPOL)`), a VSAM component or an alias. Its temporary flag is set only for a `&&` name. A `DD` with no `DSN`, a `DUMMY` or a `SYSOUT` carries no store identity.

CURRENT IMPLEMENTATION LIMITATION. ETC looks for `DSN=` only. A `DD` coded with the long form `DSNAME=` gets no store identity at the baseline, so no data flow is inferred for it, and a `DSNAME=&&...` name is not marked temporary. The pinned estates contain `DD` statements written this way. This is not a statement that the platform treats the two keywords differently.

Axis 2. Store Kind

The job's own evidence comes first: a reserved `DD` name (`SYSIN` is control input, `STEPLIB` a program library, `SYSOUT` a print sink), a `SORTWK` work-file name, a `&&` name, a trailing generation offset (seen before it is removed), a tape unit, or an unnamed `DD` with a `SPACE` allocation. Anything left is `UNKNOWN`. Three of these kinds (control input, program library, print sink) say what a `DD` is for rather than what shape the store has, so the kind axis and the role axis can both describe the same `DD`.

ETC records `INDEXED` and `RELATIVE` as a keyed file and `SEQUENTIAL` as a sequential file. An entry-sequenced VSAM data set declared `SEQUENTIAL` is not told apart from a QSAM data set.

Only `UNKNOWN` kinds are filled from COBOL, and a kind the job established is never overwritten. How the fill works, and where it goes wrong, has its own section below.

Axis 3. DD semantic role / direction

The role says what one step uses one `DD` for. ETC stores it as the `DD`'s semantic role, resolved from the program the step runs, never from `DISP`.

The rule is shown as pseudocode, not as an excerpt.

```
[PSEUDOCODE]  role of one DD in one step (the rule, not the file)
if the step's program is unknown:      UNKNOWN
else if the program only allocates:    ALLOCATION
else if the program has a DD table
    and the table covers this DD:      the table's role
else if the DD name is reserved:      the reserved role
else:                                  UNKNOWN
```

An application program with no documented table gets `UNKNOWN` for its `DD`s. That is a gap in the reading, not a finding about the file.

Axes 4 and 5. Lifecycle and Access Mode

Both come from the same `DISP` text, and ETC keeps them as two answers. The lifecycle records what the card asked of the system: status, normal end, abnormal end. The access mode is ETC's derivation of a direction from the status.

```
[PSEUDOCODE – PUBLIC EDITION] from the DISP status to an access mode
status-to-mode: NEW -> WRITE, MOD -> APPEND, OLD -> UPDATE, SHR -> READ

mode_from_status(disposition):
  if there is no DISP text:          return UNKNOWN
  status = the first subparameter, trimmed and upper-cased
  if status is empty:                return WRITE    # as in DISP=(,CATLG)
  return status-to-mode[status] if it is listed, otherwise UNKNOWN
```

Simplified. Explanatory comments are left out. The pseudocode adds no behavior.

```
[PSEUDOCODE – PUBLIC EDITION] the three DISP subparameters
parts = the DISP subparameters, each trimmed and upper-cased

status = NEW if the first part is empty,
        otherwise the recognised status, or UNKNOWN
disposition_at(i) = UNSPECIFIED if part i is missing or empty,
                   otherwise the recognised disposition, or UNKNOWN

lifecycle = { declared text, status,
              on normal end  = disposition_at(1),
              on abnormal end = disposition_at(2) }
```

Simplified. The early return for a missing `DISP` is left out here.

DISP as coded	Access Mode	Status	Normal end	Abnormal end	Warning
SHR	READ	SHR	UNSPECIFIED	UNSPECIFIED	no
(NEW,CATLG,DELETE)	WRITE	NEW	CATLG	DELETE	no
(,CATLG)	WRITE	NEW	CATLG	UNSPECIFIED	no
MOD	APPEND	MOD	UNSPECIFIED	UNSPECIFIED	no
(XYZ,KEEP)	UNKNOWN	UNKNOWN	KEEP	UNSPECIFIED	yes
none	UNKNOWN	UNSPECIFIED	UNSPECIFIED	UNSPECIFIED	yes

A status of `OLD` gives UPDATE. The table is the author's reading of the implementation, checked against probe results on 2026-10-05. The pseudocode does not show everything in it. The lifecycle of the "none" row comes from the early return for a missing `DISP`, which the pseudocode leaves out. The Warning column comes from a separate rule: a warning is raised when a named store's Access Mode is `UNKNOWN`, and the pseudocode does not show it. Both rest

on a reading of the implementation, not on the pseudocode itself. The platform documents a normal-end default of `DELETE` for a new data set and `KEEP` for an existing one. ETC does not apply it: an uncoded normal or abnormal disposition is recorded as `UNSPECIFIED`.

`DISP` and `OPEN`: two evidence dimensions

Platform: `DISP` concerns allocation, sharing and disposition at the step boundary, and `OPEN` expresses the program's access intent at run time. *ETC:* the EvidenceGraph takes direction from `DISP`; the program's `OPEN` mode is recorded separately on the program; the two are not reconciled. A READ or WRITE in the EvidenceGraph is a derivation from JCL. It is never proven COBOL I/O.

FIG-07-2 — `DISP` against `OPEN` · AUTHOR-CONCEPTUAL and SOURCE-DERIVED

	JCL lane	Program lane
Artifact (layer A)	<code>//POLMAST DD DSN=...,DISP=SHR</code> in step <code>RATE</code>	<code>QMARTE01</code> opens <code>POLICY-FILE</code> with <code>OPEN I-O</code> and rewrites records
Platform meaning	Allocation and sharing at the step boundary	The program's access intent at run time
ETC representation, Layer B / EXPECTED (layer C is <code>UNKNOWN</code>)	Layer B / EXPECTED: Access Mode READ, from <code>DISP=SHR</code> under the current rule (Layer C: no case-study run is recorded in this edition)	Layer B / EXPECTED: the open mode <code>I-O</code> , held on the program under the current rule (Layer C: no case-study run is recorded in this edition)
Relation	Not reconciled	Not reconciled
Other ETC consumers	How other ETC consumers use <code>OPEN</code> : not established for the book.	

What FIG-07-2 does not imply. The EvidenceGraph's READ is not proof that `QMARTE01` only reads. The two lanes are not merged anywhere this chapter describes. `DISP=SHR` is not read-only intent.

Omitted `DISP` (FIG-07-3)

CURRENT ETC BEHAVIOUR, COMPARED WITH THE PLATFORM. IBM defines a platform default of `DISP=(NEW,DELETE,DELETE)` when `DISP` is omitted. Current ETC behaviour preserves `UNSPECIFIED / UNKNOWN` in the reviewed analysis path and withholds the corresponding data-flow inference.

FIG-07-3 — Three cases of a missing `DISP` · SOURCE-DERIVED and AUTHOR-CONCEPTUAL

Case	Platform ([B1])	ETC
<code>DISP</code> omitted entirely: <code>//AUDIT DD</code> <code>DSN=QMA.BILL.AUDIT</code>	The default is <code>(NEW,DELETE,DELETE)</code> . If allocation succeeds, <code>DELETE</code> applies at termination; retention and lifecycle conditions may matter	Access <code>UNKNOWN</code> , lifecycle status <code>UNSPECIFIED</code> , the <i>disposition absent</i> warning, no data flow
Status omitted inside a coded <code>DISP</code> : <code>DISP=</code> <code>(,CATLG)</code>	An omitted status means <code>NEW</code>	Status <code>NEW</code> , access <code>WRITE</code> : the same reading on both sides
Unrecognised status: <code>DISP=</code> <code>(XYZ,KEEP)</code>	The four statuses are <code>NEW</code> , <code>OLD</code> , <code>SHR</code> and <code>MOD</code>	Access <code>UNKNOWN</code> , lifecycle status <code>UNKNOWN</code> , the same warning

Compared by outcome, ETC's behaviour is more conservative than applying the platform default: applying it would give the store status `NEW` , a deletion at both ends and, through ETC's own mapping, a `WRITE` access, and ETC records none of these. The reason for this behaviour is not established at this baseline.

CURRENT IMPLEMENTATION DIAGNOSTIC LIMITATION. the *disposition absent* warning is raised when the access mode could not be determined from `DISP` for a `DD` that names a store. Its text says the store has "no disposition". ETC emits that same text for a coded but unrecognised status. The wording describes the implementation's diagnostic. It is not the card and it is not platform truth. A `DD` with no store name raises no such warning.

`NONE` , `UNKNOWN` , `UNSPECIFIED` and not supplied

Term	Meaning	Example
<code>UNKNOWN</code>	The reading could not determine the value: a gap	Access Mode when <code>DISP</code> is missing
<code>UNSPECIFIED</code>	The card did not write the field, and ETC applies no default	Normal end in <code>DISP=SHR</code>
<code>NONE</code>	A known absence of access. ETC produces it only for an allocation-only <code>DD</code> whose <code>DISP</code> gave <code>APPEND</code> or <code>UPDATE</code>	A utility step deleting a data set it never reads
not supplied	The artifact itself is not in the bundle	A procedure no library holds (Chapter 6)

```
[PSEUDOCODE] generic JCL, not a Quillon member
//CLEAN      EXEC PGM=IEFBR14
//OLDFILE    DD DSN=PROD.OLD.DATA,DISP=(OLD,DELETE)
//NEWFILE    DD DSN=PROD.NEW.DATA,DISP=(NEW,CATLG),
//           SPACE=(TRK,1)
```

`DISP` is processed at the step boundary, so a step whose program does nothing with a file can still delete or create it. Read by the rules above (the author's reading, not an ETC run): `OLDFILE` has status `OLD`, which gives UPDATE, and the role is ALLOCATION, so the access is withdrawn to `NONE`. `NEWFILE` keeps WRITE. No Quillon step runs an allocation-only program, so Quillon does not illustrate `NONE`.

Temporary and Persistent, in ETC's sense

Temporary (ETC) means a `DD` whose data set name begins `&&`. **Persistent (ETC)** means a `DD` with a data set name that is not `&&`. Neither is a model of z/OS persistence, and neither depends on `CATLG` or `KEEP`. A named data set coded `DISP=(NEW,DELETE,DELETE)` is persistent in ETC's sense. The store kind `TEMPORARY_FILE` is separate and wider: ETC also assigns it to `SORTWK` work files and to unnamed `DD`s with a space allocation. ETC uses the flags to decide which accesses can carry state from one job to another.

The bundle-wide DD-name join

Platform: a `DD` statement belongs to its step. *ETC:* the kind refinement looks the `DD` name up across the whole bundle.

```
[PSEUDOCODE] fill a store's kind from COBOL (the rule, not the file)
declared = {}
for each program, in bundle reading order:
    for each SELECT clause of the program:
        if the DD name is not yet in declared:
            declared[DD name] = the file organisation

for each access in every effective step of every job:
    if the access's kind is UNKNOWN and it has a store name:
        kind = organisation_to_kind(declared.get(DD name)) # else UNKNOWN
```

The key is the `ASSIGN TO` name compared with the JCL `DD` name. ETC takes the assignment name as written, and this chapter does not claim it handles every form an assignment name can take. The reading order is the sorted `cobol/` members, then members rerouted from `asm/`, then assembler units. The rule keeps the first value. A `DD` name that no program declares stays `UNKNOWN` and is reported at INFO level.

```
[PSEUDOCODE] generic, author-created; not an estate, not a test
cobl/PGMA.cbl  SELECT CUSTFILE ASSIGN TO CUSTIN
                ORGANIZATION IS SEQUENTIAL
cobl/PGMB.cbl  SELECT CUSTFILE ASSIGN TO CUSTIN
                ORGANIZATION IS INDEXED
jcl/JOBX.jcl    //STEP1 EXEC PGM=PGMB
                //CUSTIN DD DSN=PROD.CUST.MASTER,DISP=SHR
```

FIG-07-4 — A bundle-wide join · AUTHOR-CONCEPTUAL

Reading order	Program	Declares CUSTIN as	Run by STEP1 ?
1 (first match in bundle order)	PGMA	Sequential	No
2	PGMB	Indexed	Yes
Result by the rule	STEP1 's CUSTIN	Sequential file , taken from PGMA	

Labels: no conflict reported; no regression test for conflicts; current ETC heuristic / limitation.

Caption. The pair is hypothetical and author-created. It is not an ETC run result and it is not test evidence. It does not show resolution per step, and it says nothing about Quillon.

CURRENT ETC HEURISTIC / LIMITATION. The join is on the DD name across the whole bundle. If more than one program declares that DD name, the first declaration in the bundle's reading order sets the kind, even when it comes from a program the step does not run, and no conflict is reported. No regression test covers conflicting declarations. The review saw the same behaviour once, on a different synthetic pair, with a throw-away probe on 2026-10-05. A probe is not a product test.

The test that does exist covers the single-declaration case: a keyed organisation fills a keyed kind, for a DD in a job step and for a DD reached through a procedure. It cannot tell the reader what happens when two programs disagree.

How the EvidenceGraph settles and records a store kind

ETC reads the refined kinds into the EvidenceGraph, which holds exactly one kind per store.

When the accesses report more than one kind for the same store, a fixed precedence picks one.

```
[PSEUDOCODE – PUBLIC EDITION] the graph's rule: one kind per store, by precedence
precedence = DB2 table, then keyed file (VSAM), then plain data set

observed[store] = the set of kinds seen for the store, from
    every access, in every step that carries accesses, whose store has a name,
    and every table declaration (kind: DB2 table)

for each store:
    kind = the first kind in precedence that is in observed[store]
```

Simplified. Comments are left out. The pseudocode adds no behavior.

For a store seen as both a keyed file and a plain data set, the EvidenceGraph settles on the VSAM file kind because it is the most specific. The settled kind is one answer per store, chosen by precedence. It is not proof that the other observation was wrong. The graph reads the refined kinds, so the limitation of the bundle-wide join above applies to what reaches it. A kind the graph holds is evidence-backed, and it is not a statement about how the store is used in production.

Quillon's data stores

```
[CASE-STUDY ARTIFACT] QMARATE and QMASORT (DD cards only)
//QMARATE  PROC HLQ=QMA,CYCLE=D
//SORTP    EXEC QMASORT,IN=&HLQ..BILL.EXTRACT(0)
//RATE     EXEC PGM=QMARTE01
//SORTED   DD DSN=&&SORTED,DISP=(OLD,DELETE)
//POLMAST  DD DSN=&HLQ..POLICY.MASTER,DISP=SHR
//AUDIT    DD DSN=&HLQ..BILL.AUDIT
//QMASORT  PROC IN=
//SORT     EXEC PGM=SORT
//SORTIN   DD DSN=&IN,DISP=SHR
//SORTOUT  DD DSN=&&SORTED,DISP=(NEW,PASS)
//SYSIN    DD DSN=QMA.CNTL(SORTPOL),DISP=SHR
```

The ETC columns are an expected pedagogical outcome of an author-designed scenario, not an ETC result (Layer C: no case-study run is recorded in this edition). Layer C is UNKNOWN for every row.

Store	Observed (layer A)	Expected derivation (layer B)	Interpreted
POLMAST	DISP=SHR in RATE . QMARTE01 opens POLICY-FILE I-O and rewrites	Access READ from DISP ; open mode I- O recorded on the program. Kind: keyed file, from the ORGANIZATION INDEXED that QMAEXT01 declares, a program RATE does not run. A single declaration, so no conflict is involved	"The SHR may be a leftover." A reading, not in the artifacts
AUDIT	No DISP. QMARTE01 opens AUDIT-FILE with EXTEND	Access UNKNOWN , lifecycle UNSPECIFIED , the <i>disposition absent</i> warning raised, no data flow. The warning's text is not predicted	Below, in the Failure Case
&&SORTED	(NEW,PASS) in SORT , (OLD,DELETE) in RATE	Temporary (ETC), not persistent; kind temporary file; WRITE in SORT , UPDATE in RATE	—
BILL.EXTRACT(0)	A generation group, read by SORTIN through the symbol IN	One store, QMA.BILL.EXTRACT , whatever the generation offset; kind generation group	—
SYSIN	QMA.CNTL(SORTPOL)	Identity keeps (SORTPOL) ; kind control input, by the reserved DD name	"Control text, not application data."

Current Support

Capability	Status	Evidence
Read DISP into a lifecycle and an access mode	IMPLEMENTED	Reviewed at the baseline
Take identity from DSN= with the generation offset removed	IMPLEMENTED	Reviewed at the baseline
Fill an unknown kind from COBOL organisation by DD name, across the bundle	IMPLEMENTED, as a heuristic	direct test of the single- declaration case, job and procedure routes
Record the OPEN mode on the program, apart from the DD access mode	IMPLEMENTED	Rechecked 2026-10-05
Resolve a DD semantic role	IMPLEMENTED, mechanism found by code review	Reviewed at the baseline

Current Limitation

CURRENT IMPLEMENTATION LIMITATION. These statements describe the reviewed reading at the baseline. The review records no intent behind them.

1. **The kind join is bundle-wide and first-match.** It can take a kind from a program the step does not run, reports no conflict, and has no conflicting-declaration test.
2. **DSNAME= is not read.** Such a **DD** has no store identity and no data flow.
3. **An uncoded normal or abnormal disposition is UNSPECIFIED.** ETC applies no platform default to it.
4. **A missing or unrecognised DISP yields unknown access and no data flow,** and the warning's "no disposition" wording covers both. The reason for the behaviour is not established at this baseline.
5. **Identity is not catalog-resolved.** Members, VSAM components and aliases are not canonicalised. A store name that still holds an unresolved symbol is reported as an identity gap.
6. **Graph direction is a derivation from DISP.** It is not proven program I/O.
7. **An entry-sequenced VSAM data set declared SEQUENTIAL** is not told apart from a QSAM data set.
8. **One kind per store is settled by precedence.** A store observed with more than one kind is recorded with the most specific, and the record does not show that the other observation was wrong.

Code Trace Challenge — the join

Use the **PGMA**, **PGMB** and **JOBX** listing above, with one change: **PGMA** declares **INDEXED** and **PGMB** declares **SEQUENTIAL**. **STEP1** still runs **PGMB**.

a. What kind does **CUSTIN** get, and from which program? b. Does **STEP1** run that program? c. Is a conflict reported? d. A second **DD**, **//OTHER DD DSN=PROD.UNDECLARED.FILE**, appears in **STEP1**. No program assigns to **OTHER**. What happens?

Answer (by the rule, on an author-created pair; not an ETC run). (a) A keyed file, from **PGMA**, which sorts first. (b) No. **STEP1** runs **PGMB**, which declares a sequential file. (c) No. (d) The kind stays **UNKNOWN** and an INFO issue reports that no program declares the organisation.

Level 3 — Architecture reasoning

Why five axes and not one "type"

A single type column would have to say **TMPA** is "a temporary output", **MASTER** is "a shared keyed input" and **DAILY** is "a new generation". Each phrase mixes axes whose evidence comes from different places: the card, a COBOL program, a documented utility table. When the sources

disagree, a merged column hides the disagreement. Five axes keep each answer next to its source, so a reader can see which is `UNKNOWN`.

Failure Case — a missing edge that looks like a missing dependency

`AUDIT` has no `DISP`. *Platform*: IBM defines a platform default of `DISP=(NEW,DELETE,DELETE)`. On z/OS the omitted `DISP` would default to it; if the allocation succeeded, the data set would be deleted at the end of the step. Whether that is a latent bug is an interpretation, and the owner of `QMARATE` is the one who can confirm it. *ETC*: the access is `UNKNOWN`, the lifecycle `UNSPECIFIED`, and no data flow is inferred.

The failure is a reading error. A graph with no edge between `RATE` and any other reader of `AUDIT` invites the conclusion that no job depends on `AUDIT`. The correct reading is that the access mode could not be determined from `DISP`: that is `UNKNOWN`, not `NONE`. Missing evidence is not evidence of absence.

Architect's Warning

The most likely misreading is to take a `READ` or `WRITE` in the `EvidenceGraph` as what the program does. It is what the `DISP` card asked for, mapped to a direction. `POLMAST` is `SHR` in the JCL and `I-O` in the program. A design that treats it as read-only on the strength of the graph would be wrong for the artifacts in this chapter. Cite the `OPEN` mode when the claim is about the program.

Claim Check

"ETC reconciles each DD with the program the step runs, so its store kinds are per step."

This fails twice. The kind join is by DD name across the whole bundle, and the first declaration in reading order wins even when the step does not run that program. And the `OPEN` mode is recorded on the program, not reconciled with the DD's `DISP`-derived access mode.

Quick Recall

1. Name the five axes in order, and say which one never reads `DISP`.
2. What does `DISP=SHR` fail to prove about a program's access?
3. For `DISP=(,CATLG)`, what status, access mode and abnormal-end disposition does ETC record?
4. State the platform's default for an omitted `DISP` and ETC's record of it, and say whether the chapter explains why they differ.
5. If two programs declare one DD name differently, which declaration sets the kind, and is a conflict reported?

Key Takeaways

- A `DD` carries evidence for five independent questions. ETC records them as five answers: identity, kind, DD semantic role, lifecycle and access mode.
- `DISP` is allocation and disposition. `OPEN` is the program's intent. ETC derives direction from `DISP`, records `OPEN` on the program, and does not reconcile them.
- Omitted `DISP` is a platform default on one side and `UNSPECIFIED`/`UNKNOWN` with no data flow on the other. The reason is not established at this baseline.
- `UNKNOWN`, `UNSPECIFIED`, `NONE` and not supplied are four different statements.
- Identity is the `DSN=` string without a numeric generation offset. `DSNAME=` is not read. Nothing is catalog-resolved.
- Kind refinement joins by DD name across the whole bundle and the first match wins, even from a program the step does not run. That is a current ETC heuristic and limitation, with no conflict report and no regression test.
- Quillon's data stores are layer-B expectations until the bundle is run (Layer C: no case-study run is recorded in this edition).

Chapter 8 — COBOL Programs, Copybooks and Artifact Classification

After this chapter you can:

- name the COBOL-side artifacts a bundle holds (program units, copybooks, embedded SQL, CICS and BMS members) and say where `SELECT ... ASSIGN` fits;
- keep three questions apart: where a member sits, what it is, and whether it is a translation input;
- explain why the three reference estates have 106 COBOL program units and 105 COBOL translation members, and why both numbers are correct;
- tell the `BANKDATA` story without blaming the upstream project and without saying a program was skipped.

Chapter 7 followed a DD name into a program's `SELECT ... ASSIGN TO`. This chapter steps back and asks what kind of thing a member is, and who decided. Two questions stay apart. *What does the platform define?* is platform truth. *What does ETC do with a member?* is implementation truth from the code, its tests and the measured bundles.

Level 1 — Intuition

A shelf label is not the book

A library bulk-loads a donated collection. A clerk shelves each volume by a quick look at the cover. Later a reader finds a cookbook under Chemistry. The shelf says where the clerk put the book. It does not say what the book is, and neither says whether the library will lend it out.

A bundle works the same way. Three different questions apply to every member.

Question	Name	Answered by
Where does it sit?	Physical location	the directory and file name in the bundle
What is it?	Semantic type	the language or kind the reader assigns: COBOL, assembler, screen map, unidentified
Will it be translated?	Translation eligibility	the translation-source selector

Physical location $\neq$ semantic type $\neq$ translation eligibility. The location can also be evidence the ingestion pipeline produced. A member in the wrong directory may have been put there by the machinery that built the bundle, long before anyone read it.

A member that carries the wrong label

```
[CASE-STUDY ARTIFACT]  asm/QMAVALID.asm  (opening lines)
    IDENTIFICATION DIVISION.
    PROGRAM-ID.  QMAVALID.
```

This is layer A, the fictional artifact. The fictional shop kept the member among its assembler files. The two lines open a COBOL program. The directory and the text disagree, and the text is the better evidence of what the member is.

Level 2 — Engineering mechanics

Part A. The artifacts on the platform

Platform truth first ([B3], [B4]).

Artifact	What it is	Where this book meets it
COBOL program unit	A source program with four divisions, opening with <code>IDENTIFICATION DIVISION</code> and a <code>PROGRAM-ID</code>	The population counted below
<code>FILE-CONTROL</code> and <code>SELECT ... ASSIGN TO</code>	Environment-division entries that declare each file, its organisation and its assignment name. The name may correspond to a DD name, or to another run-time assignment mechanism such as an environment variable	The join key of Chapter 7
Copybook	Text brought into a program by <code>COPY</code> . It holds data description entries (level numbers) or procedural statements	Read by copybook readers, not counted as programs
Embedded SQL	<code>EXEC SQL ... END-EXEC</code> inside a program; a <code>DECLARE TABLE</code> with host variables is the usual declaration member	Named so the reader recognises the member
CICS command	<code>EXEC CICS ... END-EXEC</code> inside a program	Named so the reader recognises the member

A BMS member is a screen-map definition, and ETC recognises one by the macro `DFHMSD`. IBM's language reference makes `IDENTIFICATION DIVISION` the required opening of a COBOL source program. It says nothing that would make a file with an `.asm` name and that opening an assembler member. This chapter makes no claim about ETC's support for SQL or CICS.

Part B. Who decides what, at the baseline

From here the chapter describes ETC at the baseline (code and tests rechecked on 2026-10-05, measurements dated 2026-10-04). There is no single classifier for the whole bundle. Different mechanisms answer different questions.

Mechanism	What it decides	Applies to
The directory	Which reader opens a member	<code>cobol/</code> , <code>copybook/</code> , <code>screen/</code> , <code>scheduler/</code>
Content classification	What a member is	members of <code>asm/</code>
A job parse, then a procedure parse	Whether a member is a job or a procedure; a member neither reads is reported the <i>unreadable job</i> issue	<code>jcl/</code>
Translation-source selection	Which members are translation inputs	<code>cobol/*.cbl</code>

The bundle reader classifies each member of `asm/` by its content. Members of the other reader directories are read by the reader their directory selects. Assembler content becomes an assembler program unit. COBOL content (a `PROGRAM-ID`) is read as a COBOL program unit and reported as misfiled. A BMS map set (`DFHMSD`) is passed to the screen reader. A member the rules cannot identify is reported unidentified and is not read. Translation sources are selected separately: only `cobol/*.cbl` members are considered.

Pseudocode — two mechanisms, two pieces of pseudocode

```
[PSEUDOCODE – PUBLIC EDITION]  classifying one member: the rule order
if the text is empty:              UNCLASSIFIED
if the member is schedule evidence: SCHEDULE
if the file suffix is off-platform: OFF_PLATFORM
for each line:
    if it is a job card:            JOB
    if it is a procedure card:      PROCEDURE
if any off-platform pattern matches: OFF_PLATFORM
for each line:
    if it is an assembler section statement
        and the line does not contain PROGRAM-ID:  FOREIGN
for each line:
    if PROGRAM-ID appears in the COBOL area-B columns:  PROGRAM
for each line:
    if a screen-map marker appears in area B:          SCREEN
```

Simplified. The rules after the screen test, in order: a `DD` or `EXEC` card with no job card (`PROCEDURE`); a `DECLARE TABLE` (`DECLARATION`); a level number (`COPYBOOK`); a COBOL statement (`COPYBOOK`); otherwise `UNCLASSIFIED`. Also left out: the evidence text each classification carries. The loop over off-platform patterns is condensed to one line. The order shown is the order ETC applies. The pseudocode adds no behavior.

The assembler-statement test reads a statement whose name field, if present, is a name, followed by one of the opcodes `CSECT`, `START` or `DFHEIENT` that ends at a blank or the end of the line. It is checked before `PROGRAM-ID`, and a line that itself contains `PROGRAM-ID` does not count.

```
[PSEUDOCODE – PUBLIC EDITION]  which members are translation inputs
translation inputs = the members of the bundle's COBOL directory
                        with the .cbl suffix, sorted by name
```

Simplified. The measurement scripts count the two populations separately: translation members are the `cobol/` members the selector would consider, and program units are the programs the bundle reader comprehends, split by language.

The three reference estates, with their denominators

Measured at the book baseline over CardDemo, CBSA and GenApp (run on 2026-10-04, rechecked on 2026-10-05): 105 COBOL translation members, 106 COBOL program units, 2 assembler program units.

Population	CardDemo	CBSA	GenApp	Total
COBOL translation members	44	30	31	105
COBOL program units	44	31	31	106
Assembler program units	2	0	0	2
All program units	46	31	31	108

Chapter 2 used 105 as the denominator for the 36 programs ETC emitted. The table says why 105 is the right figure for that, and why 106 is not a count of translation inputs. The one-program gap sits in CBSA.

The **BANKDATA** story

Here is the public wording for the gap.

In the pinned CBSA bundle, one COBOL program sits among the assembler members. ETC's own corpus import step put it there: its import record shows the member came from the CBSA project's COBOL source area, and the classifier used at import time mistook the `START` in the data name `START-KEY` for an assembler statement. That classifier has since been corrected, and the reader now reads the member as COBOL, but the frozen bundle keeps the old placement. The translation-source selector reads only the `cobo1/` directory, so the program is outside the translation denominator. That is why CBSA counts 31 COBOL program units and 30 translation members.

Five separate facts sit inside that paragraph.

Question	Answer	Basis
Upstream, at the pinned commit	<code>src/base/cobol_src/BANKDATA.cb1</code> , a COBOL source file	An independent check of the path and language (2026-10-05)
In the imported bundle	<code>asm/BANKDATA.asm</code>	The bundle; ETC's import record
Why it is there	The classifier in use when the bundle was built (2026-09-22) took <code>START</code> inside the data name <code>START-KEY</code> for an assembler statement and recorded the member as foreign	The import record; the project's history (2026-10-01)
What the reader does today	Reads it as a COBOL program unit and reports the <i>misfiled member</i> issue	Reviewed at the baseline
Whether it is a translation input	No. The selector reads only <code>cobol/</code>	Reviewed at the baseline

The upstream check covers the path and the language only. The cause of the placement is internal evidence. The corrected rule is part of the book baseline, but the bundle is frozen, so it keeps the placement it was built with.

Three statements follow, and none of them contradicts another.

- `BANKDATA` is COBOL. The reader says so, and so does the member's text.
- `BANKDATA` is not a translation member, because its location puts it outside the selector's directory.
- CBSA has 31 COBOL program units and 30 COBOL translation members.

This is not a story of upstream carelessness: the placement was made by ETC's own import step. It is not a skipped program either: the selector never considered the member, so nothing was passed over.

FIG-08-1 — Artifact classification: location, type, eligibility · AUTHOR-CONCEPTUAL

Band 1. Measured evidence (CBSA, pinned bundle).

Member	Directory in the bundle	How it got there	Type the reader assigns today	In the translation denominator?
BANKDATA	asm/BANKDATA.asm	ETC's import step, under the classifier of the time; the classifier was later corrected and the frozen bundle keeps the placement	COBOL program unit	No: the selector reads <code>cobol/</code> only

CBSA counts 31 COBOL program units and 30 translation members. Across the three estates the counts are 106 and 105. Classification and translation selection are two separate mechanisms.

Band 2. Fictional artifact (Quillon, layer A only).

Member	Directory	How it got there
QMAVALID	asm/QMAVALID.asm	The fictional shop kept it among its assembler files
QMADTCV	asm/QMADTCV.asm	A fictional, authored assembler member whose content is HLASM, kept among the assembler files

The bands are never summed. Quillon's expected classification is in the Quillon panel below. *What FIG-08-1 does not imply.* The selector did not skip a program. The CBSA project did not file `BANKDATA` under assembler. ETC did not correct an upstream mistake. A `.asm` suffix or an `asm/` directory does not mean assembler.

The six members of `asm/`

Across the three bundles, six members sit under `asm/`. According to ETC's import record, four of them were placed there by import-time classification.

Estate	Member	What the reader makes of it today
CardDemo	COBDATFT	Assembler program unit
CardDemo	MVSWAIT	Assembler program unit
CBSA	BANKDATA	COBOL program unit, reported as misfiled
CBSA	BNK1DAM	BMS map set, passed to the screen reader
CBSA	DFH\$SIP1	Unidentified; reported and not read
GenApp	SSP4A1	Unidentified; reported and not read

Two assembler, one COBOL, one BMS and two unidentified. The two unidentified members are not assembler. A member the rules cannot identify is not given a language on a guess. The four members that the current rule would not place under `asm/` are `BANKDATA`, `BNK1DAM`, `DFH$SIP1` and `SSP4A1`.

UNIT-TESTED CAPABILITY. If a rerouted member shares a stem with a member already in its destination directory, the competing rerouted duplicate is not read and is reported. A unit test exercises this. It has not been observed in the reference estates, which contain no such report.

Quillon, sorted

Observed (layer A)	Expected derivation (layer B)	Interpreted
<code>asm/QMAVALID.asm</code> opens with <code>IDENTIFICATION DIVISION. PROGRAM-ID.</code> <code>QMAVALID.</code> <code>asm/QMADTCV.asm</code> is HLASM. <code>QMAEXT01</code> declares <code>SELECT POLICY-FILE</code> <code>ASSIGN TO POLMAST ORGANIZATION</code> <code>INDEXED ACCESS SEQUENTIAL</code> and copies <code>QMAPOLCP</code>	<code>QMAVALID</code> is expected to be read as a COBOL program unit and reported as misfiled, and to fall outside the <code>cobol/*.cbl</code> denominator. <code>QMADTCV</code> is expected to be an assembler unit, outside the COBOL denominator. <code>QMAEXT01</code> is the FILE-CONTROL teaching example	"The shop filed <code>QMAVALID</code> by habit or by accident." The bundle does not say which

The layer-B column is an expected pedagogical outcome of an author-designed scenario, not an ETC result (Layer C: no case-study run is recorded in this edition). Layer C is **UNKNOWN**. The cause differs from CBSA's: Quillon's misfiling is the fictional shop's act, and CBSA's placement was made by ETC's import step. The two are not the same cause.

Current Support

Capability	Status	Evidence
Classify each member of <code>asm/</code> by content, and reroute it	IMPLEMENTED	Reviewed at the baseline
Read a COBOL program found under <code>asm/</code> as a COBOL program unit	IMPLEMENTED	measured on CBSA, 2026-10-05
Select translation sources from <code>cobol/*.cbl</code> only	IMPLEMENTED	Reviewed at the baseline

Current Limitation

CURRENT IMPLEMENTATION LIMITATION. These statements describe the baseline. The review records no intent behind them.

1. **The frozen bundles keep import-time placements.** Four members sit where the current classifier would not put them.
2. **There is no universal classifier.** Content classification applies to `asm/`. The `jc1/` split uses a job parse and then a procedure parse. Translation selection uses the directory.
3. **A member the rules cannot identify is reported and not read.** The two unidentified members are in this state.
4. **The duplicate-stem path is unit-tested only.**

5. **Imported placement is evidence of the import, not of the member.** The next chapter returns to this with two name collisions.

Code Trace Challenge — a data name that looks like an opcode

```
[PSEUDOCODE] generic member GENCOB.asm in asm/, author-created
IDENTIFICATION DIVISION.
PROGRAM-ID. GENCOB.
WORKING-STORAGE SECTION.
01 START-KEY PIC 9(10).
```

a. Does the current assembler-statement test match the last line? b. What type does the member get, and with which report? c. Is it a translation input? d. The same text sits in `cobo1/GENCOB.cb1`. Is it classified by content?

Answer (by the rules, on an author-created member; not an ETC run). (a) No. The statement's opcode position holds `01`, and a line starting `START-KEY` would not match either, because the opcode must end at a blank or the end of the line. (b) COBOL program unit, reported as misfiled. (c) No: it is under `asm/`. (d) No. Content classification applies to `asm/` only. The directory selects the reader, and the selector would consider the member.

Level 3 — Architecture reasoning

Three questions, three owners

Each of the three questions has a different owner. The location belongs to whoever placed the file, which for a reference estate was the import step. The type belongs to the reader that opens it. The eligibility belongs to the selector. An inventory built from one of them answers that question and no other. A client who says "106 programs to migrate" has quoted the second owner's count. Neither 106 nor 105 says how many programs ETC translates: it emitted Java for 36 of the 105.

Architecture Question

A prospective client lists 106 COBOL programs. What do you ask first?

Answer. Which count it is. 106 is the number of programs the reader comprehends as COBOL. 105 is the number of members the selector considers. Whether ETC emits Java for a member is a further question: it refused 69 of the 105 whole, each with a recorded reason.

ETC Lesson Learned — a path is a claim by whoever placed the file

The reference bundle taught the book a rule it would not have guessed: a directory name can be an artifact of how the estate was imported. Treat a path as testimony, to be checked against the member's text, and record who made it.

Architect's Warning

The most likely misreading is to take `.asm` or `asm/` as proof that a member is assembler. Two of the six members under `asm/` are assembler. One is COBOL, one is a screen map and two are unidentified. Neither the suffix nor the directory is semantic proof.

Claim Check

"CBSA has 31 translatable programs."

This fails three ways. 31 is the count of COBOL program units, and 30 is the count of translation members. "Translatable" is not what either number measures. And the missing program was never skipped: it was never in the selector's directory.

Quick Recall

1. What are the three questions asked of every member?
2. Which directory does content classification apply to?
3. Name the five facts in the `BANKDATA` story: upstream path, bundle path, cause, what the reader makes of it today, and eligibility.
4. How many COBOL program units, translation members and assembler units are in the three estates?
5. How many members under `asm/` are assembler, and what are the other four?

Key Takeaways

- Physical location, semantic type and translation eligibility are three questions. The location can itself be an import artifact.
- `BANKDATA` is COBOL at `src/base/cobol_src/BANKDATA.cb1` upstream. ETC's import step placed it under `asm/BANKDATA.asm` because the historical classifier took `START` in `START-KEY` for an assembler statement. The classifier was later corrected and the frozen bundle keeps the placement.
- 106 COBOL program units and 105 translation members are both correct. CBSA has 31 and 30.
- There is no single classifier. `asm/` uses content, `jc1/` uses parse attempts, translation uses the directory.
- Of the six members under `asm/`, two are assembler. The two unidentified members stay unidentified.
- Quillon's classification is a layer-B expectation until the bundle is run (Layer C: no case-study run is recorded in this edition).

Chapter 9 — The Evidence Corpus: Public Estates, Digests and Provenance

After this chapter you can:

- say what the three reference estates are, and what they are not;
- say what a pinned revision and a digest each establish, and what neither does;
- explain why a provenance record is not a guarantee that an artifact survived import;
- keep the HLASM validation corpus out of every estate figure;
- describe licence evidence without calling it legal clearance.

Chapters 7 and 8 quoted counts and named members from three reference estates. Every one of those figures measured something. This chapter answers a plain question: *what exactly did we measure?* The answer is not "the CardDemo, CBSA and GenApp repositories". It is three *imported estates*, built by ETC's own import step, and the difference matters for every number in the book. This chapter is about how ETC handles its evidence. It makes no claim about the platform.

Level 1 — Intuition

A sealed box is not the warehouse

An archivist photographs a warehouse on a given day, sorts the items into labelled boxes, seals each box with a numbered tamper tag, and writes a log of what went into which box. Later, anyone can check that a box still carries its tag. The tag says the box is the one that was sealed. It does not say the boxes hold everything the warehouse held, or that every item went into the right box.

Archive	This book
The warehouse on a given day	An upstream repository at a pinned revision
The sorted, sealed boxes	An imported estate (a bundle)
The tamper tag	The digest
The log	The import record
The day the photograph was taken	The pinned revision

Six terms

Term	In this book	Is not
Corpus	The source material the book measures or tests against: the three imported estates plus the HLASM validation corpus	One population; representative of enterprise portfolios
Bundle	The directory set ETC's bundle reader reads (<code>cobol/</code> , <code>jcl/</code> , <code>copybook/</code> , <code>screen/</code> , <code>asm/</code> , <code>scheduler/</code>)	An upstream repository tree
Imported Estate	A normalised, frozen local bundle that ETC's import step built from one pinned upstream commit, linked to it by the corpus manifest and an import record	Lossless; equivalent to the upstream repository; proved complete by its import record
Validation Corpus	The pinned HLASM language-validation repositories (zblas, zos-hlasm-kvstore, zkvs)	An estate; a source of any estate figure
Pinned Revision	The exact upstream commit (and branch) a bundle or validation repository was taken from, recorded in a manifest	A release; the current head of its branch
Digest	A hash recorded in the manifest and checked on fetch	Proof that the bundle is a complete or faithful copy of upstream

Level 2 — Engineering mechanics

How an imported estate is built

ETC's import step works on a clone of the upstream repository at the pinned commit. It does five things.

1. It looks at each text member of the clone and classifies it with the classifier in use *at import time*.
2. It copies recognised members into the reader directories by kind, and normalises names and suffixes on the way.
3. It leaves out members classified off-platform or unclassified.
4. When two members normalise to the same bundle name, only one is kept.
5. It writes the import record for the bundle, and the bundle is frozen.

The book states the result in one passage, and this chapter uses it unchanged:

The three estates the book measures are imported estates: bundles that ETC's import step built from pinned upstream commits, normalised into reader directories and then frozen. A manifest links each bundle to its upstream commit and digest, and an import record lists where each member came from. The digest proves the measured bundle is the pinned one. It

does not prove the bundle is a complete or faithful copy of the upstream repository: the import step moved some members to different directories, left out members it could not classify, and dropped one member in each of two name collisions.

Every measured estate figure in this book is a figure about the *imported inventory*, the bundle. The *upstream inventory*, the repository at the pinned commit, is a different population.

What each record establishes

Record	Where	Establishes	Does not establish
Manifest	the corpus manifest	Repository, ref, commit and digest per bundle	That the bundle holds everything at that commit
Digest	the manifest, checked by the verify step	The measured bytes are the pinned bundle	Completeness, a lossless import, correct placement, perfect metadata
Import record	per-bundle provenance record	Where each member came from, its kind, the evidence for the kind and its placement, as ETC recorded them	That each entry is accurate, or that each recorded member is in the bundle

Pinned revisions

The manifest pins each estate to a commit: CardDemo to `main`, CBSA to the branch `July2024Refresh`, and GenApp to `main`. The validation corpus carries three more pins in its own manifest. On 2026-10-05 an independent check confirmed that all six commits exist upstream. That is a statement of existence only. It does not say the commit is the current head of its branch, that the bundle is complete, or that any placement is right. The full hashes are in the note to FIG-09-1.

What the digest proves

The digest is recomputed from the files when the verify step runs. On 2026-10-05 it passed for all three bundles: 197 files for CardDemo, 189 for CBSA and 80 for GenApp. Those are files covered by the digest. They are not the members offered to the reader, which are 175, 179 and 78, or 432 in all. The two counts measure different things. In this book "offered" means the reader sense only.

The digest answers one question: *are these the bytes that were pinned?* It does not answer *are these all the upstream bytes?*, *were they sorted correctly?* or *is the import record right?* A reader who treats a matching digest as proof of completeness has moved from identity to completeness, and nothing in the digest supports that move.

Provenance is a record, not a guarantee

The import record lists where each member came from. It cannot list what the import step never copied, and it can list a member the bundle no longer holds. Two kinds of event show this.

Placement changes. Chapter 8 told one: four bundle members sit under `asm/` (three in CBSA, one in GenApp) because the import-time classifier placed them there, and the frozen bundle keeps those placements. The import record records them.

Name collisions. When two upstream members normalise to the same bundle name, one survives. According to ETC's import record, there were two such collisions.

	CardDemo	CBSA
Kept	<code>app/jcl/TRANREPT.jcl</code> (job)	<code>etc/install/base/buildjcl/BANKDATA.jcl</code> (procedure)
Dropped	<code>app/proc/TRANREPT.prc</code> (procedure)	<code>etc/install/base/installjcl/BANKDATA.jcl</code> (job)
Import record for the dropped member	<code>placed: true</code>	<code>placed: true</code>
In the bundle	one member of that name	one member of that name

The CBSA `BANKDATA.jcl` pair is unrelated to the COBOL program of the same stem in Chapter 8.

Both dropped members are recorded as placed, and neither is in the bundle. A reader who trusts the record alone would count two members that do not exist. The lesson is: **a provenance record is not a guarantee that the artifact survived import**. The two collisions and the four placements are teaching examples. They are not a catalogue of failures, and this chapter makes no claim about any other member.

FIG-09-1 — The evidence corpus · AUTHOR-CONCEPTUAL, with CORPUS-DERIVED counts

Four bands, in order: the upstream source, the import step, the imported estate and the validation corpus. Only band 3 carries the measured counts, taken on 2026-10-04 at the book baseline and rechecked 2026-10-05. Those figures are about the imported inventory, not about the upstream repositories.

Band 1. Upstream source. Not measured.

	Upstream of CardDemo	Upstream of CBSA	Upstream of GenApp
Upstream project	AWS CardDemo	CICS Banking Sample Application	CICS GenApp
Pinned ref and commit	<code>main @ 59cc6c2...</code>	<code>July2024Refresh @ 4173345...</code>	<code>main @ f6f3f4b...</code>

Band 2. Import step. ETC's import record, as read at the baseline.

	CardDemo import	CBSA import	GenApp import
asm/ members placed by import-time classification (Chapter 8)	0 of 2	3 of 3	1 of 1
Members dropped in a name collision	1	1	0

Band 3. Imported estate. As measured.

	Imported CardDemo estate	Imported CBSA estate	Imported GenApp estate
Files covered by the digest	197	189	80
Members offered to the reader	175	179	78
COBOL program units	44	31	31
COBOL translation members	44	30	31
Assembler program units	2	0	0
Batch schedule input	present (the only estate)	none	none
Reference-execution record	five programs, recorded 2026-09-20, not re-run	none claimed	none claimed

Band 4. Validation corpus.

Validation corpus: not an estate; contributes to no estate figure
zblas master @ 9ee3044... · zos-hlasm-kvstore master @ 0b34698... · zkvs master @ 0c47e52...
No member counts appear here. It has its own manifest and its own pins.

Provenance note. Each estate's digest proves the bundle, not upstream completeness. Between band 1 and band 3 sits an import step (band 2) that moved, left out and dropped members: the two rows of band 2 record the placements and the drops, and the import record cannot list what was left out. The imported estate is therefore not shown to be lossless, and no band 3 count is an upstream count. The figure does not imply representativeness of any enterprise portfolio, equivalence to the upstream repositories, or legal clearance of any licence evidence.

Note — pinned revisions (existence upstream confirmed 2026-10-05; existence only).

Corpus	Ref	Commit
CardDemo	main	59cc6c2fd7ebd7ef7925cad552a01a4b8b6e4d5e
CBSA	July2024Refresh	417334533178ab6e753cc64b0e0e5cf0b4952704
GenApp	main	f6f3f4b2580d31b7d8dcc31ce3e3676f4cceaaaa
zblas	master	9ee3044b48b4c2d26b0793da67aa6acc9c823d06
zos-hlasm-kvstore	master	0b34698b4a018454096e5f117aba987bfc3f8eab
zkvs	master	0c47e5284e395fdb072be39d9d9e9be322047ea1

Read the table for what each estate can and cannot teach. Only CardDemo carries a schedule, so schedule reading is evidenced on one estate. Only CardDemo carries assembler units, two of them. Only CardDemo has a recorded reference-execution result. CBSA and GenApp teach nothing about these three topics, and nothing in the book should be read as if they did.

The validation corpus is not an estate

The HLASM validation corpus has its own manifest. That manifest says it "is not an estate, it contributes to no estate figure". Its three repositories serve as an independent HLASM corpus for testing ETC's assembler reading. No estate count in this book includes a member from it, and no member count for it is published. One baseline test run that points the HLASM tests at the validation material passed 43 tests with none skipped and none failed. That is a count of tests passed, not of members.

Licence evidence is not clearance

Three things stay apart. *Observed repository licensing evidence* is what files and metadata say, at a revision. *Legal interpretation* is not provided by this book, and nothing here is legal advice. *Editorial publication policy* is our own conservative choice: simplified or fictional examples first, tiny excerpts only where needed, no whole programs, no redistribution of bundles. The excerpt limit is an editorial rule, not a licence term. This chapter quotes no estate source.

Corpus	What the repository evidence indicates
CardDemo	© Amazon Web Services, Inc.; the repository is licensed under Apache License 2.0, from the upstream <code>LICENSE</code> .
CBSA	Ambiguous. GitHub's licence field returns <code>null</code> . Upstream <code>main</code> is archived with an as-is notice. The root <code>NOTICES</code> file carries EPL-2.0 text among other third-party notices. The COBOL files carry IBM copyright notices without inline licence statements. At the pinned <code>July2024Refresh</code> commit the root <code>LICENSE</code> carries EPL-2.0 text, which is evidence for that branch only. No repository-wide licence is asserted here.
GenApp	© IBM Corporation; EPL-2.0, from the README declaration; the manifest records the <code>LICENSE</code> file.
zblas, zkvs	Apache-2.0.
zos-hlasm-kvstore	MIT.

Copyright notices observed in a repository are retained, not paraphrased away. Where a chapter needs to describe CBSA, it describes counts and structure rather than quoting.

Quillon, positioned

Quillon is the fictional shop of the case study. Against these estates, it is an authored bundle. It is not a corpus: it has no pinned upstream, no digest in the manifest and no import record, and it contributes no measured figure. Anything Quillon shows is an authored artifact. When an expected ETC outcome is stated for it, it is labelled as an expectation, and no Quillon number appears beside an estate number.

Current Support

Capability	Status
Import of a pinned upstream clone into a bundle, with an import record	Implemented
Manifest linking bundle, commit and digest	Implemented (the corpus manifest)
Digest recomputation on fetch	Implemented (exercised by running it, not by a dedicated test)

Current Limitation

- The digest proves identity, not completeness. It compares the bundle with the manifest, not with the upstream tree.
- The import record is not authoritative. Two entries name members that are not in the bundle.

- The frozen bundles keep import-time placements and drops. A later classifier fix does not rewrite them.
 - Upstream layout facts in this chapter rest on ETC's import record, except the `BANKDATA` path in Chapter 8, which was checked upstream.
 - The CBSA licence evidence is mixed, so this book asserts no repository-wide licence for it.
-

Level 3 — Architecture reasoning

Why the distinction pays

A migration team asked "how many programs does the estate have?" can answer from the upstream inventory or from the imported one. The two differ here, and the difference is known only because the import record and the bundle were set side by side. In a real engagement the equivalent step is to say, before any measurement, which inventory is being counted, who built it, and what the build could have lost.

Failure Case — counting the record

An analyst totals the import record for a bundle and reports "every recorded member is analysed". Two recorded members are not in the bundle, so the total overstates what ETC read. The record said where each member *came from*. It never said the member *arrived*.

Architect's Warning

The three estates are public sample applications, imported and frozen by ETC. They are not representative of enterprise portfolios, and an imported estate is not the upstream repository. A measurement over these bundles describes these bundles at the baseline and nothing larger.

Claim Check

"We measured the CBSA repository, and the digest matched, so the bundle is a complete copy."

This fails twice. The subject of the measurement is the imported bundle, not the repository. And a matching digest shows that the bundle is the pinned one, not that it is complete: the import step moved some members, left some out and dropped one member in a name collision.

Quick Recall

1. What is a reference estate in this book, and what is it not?

2. What does the digest prove, and name two things it does not prove?
3. Which two members were dropped in name collisions, and what does the import record say about them?
4. Why is the validation corpus absent from every estate count?
5. Why does this book assert no repository-wide licence for CBSA?

Key Takeaways

- A measured estate is an imported estate: a normalised, frozen bundle linked to a pinned commit by a manifest and an import record.
- A digest proves the identity of the measured bytes. It does not prove completeness, a lossless import or correct placement.
- A provenance record is not a guarantee that the artifact survived import.
- The validation corpus is not an estate and contributes to no estate figure.
- Licence evidence is not legal clearance. The CBSA evidence is mixed, so no repository-wide licence is asserted for it.
- Quillon is an authored bundle, not a corpus.

Chapter 10 — The Typed EvidenceGraph

After this chapter you can:

- say what a node and an edge are in ETC's EvidenceGraph, and why both are typed;
- explain why identity is a kind plus a name, and what goes wrong with a name alone;
- read the 33 allowed node–edge–node combinations as a schema, and say what the check that enforces them does and does not establish;
- say what an edge's supports record, and which fields mean different things on different edges;
- say why an EvidenceGraph edge is not, by itself, a business dependency or a migration plan;
- say what an estate boundary is, and what 442 `CALLS` edges are not.

Chapters 5 to 9 turned artifacts into statements. This chapter is about the structure ETC assembles them into. It describes ETC. The platform appears only to say what a relationship means in the artifacts. The graph is built by deterministic code, and no model takes part in building it.

Level 1 — Intuition

A map with pins and threads

Picture a wall map of a company's batch estate. Every pin is one thing: a job, a program, a data set. The kind of pin is part of what the pin is. A thread joins two pins and carries a label: *runs*, *reads*, *calls*. A small card on each thread says which artifact said so, who read it and where.

A rule board stands beside the map. It lists which kinds of thread may join which kinds of pin. A *job runs a program* is on the board. A *program runs a job* is not. Pins, threads, cards and board are the **nodes**, **edges**, **supports** and **allowed triples** below; a pin marked "the evidence stops here" is an **estate boundary**.

One name, two things

A name is not an identity. In a mainframe shop the same string often labels a record layout *and* the data set whose records use it. The layout lives in a copybook, the records in a data set, and neither is the other. The figure uses a made-up name, `SAMPLEREC`, to show the hazard.

FIG-10-2 — Typed identity versus name-only lookup · AUTHOR-CONCEPTUAL

PANEL 1 — name-only lookup

HAZARD: ETC's earlier design, as its own code comments record it; this is NOT current ETC behaviour

```
question : "Which jobs read the data set SAMPLEREC?"
graph    : copybook:SAMPLEREC      <-- found first
           dataset:SAMPLEREC
lookup   : by name only -> returns whichever node comes first
answer   : the copybook
           (a question about a data set answered with a copybook)
```

PANEL 2 — typed lookup

CURRENT behaviour (kind-scoped, plural)

```
question : "Which jobs read the data set SAMPLEREC?"
lookup   : name = SAMPLEREC, kinds = (dataset)
answer   : dataset:SAMPLEREC      the copybook is never offered

question : "What is called SAMPLEREC?"      (no kind given)
lookup   : name = SAMPLEREC, any kind
answer   : copybook:SAMPLEREC AND dataset:SAMPLEREC, returned together;
           the caller sees the ambiguity and decides
```

Legend. `kind:name` is a node's key. Solid text is a statement; the arrow marks the node a name-only lookup would return.

Caption. The pair (kind, name) is the identity; the name alone is not. Figure provenance: AUTHOR-CONCEPTUAL, illustrating the key rule and the typed lookup. The figure is an author-created example, not a corpus measurement. It does not imply that CardDemo, CBSA, GenApp or Quillon contains `SAMPLEREC`, that any collision was counted, or that typed keys canonicalise names (they do not).

Level 2 — Engineering mechanics

What the graph is made of

The EvidenceGraph is an immutable structure holding nodes and edges, plus a record of what the reading could not establish.

A **node** has a key, a kind and a name, with a few optional descriptive notes. An **edge** has a source, a target, a kind, one or more supports, and an operational severity.

Two judgements travel with an edge, and ETC keeps them apart.

Axis	What it describes	How it is set
Certainty	How far the <i>reading</i> can be trusted about the relationship	The best certainty among the edge's supports
Operational severity	What it would cost if the relationship is real and ignored	The worst severity seen for that edge

Neither is a probability or a weight. "Best" is deliberate: a support is one route to the conclusion, and a second, weaker route does not make the first less certain. A relationship can be weakly evidenced and costly to ignore, which is why severity is not derived from certainty.

Node kinds

The model declares 14 node kinds. Thirteen can appear in the graph. One cannot.

Group	Kinds	Status
Work	job, program	Emitted
Stores	dataset, vsam_file, db2_table	Emitted. One store name gets one kind: DB2 over VSAM over dataset when readings differ
Layout	copybook	Emitted
Scheduler	schedule_group	Emitted
Evidence limits	estate_boundary	Emitted
CICS	transaction	Declared, never emitted
Verification	verification-evidence nodes	Allowed by the schema; attached separately, outside this chapter's trace (Chapter 20)

The model names `transaction`, but no builder creates it and no allowed triple mentions it. It is the only unreachable kind, and the measurement is zero in all three imported estates. The model can *name* a CICS transaction; ETC does not *discover* one.

A `reads_vsam` edge does not authoritatively say what organisation the file has: `dataset` or `vsam_file` follows the bundle-wide, first-match DD-name heuristic of Chapter 7.

Identity: kind and name

A node's key is the kind's value, a colon, and the name:

```
job:QMABL040      program:QMARPT01      dataset:QMA.BILL.POSTED
copybook:QMAPOLCP  estate_boundary:QMACL900
```

The rule is stated once, and the builder and the validator both apply it. Nothing in it folds case, strips a qualifier or joins a library to a member. If an upstream reader already normalised a name (Chapter 7 removes a trailing numeric generation offset from a `DSN=`), the key carries that result.

Two consequences are worth stating.

1. **The same name under two kinds is two nodes.** That is allowed. It is the whole point of FIG-10-2.
2. **A name that several members claim is not one job.** In GenApp, three `JOB` card names (`GENACSD`, `GENADB2` and `GENASOAP`) are each declared by more than one member. The node name carries a member qualifier, and anything that refers to the job by card name alone, such as a scheduler declaration or a data flow, is dropped and listed among the unresolved job names. ETC does not guess which member was meant.

The lookup follows the same idea: a lookup by name is plural and kind-scoped, and a role is assigned to a store by its key, not by its name.

Edges: nineteen kinds, each with a meaning

The graph does not have a "depends on" edge. It has 19 edge kinds, and each says what one artifact does to another. The arrow always points from the first kind to the last in the triple.

Edge kind	Arrow reads	Comes from
<code>executes</code>	job → program	A step the job runs, after procedure expansion
<code>reads</code> <code>writes</code> <code>updates</code>	job → dataset	The DD disposition
<code>reads_vsam</code> <code>writes_vsam</code>	job → VSAM file	The DD disposition; a VSAM update maps to <code>writes_vsam</code>
<code>reads_db2_table</code> <code>writes_db2_table</code> <code>updates_db2_table</code>	job → table, and program → table	The DD disposition for jobs; embedded SQL for programs
<code>produces</code>	job → store, and store → estate boundary	The DD disposition; the second form marks a store nothing here reads
<code>consumes</code>	store → job	The DD disposition
<code>data_precedes</code>	producer job → consumer job	Data flows, after utility rules
<code>scheduled_precedes</code>	predecessor job → job	A scheduler declaration
<code>calls</code>	program → program, or program → estate boundary	A call the reader recorded
<code>uses_copybook</code>	program → copybook	A <code>COPY</code> the reader recorded
<code>waits_for</code>	schedule group → job	The folder or system a job is filed under
<code>external_trigger</code>	estate boundary → job	A scheduler predecessor with no member in the bundle
<code>transforms</code> <code>compares</code>	verification nodes	Chapter 20

Three points stop the table being misread.

- **`waits_for` is filing, not waiting.** It says which scheduler group a job is scheduled in. It does not say that one job waits for another (Chapter 11).
- **Access edges follow the disposition, not the program.** The COBOL `OPEN` mode is not read into any edge (Chapter 7). A `reads` edge says what the `DD` asked for.
- **A program reaches a store only through DB2 SQL.** There is no program → dataset edge. A program's file access is known from its job's JCL.

The schema: allowed relationships

Typed nodes and edges are half the structure. The other half is the list of legal combinations. A **triple** is (source kind, edge kind, target kind). The graph allows only the triples its schema lists: 28 that describe the estate and 5 for verification. FIG-10-1 shows the idea of the schema, not the full list.

FIG-10-1 — A typed evidence graph, conceptually · AUTHOR-CONCEPTUAL

TYPED NODES	TYPED RELATIONSHIPS (examples)	
job	job	--executes--> program
program	job	--reads / writes--> store
store	store	--consumes--> job
(data set, keyed file, table)	job	--data_precedes--> job
copybook	job	--scheduled_precedes--> job
scheduler group	program	--calls--> program
estate boundary	program	--uses_copybook--> copybook
	boundary	--external_trigger--> job
EVIDENCE ON EVERY RELATIONSHIP		CHECKS WHEN THE GRAPH IS BUILT
one or more supports:		1. a node's key is its kind and name
what was read		2. no two nodes share a key
by which reader		3. every relationship names two held nodes
how it was established		4. every (source kind, relationship, target kind) is one the schema lists
certainty = the best support		
severity = the worst seen		
		The checks establish SHAPE, not truth.

Legend. The arrow points from the first kind to the last. The boundary node marks where the evidence stops.

Caption. A graph of typed nodes, typed relationships, supports and a few construction checks. Figure provenance: AUTHOR-CONCEPTUAL, a teaching model. It lists examples, not the schema. The check runs when a graph is **constructed**, not when an edge is added, and it establishes **shape**, not truth. It does not show or imply: the complete set of allowed triples; validation at insert time; that a valid graph is a correct one; that a CICS transaction is ever discovered; or any edge between a program and a data set.

Where validation happens

- **The builders collect.** Seven builders add nodes and edges to a scratch structure, and an eighth step assigns store roles. None of them enforces the graph's structural rules (key form, endpoint presence, allowed triple) at the moment it adds something. An edge is often recorded in the same breath as the node it points at, and checking each insertion would make the order of construction part of the contract.
- **The builders omit; they do not refuse.** When an input is unusable, such as an unnamed store, a data definition that carries no state, a shared card name, or an `APPEND`,

`NONE` or `UNKNOWN` access mode that makes no typed edge, the builder leaves it out. Where it matters, the omission is recorded in the reading-quality record or in an issue.

- **The graph refuses.** When the finished nodes and edges become an `EvidenceGraph`, a validator checks four things and raises one error listing **every** fault, not the first.

Fault	What it means
Key mismatch	A node's key differs from <code>kind:name</code>
Duplicate key	Two nodes share a key
Missing endpoint	An edge names a node the graph does not hold
Disallowed triple	The edge's kinds are not in the 33

Pseudocode — building the graph

Pseudocode written for this book. It is not an excerpt of ETC source. The entry function is a pure function of one reading: it adds the join, the edge types and the provenance, not new knowledge. Its callers are scripts; no route in the API calls it.

```
[PSEUDOCODE – PUBLIC EDITION]  building the graph from one reading
def build_graph(reading):
    store_kind = settle_one_kind_per_store(reading)
    add the jobs and stores          # executes, access, produces, consumes
    add the calls                    # calls, unresolved-call boundaries
    add the copybook uses            # uses_copybook
    add the table accesses            # program to DB2 table
    add the data orderings            # data_precedes
    add the scheduler declarations    # scheduled_precedes, external_trigger,
                                     # waits_for
    add the outbound boundaries       # store to outbound-store boundary
    assign a role to each store       # a role on each store node
    graph = construct the graph       # sorted nodes and edges; the four
                                     # checks run here, at construction
    return graph with the reading-quality record attached
                                     # a copy: only that record changes
```

The reading-quality record carries the absences that leave no edge behind: the schedule-evidence state (Chapter 11), jobs whose predecessors are alternatives, members whose place is unestablished, members something named but the bundle lacks, and the shared card names above. An unparsed scheduler export and an export that declares nothing produce the same edges; this record tells them apart.

Pseudocode — the four checks

Pseudocode written for this book. It is not an excerpt of ETC source.

```
[PSEUDOCODE – PUBLIC EDITION] the four construction checks
key_of(kind, name) = kind + ":" + name          # no case folding, no trimming
faults = []
for node in nodes:
    if node.key != key_of(node.kind, node.name): faults.add("key mismatch")
    if node.key already seen: faults.add("duplicate key"); continue
    remember node by key
for edge in edges:
    source, target = lookup(edge.source), lookup(edge.target)
    if source missing: faults.add("no node for source")
    if target missing: faults.add("no node for target")
    if either missing: continue
    if (source.kind, edge.kind, target.kind) not in the schema's allowed triples:
        faults.add("relationship not held by this graph")
return all faults, in a stable order    # construction raises if any exist
```

A dangling edge is refused, not repaired. *Interpretation (the author's reasoning, not a claim about intent):* if the validator manufactured a node to host a dangling edge, a gap in the evidence would become an ordinary vertex. The graph has a deliberate way to record "the evidence stops here": the estate boundary, below.

Code Trace Challenge — four faults

Below is a candidate graph, author-created for this exercise and not an ETC run.

```
[PSEUDOCODE] candidate graph
nodes: job:JOBA  dataset:DATA1  dataset:DATA1  program:PGMA (key "program:pgma")
edges: 1  job:JOBA  -reads->          dataset:DATA1
       2  program:PGMA -reads->        dataset:DATA1
       3  job:JOBA  -scheduled_precedes-> job:JOBB
       4  dataset:DATA1 -consumes->    job:JOBA
```

a. How many faults does construction report, and how many errors does it raise? b. Which edges are fine?

Answer (by the rule). (a) Four faults, in one error: a key mismatch for `PGMA` (the key is `program:pgma`, the rule wants `program:PGMA`; nothing folds case); a duplicate key `dataset:DATA1`; edge 2, because `program -reads-> dataset` is not an allowed triple; edge 3, because no node `job:JOBB` exists. (b) Edges 1 and 4.

What goes in a support

An edge keeps every artifact that attests it. Each **support** records:

Field	Holds
artefact	The member, or other named source, the claim came from
artefact kind	What kind of artifact that is: <code>jcl</code> , <code>dataset</code> , <code>scheduler</code>
<code>parser</code>	The reader that produced the claim
<code>location</code>	A line or step, where one applies; often absent
evidence kind	<code>OBSERVED</code> in the text, or <code>DERIVED</code> from it
<code>certainty</code>	How far the reading can be trusted for this support
<code>detail</code>	What the artifact said

The field that needs care is the first. What fills artefact differs by edge kind.

Edge kind	artefact holds
<code>executes</code> , access edges	The member holding the cited card. For a step reached through a procedure, the procedure member, and the basis is <code>DERIVED</code>
<code>calls</code>	A name synthesised from the caller (<code><caller>.cbl</code>), with no location
<code>uses_copybook</code>	The program's origin member
<code>data_precedes</code>	The store name , with reader <code>data-flow</code>
<code>scheduled_precedes</code> , <code>external_trigger</code> , <code>waits_for</code>	The scheduler product name (for example <code>Control-M</code>), with location <code>folder <name></code>
store → boundary <code>produces</code>	The store name

For a derived or scheduler edge that source is a store or a product, not a file and line.

A second rule matters for counting. When a builder adds a support whose artefact, artefact kind, `parser` and `location` all match one already on the edge, the new one is not added: that is the same testimony twice. The same relationship found in a *different* artifact is added. Chapter 12 shows what the count of distinct artefact strings does and does not mean. In this book it is called **multiple named supports**, and it is never confirmation by independent sources. (CardDemo's `TRANBKP` →`executes`→ `IDCAMS` has two supports, a job member and a procedure member: one card reached two ways.)

From job to program: declared, effective, runtime

`executes` edges come from **effective steps**: the steps a job declares, plus the steps a procedure expands into. Three words are kept apart.

Word	Means	In the graph?
Declared	The <code>EXEC</code> card in the job member	Yes, as an observed support
Effective	The step after procedure expansion	Yes, as a derived support naming the procedure member
Runtime	What ran on a given night	No

An `executes` edge carries no condition-code evaluation: a step that `COND` would bypass is still an edge.

Calls, and what 442 means

`calls` edges join a program to a program, or to an estate boundary when the callee has no member in the bundle. One edge is recorded for every call relationship the reader found, whatever its resolution, so a dynamic call expanded into several candidate targets yields **one edge per candidate**.

<code>CALLS</code> edges	CardDemo	CBSA	GenApp
Static	5	13	38
External, source unavailable	29	4	0
Dynamic, enumerated (one edge per candidate)	408	25	13
Dynamic, unresolved	0	3	0
All <code>CALLS</code> edges	442	45	51

[CORPUS-DERIVED: author measurement, 2026-10-05, at the book baseline] EvidenceGraph figures over the three imported estates.

The 442 is 442 `CALLS` edges in CardDemo's EvidenceGraph, 408 of them from candidate expansion. They are not 442 calls observed at runtime and not 442 call sites, and a candidate edge is not an observed call.

Shared layouts are not runtime links

`uses_copybook` joins a program to a copybook it names: 297 edges in CardDemo, 119 in CBSA, 29 in GenApp. Two programs that share a copybook share a **record layout**, so they may have to agree on a data format. That is not a call, not a runtime dependency, not evidence of a business domain, and not a reason to merge two slices (Chapter 13).

The size of the graph

EvidenceGraph, imported estates	CardDemo	CBSA	GenApp
Nodes	304	261	150
Edges	1,270	784	268
Estate boundary nodes	29	5	0
Edges with multiple named supports	5	17	0

[CORPUS-DERIVED: author measurement, 2026-10-05, at the book baseline] Imported estates only. The last row counts edges whose supports name more than one artifact, not independently confirmed edges (Chapter 12).

Estate boundaries: where the evidence stops

An **estate boundary** is a node that marks where the supplied evidence stops. It has three kinds.

Boundary kind	Meaning	CardDemo	CBSA	GenApp
unresolved call	A callee with no member in the bundle	8	5	0
inbound-trigger	A scheduler predecessor with no member in the bundle	2	0	0
outbound-store	A store written here and read by nothing here	19	0	0
Total		29	5	0

[CORPUS-DERIVED: author measurement, 2026-10-05, at the book baseline] Not 30, and not "2 inbound plus 28 outbound"; older project documentation states that figure and is out of date.

A boundary is *not* an application boundary, a business domain, a service, a microservice, a deployment boundary or a migration wave. It is also not proof that something exists on the other side. The missing callee may simply not have been collected. The word *estate* in "estate boundary" names the supplied evidence set, and the boundary is the point where that evidence stops. Application boundaries and slice edges are later questions (Chapters 16 and 13).

Quillon, sorted — one step, four edges

Quillon's `QMABL040` has a step `REPORT` that reads one data set and writes another. The derivation column is the expected outcome of the named rules on the author's artifacts, not an ETC result (Layer C: no case-study run is recorded in this edition). Layer C is `UNKNOWN`.

```
[CASE-STUDY ARTIFACT] QMABL040 (jcl/QMABL040.jcl), step REPORT
//REPORT   EXEC PGM=QMARPT01
//POSTED   DD DSN=QMA.BILL.POSTED,DISP=SHR
//SUMMARY  DD DSN=QMA.BILL.SUMMARY(+1),DISP=(NEW,CATLG,DELETE)
//REPORT   DD SYSOUT=*
```

Observed (layer A)	Expected derivation (layer B)	Interpreted
The EXEC card names QMARPT01	job:QMABL040 –executes→ program:QMARPT01 ; one support: the job member, kind jc1 , OBSERVED , location step REPORT	—
POSTED has DISP=SHR	job:QMABL040 –reads→ dataset:QMA.BILL.POSTED and dataset:QMA.BILL.POSTED –consumes→ job:QMABL040	" QMABL040 waits for what QMABL030 posts." A reading, not in these edges
SUMMARY is NEW,CATLG with a generation offset	job:QMABL040 –writes→ dataset:QMA.BILL.SUMMARY and job:QMABL040 –produces→ dataset:QMA.BILL.SUMMARY . The key has no (+1)	—
REPORT is SYSOUT=*	No node and no edge: a print file carries no state between jobs	—

Every edge above has one support, so none has multiple named supports. The (+1) is removed by the identity rule of Chapter 7.

Level 3 — Architecture reasoning

Why not one "depends on" edge

Take two Quillon jobs, QMABL020 and QMABL030 . Four sentences about them can all be true:

Sentence	Edge kind
The scheduler declares QMABL030 after QMABL020 .	scheduled_precedes
QMABL020 writes a data set that QMABL030 reads.	data_precedes (and, per job, produces / consumes on the store)
Their programs share a record layout.	uses_copybook , from each program
QMABL020 runs a program that calls another program.	executes , then calls

A single "depends on" edge would flatten these four questions: keep an order, share a format, move a program with its callee, or only keep a schedule. The types are the information. For an EvidenceGraph edge, name the ordering, access, call or layout edge, and do not say "dependency" without saying which.

EvidenceGraph is not a migration plan

An EvidenceGraph edge records an evidence-backed relationship: a named artifact, read by a named reader at a recorded location, says that one thing runs, reads, writes, calls or uses another. It is not automatically any of the following.

- a business dependency;
- an application boundary;
- a deployment boundary;
- a microservice boundary;
- a migration wave;
- a target architecture decision.

Each of those is a later judgement, with its own evidence and its own owner. The graph supplies inputs to that judgement: typed relationships, their supports, their certainty and the places where the evidence stops. Every measurement in this chapter is an EvidenceGraph measurement.

The practical rule for every later chapter: say what the edge records, such as "a `scheduled_precedes` edge" or "a `calls` edge". A bare "the dependency" is not used.

Shape is not truth

The construction check guarantees that the pieces fit. It says nothing about whether a relationship is true. A `reads` edge from a wrong `DISP` is perfectly valid, and so is a `scheduled_precedes` edge from an out-of-date export. A valid graph can be wrong: the schema guards against *malformed* evidence, not *mistaken* evidence.

Failure Case — "442 calls"

An estimate sizes the CardDemo conversion by "442 calls between programs". The number is real: it counts `CALLS` edges. But 408 of the 442 are dynamic calls expanded into one edge per candidate target, and only 5 are static. The artifacts support "442 call relationships the reader recorded, most of them candidates", not "442 calls the programs make". The correct statement scopes the number ("442 `CALLS` edges in the EvidenceGraph, 408 from dynamic candidate expansion").

Architect's Warning

The EvidenceGraph looks like a dependency model, and it is easy to hand its numbers to someone who wants one. It is a structured record of what the artifacts state, with its provenance and its gaps. Its edges are evidence relationships, not business dependencies. Its boundaries are the edge of what was supplied, not the edge of an application. Present it to a decision-maker with those two sentences beside it.

Claim Check

"ETC's graph shows our application boundaries and tells us what to migrate together."

This fails on three counts. An EvidenceGraph edge records an evidence-backed relationship, not a business dependency. An estate boundary marks where the supplied evidence stops, not where an application ends. And grouping jobs or choosing a target is a later judgement that the graph informs and does not make.

Current Support

Capability	Status	Evidence
Typed nodes keyed by kind and name, with no name canonicalisation	IMPLEMENTED	Reviewed at the baseline
33 allowed triples; four fault kinds reported together at construction	IMPLEMENTED	Reviewed at the baseline
Supports with duplicate suppression; best-support certainty; worst-seen severity	IMPLEMENTED	Reviewed at the baseline
Plural, kind-scoped name lookup; roles assigned by key	IMPLEMENTED	Reviewed at the baseline
<code>calls</code> for every recorded call relationship; unresolved callee as boundary	IMPLEMENTED	Reviewed at the baseline
Store written and read by nothing here → outbound boundary	IMPLEMENTED	Reviewed at the baseline
Builders in a fixed order, then one construction	IMPLEMENTED; called only from scripts	Reviewed at the baseline

Current Limitation

CURRENT IMPLEMENTATION LIMITATION. These describe the reviewed behaviour at the baseline.

1. `transaction` **is declared and never emitted.** No CICS transaction graph exists.
2. **HLASM internals, SORT, containment, derivation, lifecycle and control flow have no triples.** An assembler module is a `program` node with no internal edges.
3. **Only scripts call the builder.** No HTTP route builds or serves the EvidenceGraph.
4. **Store kind comes partly from a bundle-wide DD-name heuristic** (Chapter 7).
5. **A shared `JOB` card name is not resolved.** Data flows and scheduler declarations that name it are dropped and listed.

6. **A dynamic call becomes one edge per candidate**, and an unresolved target is marked with the same boundary note as a missing member, although the cause is an analysis gap. Do not read the note as the reason for a dynamic target.
 7. **A scheduler-only job can have two nodes that are not linked** (Chapter 11).
 8. **The graph is shape-checked, not truth-checked.**
-

Quick Recall

1. What are the two parts of a node's identity?
2. Name the four construction faults. When is validation done? Which node kind is declared but never emitted?
3. What does artefact hold on a `data_precedes` edge?
4. What do CardDemo's 442 `CALLS` edges count?
5. What is an estate boundary, and what is it not?

Key Takeaways

- Identity is kind plus name. There are 14 declared node kinds, 19 edge kinds and 33 allowed triples; `transaction` is declared but never emitted.
- The frozen graph validates the finished whole and lists every fault. That proves shape, not truth.
- What a support's artefact names depends on the edge. The count of named supports is not independence.
- An EvidenceGraph edge records an evidence-backed relationship. It is not a business dependency, an application or deployment boundary, or a migration wave.
- An estate boundary (29, 5 and 0) marks where the supplied evidence stops, not an application boundary. A `CALLS` count is a count of edges.
- Quillon's rows are expected derivations (layer B). No Quillon graph has been built by ETC.

Chapter 11 — Scheduler Evidence

After this chapter you can:

- say what a scheduler export declares, and what it cannot tell you about a night that ran;
- state, separately, what Control-M and CA-7 mean by conditions, `AND_OR`, `SIGN`, schedule IDs, triggers and requirements, and what ETC currently reads of each;

- name the three states of scheduler evidence (read, unreadable, not supplied) and say what the first one does not promise;
- explain what `SCHEDULED_PRECEDES`, `EXTERNAL_TRIGGER` and `WAITS_FOR` edges mean, and why `WAITS_FOR` is not one job waiting for another;
- trace Quillon's scheduler export into expected edges, including a job that exists only in the export.

Chapter 4 taught the batch night: a scheduler decides when jobs are submitted, and its definition states what must be true first. Chapter 10 taught the typed graph that holds the result. This chapter joins them. It teaches what ETC reads from a scheduler export and what it makes of it.

How this chapter keeps two things apart. Scheduler products have their own meanings. ETC reads a subset of the formats and applies its own rules, which are sometimes narrower than the product. Every topic below states the **platform meaning** in one box and **what ETC currently reads** in another. The two are never joined into one sentence. Where ETC reads less than the product states, the second box carries the label `CURRENT IMPLEMENTATION LIMITATION`.

Level 1 — Intuition

A flag board

In a bakery, the night crew uses a flag board. When a crew finishes a stage it raises a flag with a name: *dough ready*. A later crew's plan says "start when *dough ready* is up". The plan does not name the crew that raises the flag. It names the flag.

Four small facts about that board explain most of the chapter.

On the board	In a scheduler	The trap
A flag has a name, a day and a kitchen	A condition is read with its date and the system it belongs to	The same name on another day, or in another kitchen, is not the same flag
"Start when <i>either</i> oven is free"	An OR join: any one of several conditions releases the job	Reading it as "start when both ovens are free"
Taking the flag down after use	A condition deleted when a job ends	Reading "take it down" as "wait for the opposite"
A weekday plan and a month-end plan for one crew	A schedule variation of one job	Reading "the month-end plan" as another crew you must wait for

A fifth fact belongs to the second scheduler. An **alarm** can put a job on the waiting list. A **gate** decides when the job on the list may be released. Putting a job on the list is not the same as opening the gate.

What ETC does with a plan on paper

ETC never sees the bakery. It sees a printed copy of the plan, if someone put one in the bundle. Three situations are possible.

Situation	Plain meaning	ETC's word
No plan was put in the bundle	Nothing is known about the schedule	NOT_SUPPLIED
A plan was put in, and ETC could not read any of it	Something was offered that ETC cannot interpret	UNREADABLE
ETC read at least one of the plans offered	A plan was read; it may be partial	READ

The next section says why `READ` is a weaker word than it sounds. The sentence to remember is this: **a schedule export tells you what was declared. It does not tell you what ran.**

Level 2 — Engineering mechanics

The formats ETC reads

(the completeness of the scheduler export is not established)

ETC reads three scheduler formats, each from exported definitions: a **Control-M XML** export, a **CA-7 LJOB** listing, and a **native JSON** document with a strict schema (version `1.0`). It chooses the reader by the content of the member: Control-M text starts with `<`, CA-7 has an `LJOB, JOB=` command line, and the native format has its own check. If no reader recognises a member, ETC raises an error that names the formats on offer.

Those are "the formats ETC reads". They are not the products' schemas. ETC reads a subset of each product's export, and nothing in this chapter says that a reading is complete, that it matches a real installation, or that it works against a live scheduler.

The three states of scheduler evidence

State	When it holds	Issue raised
NOT_SUPPLIED	No schedule member was offered	Warning: not supplied
UNREADABLE	Schedule members were offered and none parsed	Warning: unreadable, with the member recorded
READ	Any schedule member parsed	— (a member that failed still raises its own warning)

The third row is the one to hold on to. If a bundle offers two exports and one parses, the state is `READ`, even though the other failed. **READ means that a supported, supplied source was read under the current capability. It does not mean that the scheduler's evidence is complete.**

The state also changes how the comparison of scheduler order with data order is reported (Chapter 12): a data order with no declared counterpart is reported as unverified when no schedule was read, and as undeclared only when one was.

Control-M: what is in an export

Platform meaning ([B11]). The `exportdefjob` utility writes job definitions to XML. A `JOB` element describes a job and sits inside a `SMART_FOLDER` container. `INCOND` lists input conditions that must be satisfied before submission. `OUTCOND` names conditions posted when the job completes. The source does not establish that an export preserves live schedule adjustments or manual operator overrides.

What ETC currently reads. ETC reads `JOB` elements, their `INCOND` and `OUTCOND` conditions, and the folder a job sits in. It reads a job's name from `JOBNAME` or, failing that, `MEMNAME`. It does not read `TIMETO`, `ODATE`, the `OP` grouping attribute or any Control-M Server context.

ETC reconstructs order in two passes.

1. **Posting.** For every job, each `OUTCOND` whose `SIGN` is a literal `+` posts its condition name. The result is a table from condition name to the jobs that post it.
2. **Waiting.** For every job, each `INCOND` name is looked up in that table. The jobs that post it are the job's predecessors. A job is never its own predecessor.

The match is by **name, across the whole export.**

Condition identity

Platform meaning ([B11])	A Control-M condition is read together with its date and the Control-M Server it belongs to. A matching condition name alone does not establish one universal dependency.
What ETC currently reads	Conditions are matched by <code>NAME</code> only, across the whole export. <code>ODATE</code> , <code>OP</code> and server context are not read.
Label	CURRENT IMPLEMENTATION LIMITATION

What does this mean in practice? Two unrelated workflows that happen to use the same condition name, or one condition name used for different dates, look like a single dependency to ETC. Whether they are one dependency on a given installation depends on dates and servers that the reading never examines. The cited sources describe how conditions are normally scoped. They do not establish how a particular installation shares them, and this chapter makes no claim about one.

AND_OR : all, or any

Platform meaning ([B11])	The And/Or relationship decides whether all or only some of a job's prerequisite conditions must exist. OR marks alternatives.
What ETC currently reads	If a job has more than one INCOND and any one of them carries AND_OR="O", ETC marks every predecessor of that job as an alternative. It still emits each one as a SCHEDULED_PRECEDES edge with medium certainty, lists the job in the ambiguous orderings, and raises the <i>alternative predecessors</i> warning. It evaluates no expression.
Label	CURRENT IMPLEMENTATION LIMITATION : the graph records a stronger order than the scheduler enforces.

Read the ETC row twice. **Alternative candidate relationships are not all mandatory predecessors.** If a job is released when *either* condition is posted, then a graph edge from each poster to the job says each *may* release it. It does not say each *must* finish first. A reader who treats the edges as a set of required predecessors has read an OR as an AND.

Two further cautions. The flag covers the whole job, even when some of its other conditions are joined by AND. And the edge itself carries no flag: an edge that belongs to an alternative set looks the same as one that does not. The difference is in the ambiguous-orderings list and the warning.

SIGN : add and delete

Platform meaning ([B11])	+ adds the condition (the default). - deletes it, when the job ends OK. A minus sign is not a dependency, a negative predecessor or an ordering.
What ETC currently reads	Only a literal SIGN="+" is read as posting. Anything else contributes nothing to the order. Deleting a condition is not modelled.
Label	ETC's reading.

The wrong inference is "SIGN="-" means the job runs *without* the condition", or "a minus sign is a reverse edge". Neither is in the platform meaning, and ETC draws neither. ETC's treatment of - is an omission: it records no cleanup.

A condition nobody posts

Suppose a job waits on a condition that no job in the export posts.

What ETC currently reads. There is no predecessor and no edge. ETC raises an INFO issue, the *external trigger* note, and nothing else. Do not confuse the issue code with the edge kind of the same name (below).

The export cannot say who or what raises that condition: another job outside the export, an operator, a file arrival, or something else. The evidence does not distinguish them, and ETC does not fabricate a relationship to fill the gap.

CA-7: schedule IDs, triggers and requirements

Platform meaning ([B12]). The `LJOB` command lists a job's definition. With the right options its output shows schedule IDs, triggered jobs and predecessor requirements: `LIST=SCHD`, `LIST=TRIG` and `LIST=RQMT`.

What ETC currently reads. ETC reads an `LJOB, JOB=` listing and its `TRIGGERED JOBS` section. It does not read requirements. Every CA-7 listing it reads receives an `INFO` issue, the *requirements not listed* note, which records missing evidence, not absence. ETC's reader is not a complete CA-7 model.

Platform meaning: <code>SCHID</code>	A schedule ID is a 1- to 3-digit number that identifies one scheduling variation of a job. It is not a predecessor and not an ordering edge.
What ETC currently reads	If a trigger line carries any <code>SCHID</code> , ETC marks that successor's predecessors as alternatives, the same flag Control-M's OR uses, and raises the <i>conditional trigger</i> note. Variations are collapsed into one predecessor set. ETC never draws a <code>SCHID</code> as an edge.
Label	Heuristic; <code>CURRENT IMPLEMENTATION LIMITATION</code>

Platform meaning: trigger and requirement	A trigger brings a job into the request queue when the triggering element completes successfully. Requirements must be satisfied before submission and can apply to some schedule IDs only. They are different things. Entering the queue is not starting to run.
What ETC currently reads	<code>TRIGGERED JOBS</code> entries are inverted into predecessors : the triggered job gets the triggering job as a predecessor. A job named only as triggered becomes a declaration of its own. Requirements are not read.
Label	Transformation heuristic. A trigger is not a requirement, and treating one as the other is ETC's interpretation, not CA-7's.

The consequence is limited but real. A `SCHEDULED_PRECEDES` edge that came from a CA-7 trigger says "the listing triggers this job". It does not say "this job cannot be submitted before the triggering job finished", because the gating evidence (requirements) was not read.

Issues the readers raise

Code	Severity	Raised when
the <i>external trigger</i> note	Info	A waited Control-M condition is posted by no job in the export. No predecessor, no edge
<i>Alternative predecessors</i>	Warning	A job has several <code>INCOND</code> and one carries <code>AND_OR="0"</code>
the <i>folder condition</i> note	Warning	A folder, not a job, posts a condition; ETC names no predecessor for what waits on it
the <i>unnamed job</i> note	Warning	A <code>JOB</code> has neither <code>JOBNAME</code> nor <code>MEMNAME</code>
the <i>conditional trigger</i> note	Warning	A CA-7 trigger carries a <code>SCHID</code>
the <i>requirements not listed</i> note	Info	Every CA-7 listing
the <i>unreadable export</i> note	Warning	A schedule member parsed under no reader
the <i>not supplied</i> note	Warning	No schedule member was offered

From declaration to edges

Each reader produces **declarations**: a job, the scheduler product, the folder or table it sits in, and its declared predecessors. The graph builder turns them into edges, one declaration at a time. The builder reads **predecessors only**.

For each declaration	EvidenceGraph result	Support
The job itself	A <code>job:</code> node, from the declaration's own name	—
It has a folder or table name	<code>schedule_group:</code> node, and <code>schedule_group -waits_for→ job</code>	Product name; location <code>folder <name></code> ; observed
A predecessor that is a job in the bundle	<code>job:pred -scheduled_precedes→ job</code>	Product name; location <code>folder <name></code> when there is one; observed, medium certainty; high severity
A predecessor that is <i>not</i> a job in the bundle	<code>estate_boundary:pred</code> (kind <code>inbound-trigger</code>), and <code>estate_boundary:pred -external_trigger→ job</code>	As above
A name several members declare	Dropped , and listed in the unresolved job names	—

Four terms need their plain readings.

- **scheduled_precedes** runs from the predecessor to the job. It is a *declared* order: someone's declaration that can be wrong or out of date. ETC gives it medium certainty. The export shows it was said, not that it is still enforced.
- **external_trigger** runs from an estate boundary to the job. The boundary node stands for a predecessor that the export names and that is not a job in the bundle. It is not a condition nobody posts (that is the Info issue above).
- **waits_for is filing**. It runs from a schedule group to a job and says the job is scheduled in that group, which for Control-M is its folder. It does **not** say that one job waits for another. The edge's label is not its English reading, and it is not renamed. Read it as "this job is filed under the scheduler group".
- **Successors are not used**. The builder reads predecessors only. The comparison of scheduler order with data order, in Chapter 12, uses successors too, so an export that stated its order only through successors would give order findings and no **scheduled_precedes** edge. Today CA-7 triggers are inverted into predecessors, so the case does not arise. Do not read "every declared order becomes an edge".

Pseudocode — export to ordering edges

Pseudocode written for this book. It is not an excerpt of ETC source.

```
[PSEUDOCODE – PUBLIC EDITION]  reading Control-M conditions into predecessors
posted = {}                      # condition NAME -> jobs that post it
for job in export.jobs:
    for out in job.OUTCOND:
        if out.SIGN == "+":      # only a literal plus posts
            posted[out.NAME].add(name_of(job))    # JOBNAME, else MEMNAME
for job in export.jobs:
    waited = [c.NAME for c in job.INCOND]
    preds = []
    for name in waited:
        if posted has name:     preds += posters of name, excluding the job itself
        else:                   note INFO  "external trigger"                # no edge
    alternatives = len(waited) > 1 and any(c.AND_OR == "0" for c in job.INCOND)
    if alternatives:            note WARNING "alternative predecessors"
    declare(job, folder, preds, alternatives)
# ODATE, OP and server context are never consulted
```

```
[PSEUDOCODE – PUBLIC EDITION]  turning declarations into graph edges
known = names of jobs the bundle holds
for d in declarations:
    if d.name is a shared card name: skip; record among the unresolved job names
    job = node(JOB, d.name)
    if d.folder:   edge(group(d.folder) -waits_for-> job)        # filing
    for p in the predecessors d declares:
        if p is a shared card name: continue
        if p in known: edge(node(JOB, p) -scheduled_precedes-> job)
        else:         edge(boundary(p, inbound-trigger) -external_trigger-> job)
# the successors d declares are not read here
```

Quillon's scheduler export — layer A

Quillon's scheduler file is `QMANIGHT.xml`, a fictional Control-M-style export. The sketch below fixes its teaching properties. The condition names and the folder placement are illustrative, and the exact text is fixed when the member is written.

```
[CASE-STUDY ARTIFACT] QMANIGHT.xml (sketch; illustrative names and folder)
<DEFTABLE>
  <SMART_FOLDER FOLDER_NAME="QMA-BILLING">
    <JOB JOBNAME="QMACL900">
      <OUTCOND NAME="QMA-CLAIMS-DONE" ODATE="ODAT" SIGN="+"/></JOB>
    <JOB JOBNAME="QMABL010">
      <INCOND NAME="QMA-CLAIMS-DONE" ODATE="ODAT"/>
      <OUTCOND NAME="QMA-EXTRACT-OK" ODATE="ODAT" SIGN="+"/></JOB>
    <JOB JOBNAME="QMABL020">
      <INCOND NAME="QMA-EXTRACT-OK" ODATE="ODAT"/>
      <OUTCOND NAME="QMA-RATED-OK" ODATE="ODAT" SIGN="+"/></JOB>
    <JOB JOBNAME="QMABL030">
      <INCOND NAME="QMA-RATED-OK" ODATE="ODAT"/></JOB>
    <JOB JOBNAME="QMABL040">
      <INCOND NAME="QMA-EXTRACT-OK" ODATE="ODAT"/>
      <OUTCOND NAME="QMA-REPORT-OK" ODATE="ODAT" SIGN="+"/></JOB>
    <JOB JOBNAME="QMAFN100">
      <INCOND NAME="QMA-REPORT-OK" ODATE="ODAT"/></JOB>
  </SMART_FOLDER>
</DEFTABLE>
```

Three facts about this file. `QMACL900` is present **only** here, as a `JOB` element: there is no JCL, no program and no other member for it in the bundle. What it does is unknown. `QMAPV001` is absent from the file, because it is run on demand.

And the file stays inside a deliberately small subset. Every `OUTCOND` has an explicit `SIGN="+"`, every waiting job has exactly one `INCOND`, and nothing uses a schedule variation. That is a choice about this fictional artifact, made so that the expected edges follow from ETC's reading. It does **not** mean that Control-M supports only `+`, that OR conditions do not exist, that `-` is invalid, that CA-7 schedule IDs are unsupported, or that ETC models scheduler semantics fully. The sections above teach all of those distinctions.

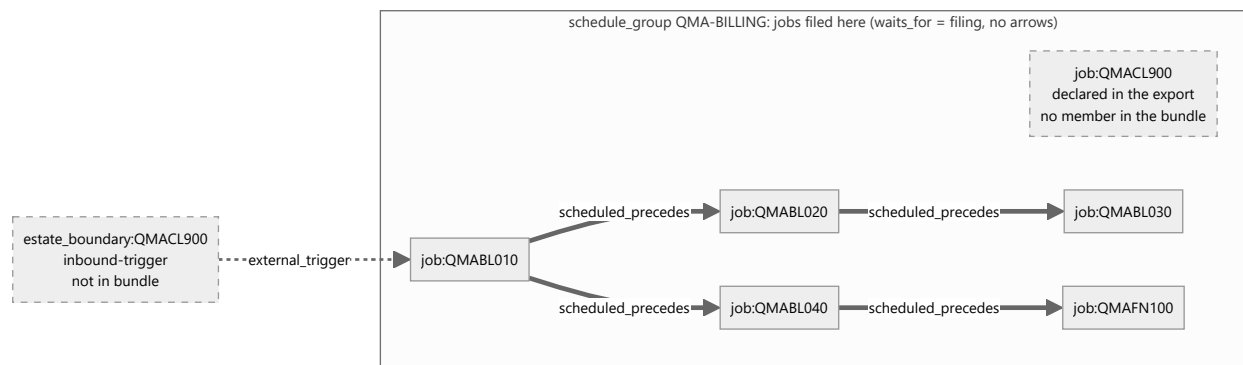
Quillon, sorted — layer B

The ETC columns are the expected outcome of the named rules on the author's artifact. They are not an ETC result (Layer C: no case-study run is recorded in this edition), and layer C is `UNKNOWN`.

Observed (layer A)	Expected derivation (layer B)	Interpreted
QMABL010 waits on QMA-CLAIMS-DONE ; QMACL900 posts it	QMACL900 is not a job in the bundle, so: estate_boundary:QMACL900 (inbound trigger) and estate_boundary:QMACL900 -external_trigger→ job:QMABL010	"QMACL900 must end before the night starts." A reading of an unknown job
QMABL010 posts QMA-EXTRACT-OK ; QMABL020 and QMABL040 each wait on it	job:QMABL010 -scheduled_precedes→ job:QMABL020 and → job:QMABL040	"The report should follow the extract." Chapter 12 returns to this edge
QMABL020 posts QMA-RATED-OK ; QMABL030 waits on it	job:QMABL020 -scheduled_precedes→ job:QMABL030	—
QMABL040 posts QMA-REPORT-OK ; QMAFN100 waits on it	job:QMABL040 -scheduled_precedes→ job:QMAFN100	—
Six jobs sit in folder QMA-BILLING	schedule_group:QMA-BILLING -waits_for→ each of the six jobs, QMACL900 included: filing, six edges	"All six are one application." Not in the artifact, and not what filing says
QMAPV001 is not in the file	No scheduler edge of any kind	—

Preview of panel A (scheduler layer) of FIG-12-2 — the full figure is in Chapter 12 · AUTHOR-CONCEPTUAL

This is panel A of FIG-12-2, shown here as a preview. The complete figure, with the data layer and the order findings, is in Chapter 12. Expected (layer B) (Layer C: no case-study run is recorded in this edition).



Legend. `==>` is `scheduled_precedes`: a declared order from one condition, predecessor to job. `-. -`
`>` is `external_trigger`, from an estate boundary to a job. The box is filing under a schedule group; it is not a set of arrows. Dashed outline: not in the bundle.

Caption. This is panel A of FIG-12-2 only, not the complete figure. The same name, `QMACL900`, is drawn as two unconnected nodes. That is what the graph is expected to hold, and it is a current ETC limitation (below), not a modelling recommendation. Figure provenance: AUTHOR-CONCEPTUAL, case study, layer B. The preview does not show an OR alternative, a minus-sign condition, a schedule variation, one job waiting for another through `waits_for`, a data order, or any run that happened. `QMACL900` has no source.

The scheduler-only job

`QMACL900` is a **scheduler-only job**: named in the export, with no member in the bundle. ETC creates a `job:QMACL900` node from the job's own declaration. It creates a separate `estate_boundary:QMACL900` node because the job is `QMABL010`'s predecessor and is not a job in the bundle. Nothing links the two nodes.

CURRENT ETC LIMITATION. The graph does not unify them. This is not a universal way to model a scheduler-only job, and the book does not normalise it away. CardDemo shows the same shape for two jobs, `CLOSEFIL1` and `CLOSEFIL2`. CardDemo's graph has 58 job nodes against 56 batch jobs read, and the difference comes from scheduler-only jobs.

The pair `QMACL900` to `QMABL010` is not assessed by the order comparison in Chapter 12, because `QMACL900` is not a bundle job.

Code Trace Challenge — an OR

Use this generic fragment. It is author-created and not a Quillon member. Assume `JOBP`, `JOBQ` and `JOBX` are all jobs in the bundle.

```
[PSEUDOCODE] generic Control-M-style fragment
JOBP  OUTCOND  COND-P  SIGN="+"
JOBQ  OUTCOND  COND-Q  SIGN="+"
JOBX  INCOND   COND-P  AND_OR="O"
      INCOND   COND-Q  AND_OR="A"
```

a. By the platform meaning, what releases `JOBX`? b. What does ETC record: edges, certainty, flags, issues? c. Which of these two sentences is correct, and why? (i) "`JOBP` and `JOBQ` must both finish before `JOBX`." (ii) "Either `JOBP` or `JOBQ` finishing may release `JOBX`." d. Change `JOBQ`'s `OUTCOND` to `SIGN="-"`. What does ETC record now?

Answer (by the rule, on an author-created fragment; not an ETC run). (a) Either condition, because OR marks alternatives. (b) `JOBP -scheduled_precedes→ JOBX` and `JOBQ -scheduled_precedes→ JOBX`, both medium certainty; `JOBX` is listed in the ambiguous orderings; one *alternative predecessors* warning. (c) Only (ii). The two edges are alternative candidates, and the graph records them as if

both were required, which is the limitation. (d) `COND-Q` is no longer posted, so `JOBX` has one predecessor (`JOBP`); `COND-Q` is waited on but posted by no one, so ETC raises the Info issue the *external trigger* note. The alternatives flag still applies, because the job has two `INCOND` and one carries `0`. No edge represents the minus sign.

Level 3 — Architecture reasoning

Declared is not run

A scheduler export carries three limits, two of which Chapter 4 named.

- It says what was **defined**. The Control-M source does not establish that an export keeps live adjustments or manual overrides.
- The CA-7 sources do not establish that schedules reduce to a simple chain, and requirements, triggers and manual release all exist.
- ETC reads a **subset** of what an export holds, so what ETC knows is narrower than what the export says.

So a `scheduled_precedes` edge is one kind of order evidence: *declared*. Chapter 4 named two others, the order the data requires and the order jobs ran in. This chapter has produced only the first. Chapter 12 sets the first beside the second, and the third is outside the bundle.

The shape of ETC's errors

Each limitation distorts the picture in a particular way. The table says how, and makes no claim about frequency.

ETC reading	Platform meaning	What can go wrong in the graph
Name-only condition match	A condition has a date and a server	Two conditions the platform keeps apart can join into one edge; the reading cannot tell
One <code>0</code> flags every predecessor	OR marks alternatives; AND-joined conditions are still required	Alternatives appear as ordinary required orderings
Only <code>+</code> posts	<code>-</code> deletes	Cleanup is invisible
Triggers inverted to predecessors	A trigger queues a job; requirements gate it	A trigger reads as a requirement
<code>SCHID</code> sets the alternatives flag	A schedule ID names a variation	A variation is read as uncertainty about predecessors
Requirements not read	Requirements gate submission	Gating evidence is missing, and the Info issue says so

The graph therefore records, at medium certainty, a number of orders the scheduler may not enforce in that form. CardDemo has 17 jobs listed in the ambiguous orderings. They come from more than one cause, and this book does not attribute them to OR alone.

Why the omission matters

A reader who sees no edge into a job may conclude "this job runs freely". The graph supports less than that. The absence may mean nothing waits on it; or that a waited condition has no poster in the export (an Info issue, no edge); or that the requirement was not read; or that the job's name was shared and dropped. Missing evidence is not evidence of absence, and the reading-quality record is where the difference is kept.

Failure Case — a name dropped, a schedule not missing

Suppose a scheduler export declares a job named `GENASOAP`, and several members of a bundle each declare a `JOB` card with that name. (GenApp's bundle supplies no scheduler export. This is an author-created supposition about what would happen.) The export says "this job runs after that one", and several members answer to the name. ETC cannot say which member is meant, so it drops the declaration, and any predecessor with the same problem, and lists the name in the unresolved job names.

A reader who looks for an edge, finds none, and concludes "`GENASOAP` is not scheduled" has made a reading error. The schedule is not missing. ETC declined to guess, and the list says so. Choosing one member would have published an ordering over a job nobody scheduled, at the certainty a read export carries.

Architect's Warning

Never carry a scheduler product's vocabulary into an ETC edge, or an ETC edge's vocabulary back into the product. "Predecessor", "waits", "trigger" and "condition" each mean one thing in Control-M or CA-7 and a narrower thing in ETC's graph. The safest habit is the one this chapter used: state the platform meaning in one sentence and ETC's reading in another. A migration plan that rebuilds a schedule from the graph should be checked against the scheduler's own definition, not against the graph's labels.

Claim Check

"The graph shows `QMABL040` waits for every job in folder `QMA-BILLING`, because each is linked to it by `waits_for`."

This fails. `waits_for` runs from a schedule group to a job and records filing. It is not an edge between jobs, and `QMABL040` has no `waits_for` edge from another job. Its declared predecessor is `QMABL010`, by a `scheduled_precedes` edge.

Current Support

Capability	Status	Evidence
Dispatch by content to Control-M, CA-7 and native readers	IMPLEMENTED	Reviewed at the baseline
Control-M order from posted + conditions matched to waited names	IMPLEMENTED	Reviewed at the baseline
Unposted waited condition: Info issue, no edge	IMPLEMENTED	Reviewed at the baseline
Alternatives flagged, warning raised, edges still emitted	IMPLEMENTED	Reviewed at the baseline
CA-7 triggers inverted into predecessors; SCHID flagged	IMPLEMENTED	Reviewed at the baseline
Declarations to <code>scheduled_precedes</code> , <code>external_trigger</code> , <code>waits_for</code>	IMPLEMENTED	Reviewed at the baseline
<code>NOT_SUPPLIED</code> , <code>UNREADABLE</code> , <code>READ</code>	IMPLEMENTED	Reviewed at the baseline

Current Limitation

CURRENT IMPLEMENTATION LIMITATION. These describe the reviewed reading at the baseline, and the book records no intent behind them. (the completeness of the scheduler export is not established)

1. **ETC reads a subset of each format.** `TIMETO`, `ODATE`, `OP`, server context, deletion and CA-7 requirements are not read. The formats are "the formats ETC reads", not the schemas.
 2. **Conditions are matched by name only.**
 3. **One OR flags every predecessor of the job**, and alternatives are still emitted as `scheduled_precedes` edges.
 4. **Only `SIGN="+"` posts.** Deletion is not modelled.
 5. **CA-7 triggers become predecessors**, a transformation heuristic, and any `SCHID` on a trigger sets the alternatives flag. Variations are collapsed.
 6. **`READ` does not mean complete.** Any one parsed member sets it.
 7. **A scheduler-only job can have two unlinked nodes.**
 8. **The graph builder ignores successors.**
 9. **Most corpus bundles supply no schedule evidence.** Only CardDemo's bundle supplies a scheduler export; the schedule state of CBSA and GenApp is `NOT_SUPPLIED`, which is not evidence of no scheduler relationships.
-

Quick Recall

1. Name the three states of scheduler evidence. Which one holds when two exports are offered and one parses?
2. In ETC, what does a waited Control-M condition that no job posts produce? Is `SIGN="-"` a negative dependency, and what does ETC do with it?
3. What is the difference between a CA-7 trigger and a requirement, and which of the two does ETC read?
4. Can `waits_for` be read as "job A waits for job B"? Say why.
5. `QMACL900` has two nodes. Are they linked? Is that a recommendation?

Key Takeaways

- A scheduler export tells you what was declared, not what ran. `READ` means a supported source was read, not that the evidence is complete.
- Platform meaning and ETC's reading are different sentences. Where ETC reads less, the book says `CURRENT IMPLEMENTATION LIMITATION`.
- A Control-M condition has a date and a server; ETC matches by name. OR marks alternatives; ETC flags them and still emits each as an ordering. A minus sign deletes; it is not an edge.
- A CA-7 schedule ID is a variation, not a predecessor. A trigger queues a job; a requirement gates it. ETC inverts triggers into predecessors and does not read requirements.
- An unposted waited condition gives no edge, only an Info issue. `waits_for` is filing under a scheduler group. `QMACL900` is scheduler-only and shows the unlinked pair.
- Quillon's rows are expected derivations (layer B). No Quillon export has been read by ETC.

Chapter 12 — Producer/Consumer Evidence, Corroboration and Conflict

After this chapter you can:

- say how ETC derives a data ordering from producer and consumer evidence, and why it and an access edge can state different directions for one job and store;
- tell apart three things easily both called "corroboration": duplicate evidence, an edge with multiple named supports, and agreement between two kinds of evidence;
- say what ETC's corroboration flag measures (5, 17 and 0), and what it does not;

- explain an order finding and its five verdicts, and why ETC reports a disagreement and never resolves it;
- name the store roles ETC currently assigns, and say why a fourth role the model declares is not among them.

Chapter 1 stated a principle: duplicate evidence is not corroboration, and Chapter 12 would turn it into a rule. This chapter shows the rule as ETC implements it, and where the implementation falls short of the principle. ETC records each fact it holds and does not force them into one "true" edge.

Level 1 — Intuition

A rota, a delivery note and a receipt

A bakery manager wants to know whether the pastry crew must wait for the dough crew. Three pieces of paper bear on it.

Paper	What it is	Why it is not the whole answer
The rota on the wall	A declaration: pastry follows dough	Someone wrote it; it may be old
The delivery note	Dough was handed to pastry in a tray	It shows a hand-off, not a rule that pastry waits
A receipt for the tray, photocopied twice	The same slip, twice	Two copies of one slip are one fact

Now suppose the rota says pastry follows dough, but the delivery notes show that the *packing* crew takes trays from pastry, and the rota says packing follows dough. The paper does not agree. A good manager keeps both and asks the people on shift.

Three kinds of store

The same bakery has a **stock shelf** (supplies nothing in the bakery makes), a **set-up delivery** (arranged once before work begins) and a **tray** (passed from one crew to the next as work proceeds). ETC calls them *reference*, *seeded* and *pipeline*.

Level 2 — Engineering mechanics

Where producers and consumers come from

A **data flow** is a producer job, a consumer job and a store. ETC builds data flows from the jobs it read.

1. **Producers.** For each job, ETC records which stores it produces: the DD accesses that produce and can carry state between jobs, plus the productions a **utility instruction** states, such as an IDCAMS or copy control statement.
2. **Consumers.** The job's consumed stores come from the same effective steps, utility reads included.
3. **Pairing.** Every producer of a store is paired with every consumer of it. A job is never paired with itself.

The EvidenceGraph turns each flow into a `data_precedes` edge from the producer to the consumer; a flow that touches a shared JCL card name is skipped at that point (Chapter 10). One support is added **per store**: its artefact is the store name, its reader is `data-flow`, its basis is derived, and its severity is critical. If two stores connect the same pair of jobs, there is one edge with two supports.

```
[PSEUDOCODE – PUBLIC EDITION] data flows
producers[store][job] = the job's strongest production of that store, from
- accesses that produce and can carry state, and
- productions a utility instruction states          # utility rules
for job in jobs:
  for store in each store the job consumes:          # utility reads count
    for producer in producers[store], excluding job itself:
      flow(producer -> job, store)                  # one per producer,
                                                    # consumer and store
```

One job, one store, two directions

The graph holds two readings of direction.

- **Access edges** (`reads`, `writes`, `produces`, `consumes`, and the typed forms) follow the **DD disposition**. This is the rule Chapter 7 taught.
- **Data orderings** (`data_precedes`) follow the **data flows**, which apply the utility rules: a utility instruction's productions are added, and a utility instruction overrides the direction the `DISP` implies.

Sometimes they differ. In the fragment below a control statement writes a data set while its DD is coded `SHR`.

```
[PSEUDOCODE] generic JCL, not a Quillon member
//COPY      EXEC PGM=IDCAMS
//TARGET    DD DSN=PROD.TARGET,DISP=SHR
//SYSIN     DD *
            REPRO INFILE(IN) OUTFILE(TARGET)
```

The `DISP=SHR` can give the job a read-type access edge to `PROD.TARGET`, while the utility statement makes the job a producer of it in the data flows. Neither reading is an error, and the graph keeps both.

The measured size of the difference is in CardDemo. Of its 58 data-flow supports (one per producer job, consumer job and store), the producer's own access edge to that store is read-type for **38**, a `produces` edge for 16, and absent for 4. CBSA shows 0 of 21, and GenApp 1 of 2.

[CORPUS-DERIVED: author measurement, 2026-10-05, at the book baseline]

Counting universe. The figures count the `data_precedes` supports in the EvidenceGraph at the baseline, by the producer's own access edge to that store. They describe how ETC's two rules relate on those estates. They are not a statement about the platform. A producer in the ordering model need not have a write-type access edge of its own.

Three things called "corroboration"

An edge keeps a list of supports. Three separate situations concern them.

FIG-12-1 — Duplicate evidence, multiple named supports, and agreement · AUTHOR-CONCEPTUAL, with case study layer B

Panels 1 and 2 illustrate the rule on author-created names. Panel 3 is expected (layer B) (Layer C: no case-study run is recorded in this edition).

```
PANEL 1 – DUPLICATE: the same testimony twice is one support
edge    job:J1 -executes→ program:P1
support artefact J1.jcl | kind jcl | reader: the JCL reader | location "step S1"
again   the same four values -> NOT added
result  1 support. Named supports: 1.   Nothing was learned the second time.

PANEL 2 – MULTIPLE NAMED SUPPORTS: more than one name; independence not tested
edge    job:PRODA -data_precedes→ job:CONSB
support artefact = store S1      (job PRODA writes S1, job CONSB reads S1)
support artefact = store S2      (job PRODA writes S2, job CONSB reads S2)
result  2 named supports -> flagged as corroborated.
        Both came from the same JCL. Nothing independent was checked.

PANEL 3 – AGREEMENT BETWEEN EVIDENCE TYPES    EXPECTED (layer B), not an ETC result
edge E1  job:QMABL020 -scheduled_precedes→ job:QMABL030
          1 support: artefact "Control-M" (the scheduler export)
edge E2  job:QMABL020 -data_precedes→ job:QMABL030
          1 support: artefact "QMA.BILL.RATED.D" (the store)
E1 and E2 stay two edges. Neither is flagged: each has 1 named support.
order finding for the pair (on the comprehension, not an edge):
        DECLARED_AND_REQUIRED    one finding
```

Legend. An arrow is read from the first node to the second. artefact is the name a support gives for its source. In panel 3 a store name and a product name are the sources.

Caption. The three panels show three different situations. A duplicate adds no support. An edge with several named supports is flagged, and the flag counts names. Agreement between a scheduler declaration and a data flow is two edges plus one order finding, and it is not recorded as a single stronger edge. Figure provenance: AUTHOR-CONCEPTUAL, with the Quillon panel as

layer B. The figure does not imply one merged edge, that the corroboration flag means independence, that agreement is corroboration of an edge, or that any panel proves the order correct.

A second *location* in the same artifact is kept as a support, but it does not raise the count of named supports (Chapter 10).

The corroboration flag is true when an edge's supports name more than one distinct artefact string. The code's other name for the count, the independent-support count, counts those distinct strings. The word "independent" in that identifier is a name, not a test.

Because artefact is a member, a store or a product depending on the edge kind, the same flag means a different thing on each kind of edge, and the book calls it **multiple named supports**.

The 5, 17 and 0

[CORPUS-DERIVED: author measurement, 2026-10-05, at the book baseline] EvidenceGraph, imported estates.

	Edges with multiple named supports	Composition
CardDemo	5	Three <code>data_precedes</code> edges that each rest on two stores; one <code>executes</code> edge, <code>TRANBKP</code> to <code>IDCAMS</code> , from a job member and a procedure member; one <code>scheduled_precedes</code> edge, <code>WAITSTEP</code> to <code>OPENFIL</code> , from two scheduler exports
CBSA	17	Nine <code>executes</code> , three read and three <code>consumes</code> edges whose supports are a job member plus procedure members; two <code>data_precedes</code> edges that rest on nine named members of one library
GenApp	0	None

These are not independently corroborated edges. Apart from `WAITSTEP` before `OPENFIL`, every one of the 22 rests on several members or data sets of the same estate.

The one measured case from two scheduler exports. In CardDemo, `WAITSTEP` before `OPENFIL` is the only measured edge with supports from two scheduler exports, CA-7 and Control-M. The supports are not statistically or causally independent. Both exports ship in the same CardDemo repository, so a single decision or a single error in the repository could produce both. And `OPENFIL` is listed in the ambiguous orderings, so the edge is not presented as a firm, mandatory order. The Control-M name recurs across several supports of the edge; repeating a name does not raise the count.

Order findings

Agreement and disagreement between the two kinds of order are reported as findings, not supports. Two terms first.

- **Scheduler order** is what a schedule declares: the `scheduled_precedes` edge.
- **Data-flow order** is what the data requires, a producer before a consumer: the `data_precedes` edge.

For each pair of jobs, the comprehension compares them and records **one order finding**. The finding is not an EvidenceGraph edge and not a third ordering. It is the comparison of one pair.

Verdict	Holds when	Meaning for a reader
DECLARED_AND_REQUIRED	The pair is declared and the data requires it	The two kinds of evidence agree
REQUIRED_NOT_DECLARED	The data requires it, a schedule was read , and it does not declare it	The data needs an order that the schedule read does not state
REQUIRED_ORDER_UNVERIFIED	The data requires it and no schedule was read	Whether anyone declared it is unknown. The channel is missing
DECLARED_NOT_REQUIRED	The pair is declared between two bundle jobs and no data flows between them	An order with no data reason in these artifacts
MUTUAL_CONTENTION	The data requires each job before the other	The data gives no order. Reported once, from the alphabetically first job

```
[PSEUDOCODE – PUBLIC EDITION] order findings
required = {(producer, consumer): stores} from the data flows
declared = {(pred, job)} from predecessors + {(job, succ)} from successors
mutual   = pairs required in both directions
for pair in required, skipping mutual pairs:
    if pair in declared:          DECLARED_AND_REQUIRED
    elif the schedule was read:   REQUIRED_NOT_DECLARED
    else:                        REQUIRED_ORDER_UNVERIFIED
for pair in declared, not in required:
    if both jobs are jobs in the bundle: DECLARED_NOT_REQUIRED
    # a declared pair with a party outside the bundle is not assessed
# Store Role is not consulted
```

Two points to carry.

- **One finding per pair.** Agreement is the single verdict `DECLARED_AND_REQUIRED`. It is not two corroborating findings, and it does not merge the two edges.
- **Findings live on the comprehension.** The full set is returned by the comprehension endpoint; its risks endpoint returns only the required-not-declared, mutual-contention

and unverified findings, with the total. Store Role is not consulted, so an order through a seeded store is still reported.

Findings emitted [CORPUS-DERIVED: author measurement, 2026-10-05, at the book baseline]	CardDemo	CBSA	GenApp
DECLARED_AND_REQUIRED	2	0†	0†
REQUIRED_NOT_DECLARED	47	0†	0†
DECLARED_NOT_REQUIRED	27	0†	0†
MUTUAL_CONTENTION	3	1	0
REQUIRED_ORDER_UNVERIFIED	0†	3	2

† A marked zero is the comprehension's output for the recorded schedule state: `NOT_SUPPLIED` for CBSA and GenApp, `READ` for CardDemo. No such finding can be emitted in that state. The zero is not evidence that the estate has no scheduler relationships.

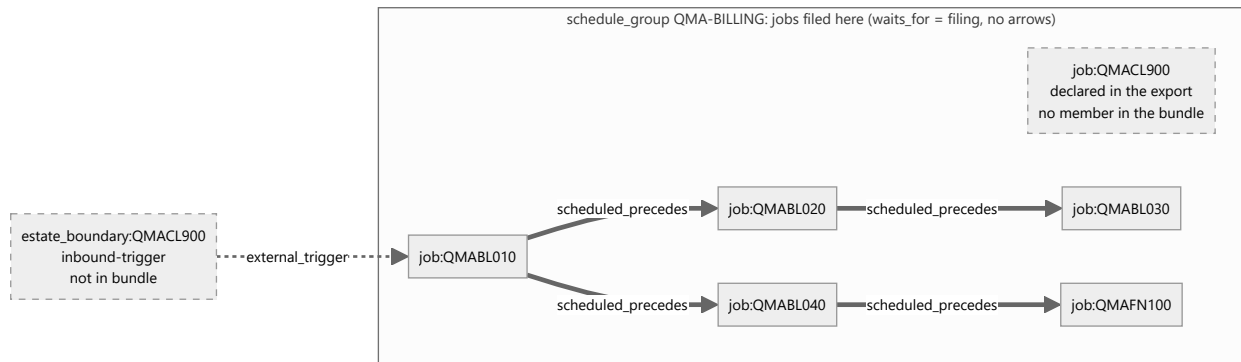
Only CardDemo's bundle supplies a schedule export; CBSA's and GenApp's schedule state is `NOT_SUPPLIED`. CBSA and GenApp therefore cannot have a declared pair, and a one-way required order there can only be unverified.

ETC reports these findings. It does not resolve them, choose a correct order or warn an operator at run time.

FIG-12-2 — Scheduler vs producer/consumer evidence · AUTHOR-CONCEPTUAL (case study), layer B

Expected pedagogical outcome, not an ETC result (Layer C: no case-study run is recorded in this edition); layer C is `UNKNOWN`. An author's drawing that places two edge sets and the findings on one page; ETC produces no such picture. Chapter 11 previews panel A.

Panel A — scheduler layer. Each arrow is one declared order from one condition.



Panel B — data layer. Each arrow is a producer to a consumer through the named store.



Panel C — the order findings (on the comprehension, not edges).

Pair	Scheduler edge	Data edge	Order finding
QMABL010 → QMABL020	yes	yes	DECLARED_AND_REQUIRED
QMABL020 → QMABL030	yes	yes	DECLARED_AND_REQUIRED
QMABL040 → QMAFN100	yes	yes	DECLARED_AND_REQUIRED
QMABL010 → QMABL040	yes	no	DECLARED_NOT_REQUIRED
QMABL030 → QMABL040	no	yes	REQUIRED_NOT_DECLARED
QMAPV001 → QMABL020	no	yes	REQUIRED_NOT_DECLARED
QMACL900 → QMABL010	external_trigger from a boundary	no	Not assessed: QMACL900 is not a bundle job

Legend. ==> is `scheduled_precedes`; -.-> is `external_trigger`; --> is `data_precedes`. Dashed outline: not in the bundle. The box in panel A is filing under a schedule group; it is not a set of arrows.

Caption. The same jobs, two kinds of evidence, different orders. QMACL900 appears as two unlinked nodes, a current ETC limitation. Figure provenance: AUTHOR-CONCEPTUAL, case study, layer B. The figure does not imply that ETC resolves the disagreement, that the drawing is one ETC graph, that QMACL900 has source, or any run that happened.

What the layer-B findings say. QMABL020 to QMABL030 is two single-support edges plus one DECLARED_AND_REQUIRED finding (FIG-12-1, panel 3). For QMABL040, QMABL010 → QMABL040 is declared and not required: the scheduler orders the report after the extract, and no store flows between them. QMABL030 → QMABL040 is required and not declared: the report reads what posting writes, and nothing declares it. The model retains the disagreement. There is no canonical direction, and the chapter picks no winner. A real night could run the report before posting completes; no artifact answers that, and layer C is UNKNOWN.

QMAPV001 → QMABL020 is required and not declared, and so inside the risks subset: the schedule does not schedule QMAPV001 at all.

Store Role

Store Role, the term Chapter 7 reserved for this chapter, is a store's part in the estate across jobs, derived from the data flows and from the purposes of the jobs that write it. It is not a DD semantic role, which Chapter 7 taught per DD per step.

ETC currently assigns **three** roles.

Role	Derivation, as ETC currently runs it
REFERENCE	No data flow writes the store
SEEDED	Every data-flow writer is a provisioning or build job
PIPELINE	Otherwise

The model declares a **fourth**, `TERMINAL` : a store produced here and read by nothing here. **The current derivation never assigns it.** The branch that would assign it needs a store that has a writer and no consumer. But every data flow is built from a producer *and* a consumer, so a store with a writer in the flows always has a reader there. The branch exists in the code and cannot be reached. Measured: no store in any estate has the role.

A declared enum value is not a reachable implementation behaviour. **A data set written here and read by nothing here is reported as REFERENCE, with an outbound estate boundary.** CardDemo has 19 such stores, and all are REFERENCE.

REFERENCE covers more than it sounds like, and none of it means "reference data" or "static data" or "system of record".

REFERENCE also covers
A store no job writes
A store written here and read by nothing here (and so given an outbound boundary)
A store written and read only by the same job
A DB2 table written only through SQL

SEEDED, exactly. A **provisioning** job is one whose utility statements create or delete stores (for example IDCAMS `DEFINE` or `DELETE`) and that runs no program from the bundle. A load by `REPRO` alone does not make a job a provisioning job. A store whose only writers are provisioning or build jobs is SEEDED. What slice discovery does with it is Chapter 13.

```

[PSEUDOCODE – PUBLIC EDITION] store roles
writers[s], readers[s] = producers and consumers of s, from the data flows only
for each store node s:
    if no writers[s]:                role = REFERENCE
    elif no readers[s]:              role = TERMINAL    # not reachable:
                                                # every data flow
                                                # has a consumer

    elif every writer is PROVISION or BUILD:  role = SEEDED
    else:                                    role = PIPELINE

```

[CORPUS-DERIVED: author measurement, 2026-10-05, at the book baseline]	REFERENCE	SEEDED	PIPELINE	TERMINAL
CardDemo	68	9	8	0
CBSA	130	9	0	0
GenApp	69	1	1	0

FIG-12-3 — Store roles and the seeded store · AUTHOR-CONCEPTUAL, with case study layer B

Panels A and B are separate and must not be read together. Panel A is measured; panel B is expected (Layer C: no case-study run is recorded in this edition).

```

THREE ROLES ETC CURRENTLY ASSIGNS
REFERENCE  no data flow writes the store
SEEDED     every data-flow writer is a provisioning or build job
            (provisioning: a utility creates or deletes stores,
            and no bundle program runs; REPRO alone does not qualify)
PIPELINE   otherwise

- - - - -
TERMINAL   declared in the model; NOT assigned by the current
            derivation. A store written here and read by nothing
            here is reported as REFERENCE, with an outbound boundary.
- - - - -

PANEL A – measured (CardDemo, imported estate)
19 stores written and never read      -> all 19 REFERENCE
roles overall: REFERENCE 68 | SEEDED 9 | PIPELINE 8 | TERMINAL 0

PANEL B – expected (Quillon, layer B; not an ETC result)
QMA.RATE.TABLE    written only by QMAPV001 (utility steps, no bundle
                  program)      -> SEEDED

```

Legend. A dashed rule marks a value the model declares and the derivation does not assign.

Caption. The roles ETC currently assigns, and the one it does not. Figure provenance: AUTHOR-CONCEPTUAL, with the Quillon example as layer B. The figure does not imply that `TERMINAL` is emitted, or that a seeded store is ignored: its ordering is recorded and reported, and only its effect on grouping is withheld (Chapter 13).

Quillon's `QMAPV001` and `RATE.TABLE`

`QMAPV001` is the book's load-on-demand job. The case-study specification fixes these properties; the member is written at materialisation. The layer-B column is expected behaviour (Layer C: no case-study run is recorded in this edition).

Element	Layer A: what the artifact must contain	Layer B: expected, by the named rule
Job shape	<code>IDCAMS</code> steps only; no bundle program runs	The job's purpose is provisioning. That needs a utility create or discard instruction and no bundle program
Step 1	<code>DELETE QMA.RATE.TABLE</code>	The delete is a discard instruction
Step 2	<code>REPRO</code> from in-stream input to DD <code>RATETAB = QMA.RATE.TABLE</code> , <code>DISP=(NEW,CATLG,DELETE)</code>	The access edge and the data ordering state the same direction for this job and store
Scheduling	Absent from <code>QMANIGHT.xml</code>	No scheduler edge

Expected consequences (layer B): `RATE.TABLE` is SEEDED; `data_precedes` runs from `QMAPV001` to `QMABL020`; the finding `QMAPV001 → QMABL020` is `REQUIRED_NOT_DECLARED`. ETC has not been run over Quillon.

Quillon, sorted — four stores

The derivation column is expected behaviour (Layer C: no case-study run is recorded in this edition).

Store	Observed (layer A)	Expected derivation (layer B)	Interpreted
POLICY.MASTER	DISP=SHR in QMABL010 and QMABL020; QMARTE01 opens it I-O	A reads_vsam access edge for each job; no data-flow writer; role REFERENCE	"Read-only master." The role does not say that; the program opens it I-O
BILL.AUDIT	No DISP; QMARTE01 opens it EXTEND	A store node, access UNKNOWN, no typed access edge, no data flow; role REFERENCE	"Nothing depends on it." Not supported: the access mode is unknown, not none
PREMIUM_LEDGER	The program text of QMAPST01 contains EXEC SQL INSERT INTO PREMIUM_LEDGER	A DB2 write edge from the program; role REFERENCE, because SQL writes are not data flows	"No job posts to the ledger." SQL writes are outside the data flows
BILL.SUMMARY	Written by QMABL040's REPORT step; read by QMAFN100	PIPELINE: the writer runs a bundle program, so it is not a provisioning job	—

Pseudocode — three rules, one path

Pseudocode written for this book. It is not an excerpt of ETC source. The three blocks above, (a) data flows, (b) order findings and (c) store roles, are one path. The flows feed both the order findings and the roles. Rule (b) never consults the roles, and rule (c) never consults the schedule. Left out: the sorting of results, the per-store mode bookkeeping and the text of the findings.

Code Trace Challenge — flows, findings and roles

Use this generic fragment. It is author-created and not a Quillon member.

```
[PSEUDOCODE] generic estate; all five jobs are in the bundle
JOBA runs bundle program PGMA, which writes DS.X (DISP NEW,CATLG)
JOB B runs bundle program PGMB, which reads DS.X and DS.Y
JOBC runs bundle program PGMC, which reads DS.X
JOB X runs IDCAMS only: DELETE DS.Y, then REPRO from in-stream input to DS.Y
JOB D runs bundle program PGMD, which writes DS.Z (nothing reads DS.Z)
Schedule export (read): declares JOBA before JOBB; nothing else
```

a. Which data flows does ETC derive? b. Which order finding applies to each pair of jobs? c. What role does each of DS.X, DS.Y and DS.Z receive? d. If no schedule export is supplied, what changes?

Answer (by the rules, on an author-created fragment; not an ETC run, and not a tested result). (a)

JOBA → JOBB and JOBA → JOBC through DS.X, and JOBX → JOBB through DS.Y. DS.Z yields no flow, because no job reads it. (b) JOBA → JOBB: DECLARED_AND_REQUIRED. JOBA → JOBC:

`REQUIRED_NOT_DECLARED`, because a schedule was read. `JOBX` → `JOB`: `REQUIRED_NOT_DECLARED`; the seeded store does not exempt it. `JOB` has no pair. (c) `DS.X` is `PIPELINE`: its writer runs a bundle program. `DS.Y` is `SEEDED`: its only writer is a provisioning job. The `DELETE` is what makes `JOBX` a provisioning job; a job whose only utility statement is `REPRO` would not be one, because its purpose would be undetermined, and `DS.Y` would be `PIPELINE`. `DS.Z` is `REFERENCE`, not `TERMINAL`: it has no data-flow writer, because no flow carries it. (d) All three pairs become `REQUIRED_ORDER_UNVERIFIED`: nothing was read, so nothing is declared. The flows and roles do not change.

Level 3 — Architecture reasoning

From principle to rule

Chapter 1 principle	What ETC implements	The gap
Two readings of one artifact are duplicate evidence	A support matching an existing one on artefact, kind, reader and location is not added	A different location in the same artifact is kept, though it adds no new name
Corroboration needs more than one distinct artifact	The flag counts distinct artefact strings on one edge	A string is a member, a store or a product depending on the edge kind. Distinct strings are not independent observations
A scheduler declaration and a data hand-off pointing the same way are two artifacts	Two edges plus one order finding	No single edge records the agreement, and nothing here measures independence

The book introduces **no independence model**. At the level of the claim, `QMABL020` before `QMABL030` rests on two distinct artifacts, and Chapter 1 called that corroboration. ETC does not record it as corroboration of one edge, and neither that agreement nor the flag proves the order is right.

Conflict is kept, not resolved

The EvidenceGraph holds two disagreements, and in neither does it choose. For one job and store, the access edge and the data ordering are both edges, and the graph keeps both. For one pair of jobs, scheduler order versus data-flow order is an order finding on the comprehension, which records the verdict.

The phrases "the true direction" and "the real order" are out of bounds for both. Name the two sources instead.

Failure Case — choosing a winner for QMABL040

A planner rebuilds the night on a target scheduler. Reading the scheduler export, she makes QMABL040 follow QMABL010. Reading the data, a colleague makes it follow QMABL030. Each treats one finding as "the true order" and discards the other. The first plan lets the report run before posting completes. The second removes a serialisation the schedule declared, for a reason the artifacts do not give. The correct output is the pair of findings, with the question handed to the people who run the night. Whether the report really runs before posting completes is UNKNOWN: layer C is unanswered.

Architect's Warning

Corroboration is not correctness, and **multiple named supports are not independent corroboration**. A flagged edge means several names appear among its supports. An agreement between a scheduler and a data flow means two kinds of paper point the same way. Either can be wrong together, because the artifacts may share an author, a repository or a mistake. The report to a decision-maker should say "several named sources", not "confirmed".

Claim Check

"ETC corroborates QMABL020 → QMABL030: the scheduler and the data both support the edge."

This fails three ways. No ETC run over Quillon exists, so the sentence states an expectation as a result. The scheduler and the data are two *separate* edges, each with one support, and neither is flagged. And their agreement is one order finding, DECLARED_AND_REQUIRED, not one stronger edge.

Current Support

Capability	Status	Evidence
Data flows from producers and consumers, with utility rules	IMPLEMENTED	Reviewed at the baseline
data_precedes with one support per store	IMPLEMENTED	Reviewed at the baseline
Duplicate suppression; the multiple-named-supports flag	IMPLEMENTED	Reviewed at the baseline
Five order verdicts per job pair; mutual contention once	IMPLEMENTED	Reviewed at the baseline
REFERENCE, SEEDED and PIPELINE by the derivation above	IMPLEMENTED	SEEDED by preset in

Current Limitation

CURRENT IMPLEMENTATION LIMITATION.

1. **TERMINAL is declared and never assigned.** A store written and read by nothing here is REFERENCE. No test assigns TERMINAL.
 2. **The flag counts names, not independence.** The identifier the independent-support count does not test independence.
 3. **Access edges and data orderings can disagree**, and the graph keeps both. The CardDemo count of 38 of 58 is an observation about ETC's two rules, not a platform rule.
 4. **Order findings are not graph edges.** They are on the comprehension.
 5. **A declared pair with a party outside the bundle is not assessed**, so QMACL900 → QMABL010 has no finding.
 6. **Findings depend on a schedule having been read.** Without one, a required order can only be unverified.
 7. **Quillon is a case study.** Every Quillon outcome here is layer B.
-

Quick Recall

1. Why does a repeat reading of one DD not raise an edge's count of named supports?
2. Name the five order verdicts. Which one means the schedule was not read?
3. WAITSTEP before OPENFIL has supports from CA-7 and Control-M. Is it independently confirmed? Give two reasons.
4. Which store roles does ETC currently assign? What role does a store written and never read receive?
5. For QMABL040, which verdict applies to QMABL010 → QMABL040 and which to QMABL030 → QMABL040?

Key Takeaways

- Data orderings follow producers and consumers after ETC's utility rules; access edges follow the disposition. They can disagree (38 of CardDemo's 58 data-flow supports), and the graph keeps both.
- A duplicate is one support; multiple named supports are several names, independence untested; agreement between evidence types is two edges plus one order finding.
- The 5, 17 and 0 are edges with multiple named supports. WAITSTEP before OPENFIL has supports from two scheduler exports in one repository.
- An order finding compares one pair of jobs; ETC never resolves it.
- ETC assigns three store roles. TERMINAL is declared but unreachable; a store written and never read is REFERENCE.

- Quillon's rows are expected derivations (layer B), with layer C UNKNOWN: QMABL040 is two separate findings with no winner.

Special Early Review Section

HLASM / Assembler Comprehension

These chapters are intentionally included ahead of Chapters 13–20 so that mainframe and assembler practitioners can review and challenge the treatment while the remaining handbook is being completed.

Chapters 21–24 are not renumbered. Chapters 13–20 and 25–33 are forthcoming and appear later in this edition by title only.

Chapter 21 — Reading Assembler as Evidence

SPECIAL REVIEW SECTION — EARLY REVIEW EDITION. This HLASM section appears ahead of some intervening chapters in the Early Review Edition so that assembler and modernization practitioners can review the technical treatment early. The permanent book ordering remains Chapters 1–33. Where a later chapter is not yet in this edition, the text restates what it needs or says the topic is covered in a forthcoming chapter.

WHAT THIS SECTION BUILDS. Deterministic, bounded static comprehension of HLASM source: statements, symbols, locations, raw control flow, bounded linkage evidence, and return proof or refusal.

WHAT THIS SECTION DOES NOT BUILD. A complete assembler. Macro expansion. An emulator. Complete assembler data analysis or CICS analysis. HLASM-to-Java transformation. The limitations stated in each chapter below still apply.

After this chapter you can:

- say why a member under an `asm/` directory is not, by that fact, assembler, and how ETC decides that a member is an assembler program unit;
- read an HLASM statement under the standard source format, and say why a reader's column rules are an assumption it must state;
- tell apart what a directive declares from what a machine instruction does at run time;
- explain what ETC records for symbols, sections, `USING`, `COPY` and macros, and where its location counter gives up;
- state what ETC does not examine in assembler (data access, CICS behaviour) and that it does not translate assembler at all.

Assembler is the hard case for a deterministic reader. A COBOL program says `CALL 'X'`. An assembler module may say nothing that plain: it loads an address into a register and branches through it. Part V is about how far a deterministic reader can follow that without guessing. This chapter covers the ground floor: what can be read from the source text before any claim about control flow or behaviour is made.

Level 1 — Intuition

A ledger in a hand you half read

Imagine an old ledger written in a clerk's private shorthand. You can see the column rules, the dates and the names in the margin. You can see where one page continues onto the next. You can count entries. What you cannot do, without much more work, is say what money moved, because some lines say "as per the usual arrangement" and the usual arrangement is not written down.

Assembler source is like that ledger. The layout is strict enough to read. The names, sections and declarations are visible. But a large part of the behaviour lives in registers whose values the text never states, and in macros whose bodies may be somewhere else entirely.

The evidence ladder

Part V uses one ladder throughout. Each rung is a different kind of statement.

SOURCE SYNTAX	what the text says: fields, names, operands
ARCHITECTURAL EFFECT	what the processor does when an instruction runs
SOFTWARE / LINKAGE CONVENTION	what cooperating programs agree to do with registers and storage
STATIC EVIDENCE	what a reader can establish from the text alone
ETC DETERMINISTIC PROOF	what ETC's rules prove from that evidence
RUNTIME TRUTH	what actually happens on a running system

A statement on one rung does not prove a statement on the next. The text can contain a familiar pattern without the convention behind it holding. The convention can hold without ETC proving it. ETC can prove a relationship without anyone observing it at run time. This chapter stays on the first and fourth rungs.

Level 2 — Engineering mechanics

Physical location is not language type

Chapter 8 established that a member's directory, its semantic type and its translation eligibility are three separate questions. Assembler is where that lesson matters most, because estates often keep assembler in a directory named for it.

Across the three imported estates, six members sit under `asm/`.

Estate	Member	What ETC's reader makes of it today
CardDemo	COBDATFT	Assembler program unit
CardDemo	MVSWAIT	Assembler program unit
CBSA	BANKDATA	COBOL program unit, reported as misfiled
CBSA	BNK1DAM	BMS screen map
CBSA	DFH\$SIP1	Unidentified
GenApp	SSP4A1	Unidentified

Two are assembler, one is COBOL, one is a screen map and two are unidentified. "Under `asm/` " is a location. "Assembler" is a reading of the member's text.

`BANKDATA` is the sharpest example, and Chapter 8 tells it in full. In brief: upstream, CBSA keeps `BANKDATA` as a COBOL source file. ETC's own corpus import step placed it under `asm/`, because the classifier in use at import time mistook the `START` inside the COBOL data name `START-KEY` for an assembler statement. The classifier was later corrected, and the reader now reads the member

as COBOL, but the frozen bundle keeps the old placement. The CBSA project did not file it there. The lesson for this chapter is narrower: a directory name, and even an `asm/` placement made by a careful tool, does not decide what a member is.

Neither do the two unidentified members become assembler by being next to assembler. ETC reports them and does not read them as programs.

How a member becomes an assembler unit

ETC's classifier recognises a member as assembler when one statement has the shape of a section-starting line: an optional name, then blanks, then `CSECT`, `START` or `DFHEIENT` as the operation, ending at a blank or the end of the line. The requirement that the operation end at a blank is what now keeps `START-KEY` out. A matching line that also contains `PROGRAM-ID` is not taken as assembler, and the assembler test runs before the COBOL test.

CURRENT ETC INTERPRETATION. One matching statement makes a member an assembler unit. The classifier does not assemble the member, does not check the rest of it, and treats `DFHEIENT` as an assembler marker, not as evidence of CICS behaviour.

No precision or recall figure exists for this classifier, and the book gives none. It is a deterministic marker test, and Chapter 8's caution applies: it answers "does the text contain this shape?", not "would this assemble?".

Program units: two, not six

When the classifier accepts a member, ETC builds an **assembler program unit** for it. Across the three estates there are exactly two: CardDemo's `COBDATFT` and `MVSWAIT`. CBSA and GenApp have none.

A unit is named after the member's file stem, not after the section name inside it. Its coverage record says what was and was not examined: calls and includes are partial, and data access, relational access and transaction usage are `NOT_EXAMINED`. Keep the two counts apart: six members under `asm/`, two assembler program units.

The validation corpus used later in Part V is not program units at all. It is a set of pinned public assembler repositories used to test the reader (Chapter 9), and it is never read as an estate.

The standard source format

PLATFORM / IBM SEMANTICS ([B5]). Using the standard HLASM source format, a source statement is laid out on an 80-character record. Columns 1–71 hold the statement. A non-blank character in column 72 marks a continuation: the statement carries on in the next record, under the standard continuation convention. Columns 73–80 are an identification and sequence area that the assembler does not treat as part of the statement.

Within the statement field there are up to four entries, in order, separated by one or more blanks:

Entry	Where it is	What it holds
Name	Begins in column 1 under the standard format; a blank column 1 means no name	A symbol naming this statement
Operation	After the name, or after the leading blanks	A machine instruction, an assembler instruction (directive) or a macro name
Operand	After the operation	Zero or more operands separated by commas, with no embedded blanks outside quoted strings
Remarks	After the blank that ends the operands	Free text, ignored by the assembler

The phrase "standard source format" carries weight. A tool that hard-codes column positions is reading that format, and should say so. These positions are defaults of the standard format, not claimed here as invariants of every HLASM parser. The column rules are an assumption the reader makes, and the book states it.

CURRENT ETC INTERPRETATION. ETC's reader handles the standard format only. A module that contains an `ICTL` statement is refused, not read.

Three kinds of operation

The operation entry decides what kind of statement this is, and ETC records one of three classes.

- **Machine instruction.** The assembler turns it into processor instructions, such as `L`, `ST`, `LM`, `BALR` or `BR`. Its run-time effect is defined by the processor architecture, not by the assembler (Chapter 22).
- **Assembler instruction (directive).** It instructs the assembler: start a section, define a constant, declare a symbol external, establish a base register. `CSECT`, `START`, `DSECT`, `ENTRY`, `EXTRN`, `DC`, `DS`, `EQU`, `USING` and `COPY` are directives. A directive is not executed by the processor. Some directives reserve storage or produce data, but none of them is an action at run time.
- **Unknown.** Anything else, including every macro invocation. `CALL`, `SAVE`, `RETURN`, `OPEN`, `GET`, `PUT`, `DCB` and `WTO` are macros, not instructions, and ETC deliberately has no instruction with any of those names. They are recorded as unknown operations.

That last class is the first big limit of the whole section. A COBOL engineer expects `CALL` to be a call. In assembler, `CALL` is a macro whose expansion ETC does not see, so ETC does not count it as one.

Sections: `START`, `CSECT` and `DSECT`

A **control section** is a block of code or data that the assembler and the binder treat as a unit for relocation. One source module can contain several.

PLATFORM / IBM SEMANTICS ([B5]). `START` begins the first executable control section of a source module and can set an initial location counter value. `CSECT` begins an executable control section, or resumes one already begun under the same name. `DSECT` describes a layout (a dummy section) without reserving storage in the object module. A section name is a symbol the binder can see.

Two things follow, and both are easy to get wrong. `START` begins the first executable control section. It does not mean the module has only one, because `CSECT` statements can follow it. And a control section is an assembler and binder construct. It is not, by itself, a business module, a subroutine, a service or an application boundary.

CURRENT ETC INTERPRETATION. ETC records each section with its kind (control or dummy) and uses the section as the unit of its later analysis. A `DSECT` is not treated as a record layout in the EvidenceGraph.

`ENTRY`, `EXTRN` and address constants: declarations

Three directives mention other modules, and all three are declarations.

PLATFORM / IBM SEMANTICS ([B5]). `ENTRY` names a symbol defined in this module so that other modules can refer to it. `EXTRN` declares a symbol that this module refers to and another module defines; the binder resolves it. A `DC V(name)` address constant also makes `name` an external reference. These directives establish symbol visibility and reference relationships for the binder.

None of them says that anything runs. An `ENTRY` symbol may never be entered. An `EXTRN` symbol may never be branched to. A module can carry a V-type constant for a routine it never calls. Chapter 23 shows exactly when ETC will promote a reference into a call; here it is enough to say that a declaration is not a call, not an execution and not a runtime dependency.

CURRENT ETC INTERPRETATION. ETC records `ENTRY`, `EXTRN` and address constants as symbols. None of them, on its own, creates a call or a graph edge.

Symbols, EQU and USING

ETC builds a symbol table for each module: labels, section names, EQU definitions and declared externals. Operands are kept as text. There is no general expression evaluator, so an EQU whose value needs arithmetic over other symbols is recorded, not computed.

USING tells the assembler which register to assume holds a base address, so that it can turn a symbolic reference into base and displacement. It is a promise the programmer makes to the assembler. It is not an instruction and not a data access.

CURRENT ETC INTERPRETATION. ETC records the USING table. It does not treat USING as evidence that storage is read or written, and it does not check that the promised register really holds that address at run time.

COPY: recorded, not expanded

COPY brings another member's source text into the assembly at that point. ETC records the COPY and the member name. It does not open the member and splice its statements in. The includes coverage on an assembler unit is therefore partial: ETC knows a member was named, not what it contributed.

MACRO ... MEND: tracked, not expanded

A macro definition sits between MACRO and MEND. ETC tracks the definition's extent so that its body is not mistaken for ordinary code. A macro *invocation* is an unknown operation, as described above. ETC never expands a macro, whether the definition is in the same module or in a library it was not given.

This is not a small limitation. Macros routinely generate branches, linkage and I/O. On the validation corpus, 228 branch targets are refused because reaching them would require expanding a macro (Chapter 22).

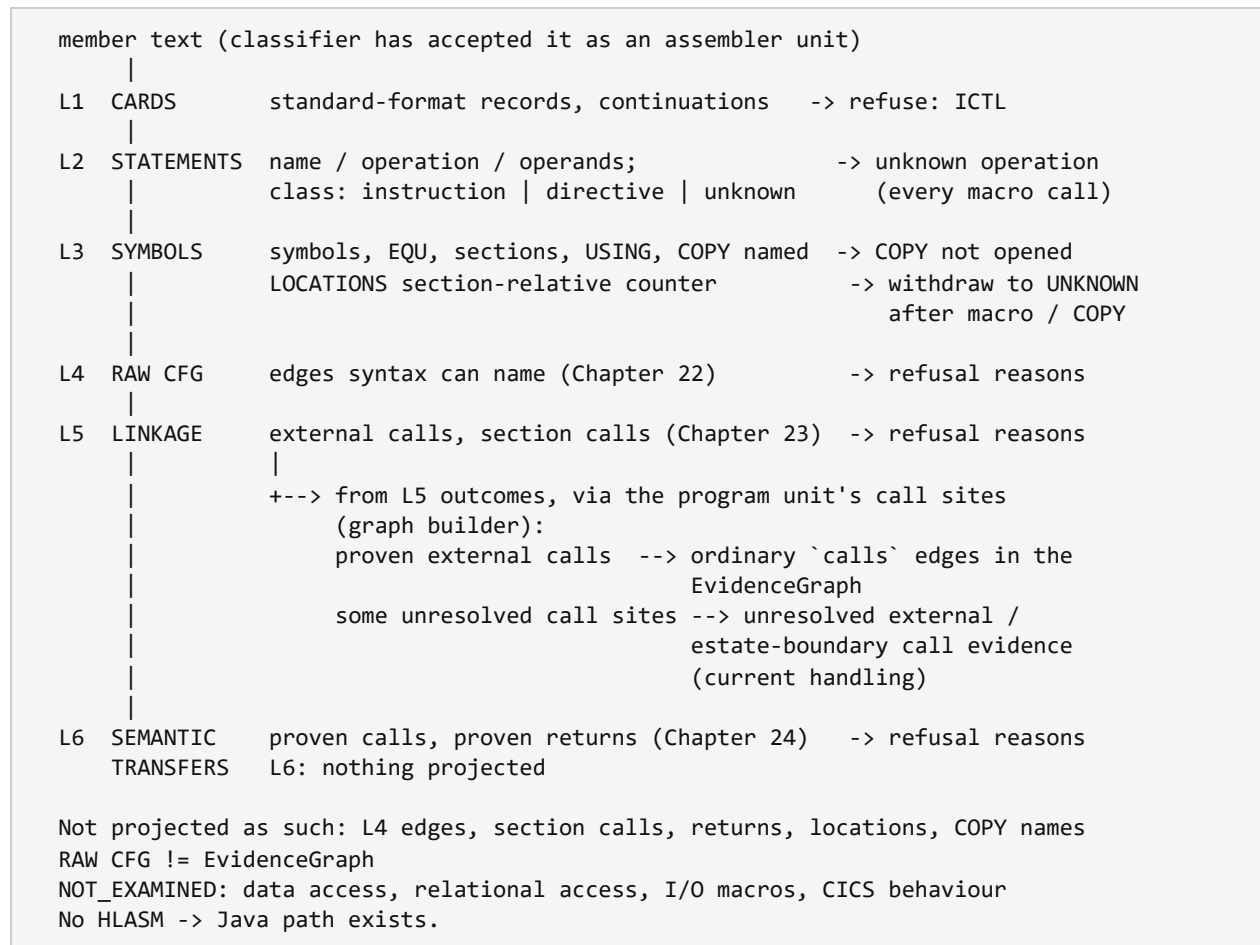
The location counter, and where it gives up

The assembler keeps a **location counter** per section: the offset at which the next byte will be placed. Labels take their value from it, which is how the assembler knows where LOOP is relative to the start of its section.

CURRENT ETC INTERPRETATION. ETC keeps a section-relative location counter for statements whose length it knows. After an unexpanded macro or a COPY, it cannot know how many bytes were produced, so it withdraws to an unknown location for every section, to the end of the module.

The withdrawal is deliberate, not a crash. A location after the first macro could be off by any amount, and every later location would inherit the error. ETC prefers "unknown" to a plausible wrong offset. ETC does not claim assembler-level address calculation: it tracks offsets the text fixes, and stops where the text stops fixing them.

FIG-21-1 — HLASM reading layers · AUTHOR-CONCEPTUAL



Legend. A vertical bar means "the next layer reads the output of this one". An arrow to the right of a layer names the refusal or withdrawal that layer records instead of guessing. The side exits start from layer-5 outcomes, which the graph builder consumes through the program unit's call sites; they are the routes by which linkage analysis contributes `calls` relationships to the EvidenceGraph. Layer 6 contributes nothing to the graph. Nothing else in the figure is projected as an edge.

Caption. ETC reads assembler in six layers, each recording what it cannot establish. Proven external calls are one route by which assembler analysis contributes `calls` relationships to the EvidenceGraph, as ordinary `calls` edges. Current handling can also project some unresolved call sites to external or estate-boundary call evidence. The projection starts from layer-5 linkage outcomes; layer 6 projects nothing. A raw control-flow edge is not projected merely because it is

an edge. Figure provenance: AUTHOR-CONCEPTUAL, from the reader's layer structure as reviewed for this section. The figure does not imply macro expansion, that assembler produces no graph relationships at all, or any translation or conversion path.

Data access and CICS: not examined

CURRENT ETC INTERPRETATION. On an assembler unit, data access, relational access and transaction usage are `NOT_EXAMINED`. I/O macros such as `OPEN`, `GET`, `PUT` and `DCB` are unknown operations. `USING` is not a data access and a `DSECT` is not a record layout. CICS macros are unknown operations, `DFHEIENT` is only a classifier marker, and ETC builds no CICS transaction graph for assembler.

"Not examined" is not "none". ETC's reader does not look, and the coverage record says so. Current ETC derives no CICS transaction behaviour from assembler, and the presence of `DFHEIENT` does not establish such support. A sentence such as "this module touches no files" is not supported by anything in this section.

Pseudocode — from text to locations

Pseudocode written for this book. It is not an excerpt of ETC source.

```
[PSEUDOCODE] reading one assembler member (layers 1-3)
read_member(member):
  for each record:
    if ICTL seen:                refuse module format; stop
    join continuation records    (column 72 marks a continuation)
    split statement into name, operation, operands, remarks
    classify operation as INSTRUCTION | DIRECTIVE | UNKNOWN
    (a macro name is UNKNOWN; MACRO..MEND extents are tracked)
  collect_symbols(statements):
    record labels, EQU names, sections (control | dummy), ENTRY, EXTRN,
    address constants, USING table, COPY member names
    (no expression evaluation; no COPY member is opened)
  assign_locations(statements):
    per section, advance the counter by each known length
    on an UNKNOWN operation or a COPY: every section -> UNKNOWN location
```

The trace covers the reader, the symbol pass and the location pass. Left out: error text, the full instruction-length table and bookkeeping. The location withdrawal after a macro was not separately probed during the review; the rule is read from the code.

Code Trace Challenge — what the reader records

Use this author-created fragment. It is not a Quillon member and not taken from an estate.

```

[AUTHOR EXAMPLE]
PAYX      CSECT
          USING PAYX,12
          L      3,COUNT
          TRACEIT X'02'
LOOP      BCTR  3,0
          ENTRY  PAYXENT
PAYXENT    DS     0H
          EXTRN  TAXCALC
COUNT     DC     F'10'
          END    PAYX

```

a. Which statements are machine instructions, which are directives and which are unknown? b. Does `EXTRN TAXCALC` show that `PAYX` calls `TAXCALC`? c. What location does ETC give `LOOP`?

Answer (by the rules, on an author-created fragment; not an ETC run). (a) `L` and `BCTR` are machine instructions. `CSECT`, `USING`, `ENTRY`, `DS`, `EXTRN`, `DC` and `END` are directives. `TRACEIT` is unknown: it is a macro, or at least not an instruction. (b) No. `EXTRN` declares a reference for the binder; nothing here branches to `TAXCALC`. (c) Unknown. The macro invocation before `LOOP` produced an unknown number of bytes, so the location counter has withdrawn for the rest of the module.

Quillon's assembler member

AUTHOR-DESIGNED SCENARIO – not an ETC result. Quillon, the book's fictional insurer, keeps one assembler member, `QMADTCV`, under `asm/`. Its rating program `QMARTE01` calls it with a COBOL `CALL 'QMADTCV'`. The layer-A content is authored for this book; ETC has not read it.

For this chapter, three facts about the fiction matter:

- `QMADTCV` is one assembly with three executable control sections: `QMADTCV`, `QMADTLP` and `QMADTRT`. The module begins with `QMADTCV CSECT`.
- It defines one macro, `QMATRACE`, between `MACRO` and `MEND`, and invokes it once in the main section.
- It contains no file, database or CICS statement.

EXPECTED DETERMINISTIC RESULT under the rules as reviewed: the classifier accepts the member on its first `CSECT`, so `QMADTCV` is one assembler program unit; the reader records three control sections; the macro invocation is an unknown operation; the location counter withdraws after it; data access, relational access and transaction usage are `NOT_EXAMINED`. NOT AN ETC RUN. LAYER C: UNKNOWN. Chapters 22–24 use the same module for control flow, linkage and return proof.

Level 3 — Architecture reasoning

Why assembler needs evidence discipline

In COBOL, much of what an analyst wants is written in the text: the called program's name, the files opened, the paragraphs performed. In assembler, the same facts are often in registers and storage, set up by instructions or macros the reader cannot fully see. The temptation is to fill the gap with what assembler programs "usually" do.

Part V does not do that. Each layer records what the text fixes and refuses the rest. A refusal is a finding: it tells a reviewer where a person must look. A plausible guess is worse than a refusal, because it is indistinguishable from a fact once it is in a report.

Comprehension, not transformation

CURRENT ETC INTERPRETATION. ETC analyses assembler as evidence. It does not translate it. Current ETC has no HLASM-to-Java transformation path.

Nothing in Part V is a step towards automatic conversion. When a COBOL program calls an assembler module, a conversion of the COBOL caller cannot include the assembler; whether to keep, wrap or rewrite that module is a decision for a person. Part V helps that person by showing what can be established about the module, and what cannot.

Failure Case — the directory inventory

A planning team sizes an assembler rewrite by counting files under `asm/` across three estates: six. They budget for six assembler modules. Two of the six are assembler. One is a COBOL program, one is a screen map and two are members nobody has identified. The budget is wrong by a factor of three, and the COBOL program is now scheduled for an assembler specialist. The correct input to the plan was the program-unit count, two, plus a review task for the two unidentified members.

Architect's Warning

A directive is a declaration. `ENTRY`, `EXTRN`, `DC V(...)` and `CSECT` describe what a module offers, refers to and contains. They do not say that anything ran. An inventory of `EXTRN` names is a list of things the module could reach, which may be a useful starting point for review, but it is not a call graph.

Claim Check

"ETC found six assembler modules and mapped the external routines each one calls from its `EXTRN` statements."

This fails twice. There are six members under `asm/`, of which two are assembler program units. And `EXTRN` declares a reference; it does not show a call. Chapter 23 gives the narrow rule under which ETC does prove an external call.

Current Support

Capability	Status	Evidence
Assembler classification on a section-starting statement	IMPLEMENTED	classifier probe on the six <code>asm/</code> members
Standard-format cards, continuations, statements, operation class	IMPLEMENTED	reader and conformance tests
Symbols, sections, <code>USING</code> , <code>COPY</code> recorded	IMPLEMENTED	symbol tests
Section-relative location counter with withdrawal	IMPLEMENTED	location tests
Assembler program units with a coverage record	IMPLEMENTED	estate probe (two units)

Current Limitation

CURRENT IMPLEMENTATION LIMITATION.

1. **ICTL is refused.** A module that contains an `ICTL` statement is not read.
 2. **No macro expansion and no `COPY` expansion.** Macro invocations are unknown operations; `COPY` members are named, not opened.
 3. **No expression evaluator.** Symbol values that need arithmetic are recorded as text.
 4. **Locations withdraw** after the first unexpanded macro or `COPY`, for the rest of the module.
 5. **Data access, I/O, relational access and CICS are NOT_EXAMINED.**
 6. **No HLASM translation.** None exists and none is described here.
 7. **Quillon is a case study.** Every `QMADTCV` outcome in Part V is layer B.
-

Quick Recall

1. Six members sit under `asm/` across the three estates. How many are assembler program units, and why is the directory not enough?
2. Why does this chapter say "under the standard source format" before giving column numbers?
3. Which of these are declarations rather than run-time actions: `CSECT`, `EXTRN`, `BALR`, `DC V(X)`, `USING`?
4. Why does ETC's location counter stop giving offsets after a macro invocation?

5. What does `NOT_EXAMINED` on an assembler unit's data access permit you to say, and what does it not?

Key Takeaways

- Location is not type: of six members under `asm/`, two are assembler program units. `BANKDATA` is COBOL, placed there by ETC's import step, not by CBSA.
- One section-starting statement makes a member an assembler unit; the classifier does not assemble it, and `DFHEIENT` is a marker, not CICS evidence.
- Column rules are the standard source format, stated as an assumption; ETC refuses a module that contains `ICTL`.
- Directives declare. `START`, `CSECT`, `ENTRY`, `EXTRN` and address constants do not show that anything runs.
- `COPY` is named and macros are tracked, neither is expanded, and the location counter withdraws rather than guess.
- Data access and CICS behaviour are not examined, and ETC does not translate assembler.

Chapter 22 — Bounded Control Flow: What Syntax Can Name

SPECIAL REVIEW SECTION — Early Review Edition (see Chapter 21).

After this chapter you can:

- define ETC's raw control-flow graph and say what it is not;
- state, for `BR`, `BAL / BALR`, `BAS / BASR` and `BCTR`, what the processor does, and keep that apart from what ETC concludes;
- explain why `BALR`, `BASR` and `BCTR` with register 0 as the second operand do not branch, and why that makes the mnemonic alone a poor guide;
- say why a branch through a register stays unnamed at this layer, and why an unnamed target is better than an invented one;
- read ETC's refusal reasons on the validation corpus without treating them as estate findings.

Chapter 21 stopped at statements, symbols and locations. This chapter asks the next question: which control-flow edges can be named from the source text alone, deterministically? The answer is "the ones the text names, and not the ones reached through a register", and the reason is registers.

Level 1 — Intuition

Signposts and phone numbers

A road map shows signposts: "Leeds 12 miles". You can draw that edge without leaving your desk. Now picture a signpost that reads "turn here and go wherever the note in your glovebox says". You can draw that there is a turn. You cannot draw where it goes until you know what is in the glovebox, and who put it there.

Assembler branches come in both kinds. `B LOOP` names its destination in the text. `BR 14` says "go to the address in register 14", and the text at that point does not say what register 14 holds. A careful map-maker draws the first kind as an edge and marks the second as a turn to an unknown place. A careless one writes in the destination he expects and moves on.

Three separate questions

For each branch instruction, Part V keeps three questions apart.

Label	Question
PLATFORM / IBM SEMANTICS	What does the processor do when this instruction runs?
CURRENT ETC INTERPRETATION	What does ETC record or prove about it today?
BOOK / EVIDENCE METHODOLOGY	What would it take to claim more, and who supplies that?

IBM's documentation answers the first question. ETC's code answers the second. The third is this book's method. Documentation never proves what ETC does, and ETC's behaviour never redefines what an instruction does.

Level 2 — Engineering mechanics

The raw control-flow graph

A **control-flow graph** (CFG) is a set of blocks of statements and the edges along which control can pass between them. ETC builds a bounded one for each assembler section. The book calls it the **raw CFG**, because it is built from syntax and local symbols alone, before any reasoning about what registers hold.

Every outgoing transfer gets one of four kinds:

Kind	Meaning
FALLTHROUGH	Control continues with the next statement
STATIC_DIRECT_BRANCH	An unconditional branch to a label the text names
STATIC_CONDITIONAL_BRANCH	A branch to a named label under a condition, plus the fall-through
UNRESOLVED_TRANSFER	A transfer whose destination the text does not fix, with a refusal reason

There is no `CALL` kind and no `RETURN` kind. That is not an omission. Whether a transfer is a call or a return is not something syntax can establish, and the raw CFG does not pretend otherwise.

The raw CFG is not the EvidenceGraph of Chapter 10. It is not an application architecture diagram either. It lives inside one assembler section and is never projected into the graph.

Where the raw CFG sits

Three of the six layers in FIG-21-1 deal with control. This chapter is about the first; the next two chapters cover the others.

- **Layer 4 — raw CFG.** Edges the text can name, and refusals for the rest.
- **Layer 5 — linkage.** Narrow patterns that let ETC prove an external call or a call between sections of one assembly (Chapter 23).
- **Layer 6 — semantic transfers.** Pairing layer-4 transfers with layer-5 evidence and register provenance to prove a call or a return, or refuse (Chapter 24).

A layer may refuse what a later layer resolves. A layer-4 `UNRESOLVED_TRANSFER` is not an error. It is the place where layer 5 and layer 6 are allowed to look.

The control-transfer vocabulary

ETC carries a table of 31 instruction spellings that can transfer control. Each has a role, `BRANCH` or `CALL`, a target form (label, register or relative offset) and a condition. The `CALL` role is attached to the branch-and-link family, `BAL`, `BALR`, `BAS`, `BASR`, `BRAS` and `BRASL`. Everything else in the table, including `BR`, `BCR`, `BCT`, `BCTR`, the conditional branches and the relative jumps, has the `BRANCH` role.

Two warnings. First, "CALL role" is ETC's label for an instruction family. It is a reminder that the instruction stores a link, not a finding that a call happened. Second, the table also carries an evidence-tier column recording how ETC's authors observed each spelling. That column is an ETC provenance label. It is not a statement about the platform.

Instructions outside the table, such as `BXH`, `BXLE`, `BSM` and `BASSM`, are not modelled. Where one of them may transfer control, the raw CFG records `OPERATION_NOT_MODELLED`. That is a refusal, not a statement that no branch happens.

BR : a branch through a register

PLATFORM / IBM SEMANTICS ([B7]). `BR R` is the extended mnemonic for `BCR 15,R`, branch on condition with a mask that is always satisfied. Control passes to the address in register R. No link address is stored anywhere. If the register field is 0, no branch occurs.

`BR 14` is the most familiar line in mainframe assembler, because register 14 conventionally holds a return address (Chapter 24). But the processor does not know that. It branches to whatever address is in register 14 at that moment, whoever put it there.

CURRENT ETC INTERPRETATION. At layer 4, `BR R` with a non-zero register is an `UNRESOLVED_TRANSFER` with the refusal `REGISTER_TARGET`, and no fall-through. ETC treats `BR` as branch evidence only. It does not, at this layer or from the register number, conclude that a `BR` is a return, a call or an external invocation.

BAL and BALR : branch and link

PLATFORM / IBM SEMANTICS ([B7]). `BAL R1,address` and `BALR R1,R2` place link information, which includes the address of the next sequential instruction, in register R1, and then branch: `BAL` to the address its operand designates, `BALR` to the address in register R2. When the R2 field of `BALR` is 0, the link information is placed in R1 and no branch occurs.

"Branch and link" names what the instruction does to the processor's state. It is the instruction a programmer uses when the destination is expected to come back, so it is the natural building block of a subroutine call. But a branch-and-link instruction is not an application-level call. It can enter a local routine three lines away, a section of the same assembly, another program or a table of branches. Some programs use it with no intent to return at all.

CURRENT ETC INTERPRETATION. `BAL 14,LABEL` to a label is a `STATIC_DIRECT_BRANCH` at layer 4, keeping its `CALL` role, plus a fall-through edge for the expected return. Layer 5 reads only the register forms, so a `BAL` to a label is never an ETC external call or section call. `BALR` with a non-zero R2 is an `UNRESOLVED_TRANSFER` with `REGISTER_TARGET` at layer 4. It becomes a proven call only if layer 5 recognises one of the narrow patterns in Chapter 23.

BAS and BASR : branch and save

PLATFORM / IBM SEMANTICS ([B7]). `BAS R1,address` and `BASR R1,R2` place the address of the next sequential instruction in register R1 and branch, as `BAL` and `BALR` do. They differ from `BAL` and `BALR` in what else is placed in R1 in the older addressing modes. When the R2

field of `BASR` is 0, the address is placed in R1 and no branch occurs.

The difference between link and save information matters to a programmer who inspects the high-order bits of R1 later. It does not change anything in this chapter, and the book does not need it.

CURRENT ETC INTERPRETATION. As for `BAL` and `BALR`: a label target is a static direct branch with role CALL; a register target with non-zero R2 is unresolved at layer 4, and only Chapter 23's patterns can promote it.

Register zero: when branch-and-link does not branch

This is the best small example in Part V of why a mnemonic is not a meaning.

PLATFORM / IBM SEMANTICS ([B7]). In `BALR R1,0` and `BASR R1,0`, register 0 in the second operand means "no branch address". R1 receives the link or next-instruction address, and execution continues with the next instruction.

The classic use is establishing a base register:

[AUTHOR	EXAMPLE]	
BALR	12,0	R12 <- address of the next instruction; no branch
USING	*,12	tell the assembler R12 holds this address

The instruction's family name says "branch and link". This form links and does not branch. An analyser that looks only at the mnemonic will see a call to somewhere, when nothing was called at all.

CURRENT ETC INTERPRETATION. At layer 4, ETC gives `BALR R1,0` and `BASR R1,0` a fall-through edge only, with no unresolved transfer. Static analysers need an explicit special case for the no-branch register-zero form at every layer that reasons about branch-and-link instructions. Handling of that form outside layer 4 is a current implementation limitation.

The validation corpus contains no `BALR` or `BASR` with register 0, so no measured figure in this chapter depends on it. Quillon's `QMADTCV` is designed without it; its base register is set by `LR 12,15`.

`BCTR`: decrement, then perhaps branch

PLATFORM / IBM SEMANTICS ([B7]). `BCTR R1,R2` subtracts 1 from register R1. If the result is not zero and R2 is not 0, control passes to the address in R2. If R2 is 0, R1 is still

decremented and no branch occurs, whatever the result.

`BCTR R1,0` is therefore a two-byte "subtract one" that never branches, and programmers use it that way. The count behaviour and the branch behaviour are separate: the decrement always happens, the branch depends on both the result and the second operand.

The register-zero rule is specific to each instruction that defines it. It holds for `BALR`, `BASR`, `BCTR` and `BCR` (so also for `BR 0`), because those instructions say so. It is not a general statement that register 0 means "nothing" in every operand of every instruction.

CURRENT ETC INTERPRETATION. `BCTR R1,0` is `EXECUTE_WITHOUT_BRANCHING` at layer 4: a fall-through only. `BCTR R1,R2` with a non-zero `R2` is an unresolved transfer with `REGISTER_TARGET`. ETC does not examine it further at layer 6; it is out of scope for that phase.

Label targets: static, and not calls between programs

When a branch names a label in the same section, ETC can draw the edge. That is true of `B`, of the conditional branches and also of `BAL` and `BAS`.

CURRENT ETC INTERPRETATION. `BAS 14,LABEL` produces a `STATIC_DIRECT_BRANCH` with role `CALL`, plus a fall-through. It is a branch within the module, not a call between programs, not a section call and not an EvidenceGraph relationship.

A COBOL engineer may find that strange: a local subroutine entered by `BAS` is very like a `PERFORM`. The book agrees with the analogy and keeps the distinction anyway. The `BAS` names a destination in the same section; layer 4 draws it. Whether control comes back from that routine is a layer-6 question, and as Chapter 24 shows, a static `BAS` gives layer 6 no proven call context.

Branch through a register: why it stays unnamed

Every branch whose destination is a register value is unresolved at layer 4: `BR R`, `BALR R1,R2`, `BASR R1,R2`, `BCTR R1,R2` and `BCR M,R`, all with a non-zero register. The reason is not that ETC has not tried hard enough. At layer 4 the reader has the text and local symbols, and the text at a `BR` does not contain the address. The value was set somewhere else: by a load three lines earlier, by a load from storage, by the caller, by a macro, or by arithmetic.

BOOK / EVIDENCE METHODOLOGY. An unnamed destination is a true statement about the evidence. A named one that was guessed is a false statement that looks like evidence. In a review, the first is a work item; the second is a hidden defect.

Refusal reasons

Each `UNRESOLVED_TRANSFER` carries one reason. On the validation corpus (`VALIDATION CORPUS – NOT AN ESTATE`), at the book baseline, the census records:

Raw outcome	Count
<code>FALLTHROUGH</code>	4,664
<code>STATIC_CONDITIONAL_BRANCH</code>	515
<code>STATIC_DIRECT_BRANCH</code>	156
<code>EXECUTE_WITHOUT_BRANCHING</code>	84
<code>UNRESOLVED_TRANSFER</code>	341

and the 341 unresolved transfers by reason:

Refusal	Count	What it means
<code>TARGET_REQUIRES_MACRO_EXPANSION</code>	228	The destination is, or depends on, a macro invocation
<code>REGISTER_TARGET</code>	96	The destination is in a register
<code>TARGET_OPERATION_UNKNOWN</code>	7	The destination statement's operation is unknown
<code>OPERAND_NOT_READ</code>	4	The operand could not be read
<code>TARGET_NOT_DECLARED</code>	4	The named destination is not a symbol the reader holds
<code>EXPLICIT_ADDRESS_EXPRESSION</code>	2	The operand is an explicit base and displacement

All 84 execute-without-branching outcomes are branch-on-count forms with register 0 as the branch address. These are validation-corpus figures. They describe how ETC's reader behaves on a set of public test repositories; they are not counts for any estate. Chapter 24 follows the 96 register targets through layers 5 and 6.

These reasons are what ETC records today, not an exhaustive account of every way an assembler branch can be hard to resolve.

What stays out of the EvidenceGraph

CURRENT ETC INTERPRETATION. No layer-4 edge reaches the EvidenceGraph: not a fall-through, not a static branch, not an unresolved transfer. Three passing projection tests guard that boundary for control flow, section calls, returns and locations. Assembler analysis contributes `calls` relationships by another route: proven external calls become ordinary `calls` edges, and current handling can also project some unresolved call sites to external or estate-boundary call evidence (Chapter 23).

The point of the boundary is to keep two kinds of graph apart. The raw CFG describes control inside one section of one module. The EvidenceGraph describes relationships between programs, jobs and stores. Putting the first into the second would turn every branch into an apparent dependency.

FIG-22-1 — Raw CFG vs semantic transfer · AUTHOR-CONCEPTUAL, with case study layer B

Expected under the rules as reviewed; not an ETC result (Layer C: no case-study run is recorded in this edition).

QMATDCV main section – statements in order (excerpt; layer A)		LAYER 4 (raw CFG)
B	QMATDTR branch to the macro invocation	UNRESOLVED
TARGET_REQUIRES_MACRO_EXPANSION		
BE	QMATDNT conditional, over the macro, to a label	
STATIC_CONDITIONAL_BRANCH		-> QMATDNT, + fall-
through		
BAS 14,QMATDFX	branch-and-link to a label	STATIC_DIRECT_BRANCH,
role CALL		-> QMATDFX, + fall-
through		
L 15,=V(QMALOG)		
BALR 14,15	register target	UNRESOLVED
REGISTER_TARGET		
L 15,=A(QMATDLP)		
BASR 14,15	register target	UNRESOLVED
REGISTER_TARGET		
BR 14	register target (main exit)	UNRESOLVED
REGISTER_TARGET		
layer 4 stops here: registers are not read		
v		
LAYER 5 / 6 (semantic transfer; Chapters 23-24)		
BALR 14,15 after L 15,=V(QMALOG)	-> proven external call to QMALOG	(Chapter 23)
BASR 14,15 after L 15,=A(QMATDLP)	-> proven section call	(Chapter 23)
BR 14 (main exit)	-> refused	(Chapter 24)
BAS 14,QMATDFX	-> nothing: a static branch is not a layer-5 call	

Legend. -> after a layer-4 kind names the destination the text fixes. In the lower panel, -> names the layer-5 or layer-6 outcome expected for that statement. Register-zero forms do not appear: the module contains none, and at layer 4 they would not branch.

Caption. Layer 4 draws what syntax names and marks register destinations as unresolved. Layers 5 and 6 may later pair some of those with linkage evidence. The static BAS is a branch, not a call between programs, and the macro target is refused. Figure provenance: AUTHOR-CONCEPTUAL, with the Quillon content as layer B. The figure does not imply that the raw CFG is wrong to leave register destinations unnamed, or that any BR is a return.

Pseudocode — one section's raw CFG

Pseudocode written for this book. It is not an excerpt of ETC source.

```
[PSEUDOCODE] layer 4 for one control section
build_raw_cfg(section):
    for each statement in order:
        if operation is not a control transfer:           FALLTHROUGH
        elif operation is not in the vocabulary
            and may transfer control:                     UNRESOLVED (OPERATION_NOT_MODELLED)
        elif branch-address operand is register 0
            (BALR, BASR, BCTR, BCR):                     FALLTHROUGH only
        elif target is a register:                       UNRESOLVED (REGISTER_TARGET)
        elif target needs a macro expanded:              UNRESOLVED
        (TARGET_REQUIRES_MACRO_EXPANSION)
        elif target is a declared label:                 STATIC_DIRECT or STATIC_CONDITIONAL
                                                         (role kept: BRANCH or CALL)
                                                         + FALLTHROUGH where control may
    continue
    else:                                                 UNRESOLVED (with the matching reason)
```

Left out: the full reason list, the mask arithmetic for conditional branches and the block-building bookkeeping. The census figures above come from a test that runs this layer over the validation corpus.

Code Trace Challenge — name what you can

Use this author-created fragment, assumed to be one control section with no macros.

```
[AUTHOR EXAMPLE]
    LR    12,15
    USING SUBA,12
    LA     3,5
LOOP      BCTR 3,0
          LTR  3,3
          BNZ  LOOP
          BAL  14, FIXUP
          L    15, ADDR
          BALR 14,15
          BR   14
FIXUP     BR   14
ADDR      DC   A(LOOP)
```

a. Which transfers does layer 4 name, and which are unresolved? b. Does `BCTR 3,0` branch? c. Is `BAL 14, FIXUP` a call between programs?

Answer (by the rules, on an author-created fragment; not an ETC run). (a) Named: `BNZ LOOP` (conditional, plus fall-through) and `BAL 14, FIXUP` (static direct, role CALL, plus fall-through). Unresolved with `REGISTER_TARGET : BALR 14, 15` and both `BR 14`. (b) No. It decrements R3 and falls through; the loop is made by `BNZ`. (c) No. It is a branch to a label in the same section. Layer 4 draws it; layer 5 does not read label forms.

Level 3 — Architecture reasoning

Unknown is a better answer than invented

Every unresolved transfer is a place where the module's behaviour depends on something the text does not fix. A tool that fills those places with expected destinations produces a cleaner diagram and a less truthful one. If such a diagram is used to scope work, the invented edge can become work nobody budgeted for, or a dependency nobody tested. This is the book's methodology, not a finding about any programme.

ETC's raw CFG is bounded on purpose. It names what syntax can name, and it says why it stopped everywhere else.

What a reviewer does with an unresolved transfer

An unresolved transfer is only useful if someone acts on it, and the refusal reason says what kind of action fits.

Reason	What it usually asks of a reviewer
<code>TARGET_REQUIRES_MACRO_EXPANSION</code>	Find the macro definition and the assembler listing for this module; the expansion decides where control goes
<code>REGISTER_TARGET</code>	Wait for layers 5 and 6. If they also refuse, trace by hand where the register was set, or rely on a runtime trace
<code>TARGET_OPERATION_UNKNOWN</code> , <code>OPERATION_NOT_MODELLED</code>	Identify the operation; it may be a macro, a privileged instruction or an instruction outside ETC's vocabulary
<code>TARGET_NOT_DECLARED</code> , <code>OPERAND_NOT_READ</code>	Check whether the source is complete: a missing <code>COPY</code> member or a library macro often explains it
<code>EXPLICIT_ADDRESS_EXPRESSION</code>	Read the base register's <code>USING</code> and set-up; the destination depends on run-time register contents

These are suggestions about where evidence is likely to be, not ETC behaviour. The useful habit is to sort unresolved transfers by reason before reading them one by one. A module whose transfers are mostly macro targets needs its macro library. A module whose transfers are mostly

register targets needs the linkage analysis of the next two chapters, and possibly a person who knows how it is called. Either way, the refusal has already done the first part of the work: it has said precisely which question the text could not answer.

The same habit protects against a quieter error. A module with no unresolved transfers is not thereby "fully understood". It may simply be small, or its interesting behaviour may sit in a macro whose invocation is not a branch at all. The absence of refusals is a statement about branches, not about the module.

Failure Case — the base register that became a call

An analyser scans an estate for branch-and-link instructions and reports each one as "calls an external routine". It finds dozens of `BALR 12,0` lines at module entries and reports dozens of calls to an unknown routine. A planning team schedules an investigation into a routine that does not exist. The architecture is clear: with register 0 as the second operand, `BALR` does not branch. The mistake was letting the instruction family name stand in for its effect.

Architect's Warning

The CALL role in ETC's vocabulary, the words "branch and link", and the register number 14 are all hints. None is proof of a call, and none is proof of a return. Treat a branch-and-link as "an instruction that stores a link and may branch", and ask the next two layers what can be proved.

Claim Check

"ETC's control-flow graph shows that `QMADTCV` calls `QMADTFX` and returns to its caller through `BR 14`."

This fails three ways. The raw CFG has no call or return kinds: `BAS 14, QMADTFX` is a static branch within the section. `BR 14` is unresolved at layer 4. And `QMADTCV` is a case-study design that ETC has not read; any outcome for it is expected, not observed.

Current Support

Capability	Status	Evidence
Raw CFG with four transfer kinds per section	IMPLEMENTED	CFG tests
31-spelling control-transfer vocabulary with roles	IMPLEMENTED	vocabulary tests
Register-zero forms of <code>BALR</code> , <code>BASR</code> , <code>BCTR</code> as no-branch at layer 4	IMPLEMENTED	direct probe
Refusal reasons on every unresolved transfer	IMPLEMENTED	census on the validation corpus
No layer-4 edge in the EvidenceGraph	IMPLEMENTED	projection tests (passing)

Current Limitation

CURRENT IMPLEMENTATION LIMITATION.

1. **Register destinations are unnamed at layer 4**, by design. Only layers 5 and 6 can pair some of them with evidence.
2. **Macro targets are refused** (228 on the validation corpus): no macro is expanded.
3. **Some instructions are not modelled** (`BXH`, `BXLE`, `BSM`, `BASSM` and others) and are refused where they may transfer control.
4. **The register-zero form outside layer 4** is a current implementation limitation.
5. **All census figures are validation-corpus figures**, not estate figures.
6. **Quillon is a case study**. `QMAADTCV`'s CFG is layer B.

Quick Recall

1. Why does the raw CFG have no `CALL` or `RETURN` kind?
2. What does the processor do for `BALR 12,0`, and what lesson does that hold for an analyser that reads mnemonics?
3. `BCTR 5,0` and `BCTR 5,7`: which can branch, and when?
4. Why is `BAS 14,LABEL` a static branch and not a call between programs in ETC's raw CFG?
5. Why does the book prefer an unresolved transfer to an expected destination?

Key Takeaways

- The raw CFG names what syntax fixes and refuses the rest, with a reason. It is not the EvidenceGraph and not an architecture diagram.
- `BR` branches to whatever its register holds and stores no link. ETC treats it as branch evidence only.

- Branch-and-link stores a link and may branch. It is not, by itself, an application-level call.
- With register 0 as the second operand, `BALR`, `BASR` and `BCTR` do not branch. The rule is specific to those instructions.
- Register destinations stay unnamed at layer 4. Layers 5 and 6 decide what can be proved.
- Census figures belong to the validation corpus, not to any estate.

Chapter 23 — Linkage: External Calls and Intra-Assembly Section Calls

SPECIAL REVIEW SECTION — Early Review Edition (see Chapter 21).

After this chapter you can:

- explain what V-type and A-type address constants express, and why neither is an executed call;
- state the exact pattern under which ETC proves an external call, and the five facts that pattern needs;
- say what "logical, not physical" means for an ETC external call;
- explain an intra-assembly section call and why it is kept out of application `CALLS`;
- say which assembler-derived relationships can reach the EvidenceGraph, and which never do.

Chapter 22 left most register branches unresolved. This chapter asks when there is enough evidence to promote one of them into a proved call. ETC's answer is narrow on purpose: two patterns, each built from a few adjacent statements whose meaning the text fixes. Everything else stays unresolved or, in two cases, is recorded as not a call.

Level 1 — Intuition

A directory entry is not a phone call

An office phone directory lists the extension for the payroll team. That entry shows that the office can reach payroll. It does not show that anyone rang them today. Now suppose you find a note on a desk: "picked up the receiver, dialled the number written on this card". If the card is stapled to the note, you know who was called. If the note says "dialled the number on the card in the drawer", you know a call was made but you need to open the drawer to know where.

In assembler, an address constant is the directory entry. A branch-and-link through a register is the dialling. ETC proves a call only when the card is stapled to the note: when the load of the address and the branch through it sit next to each other and the address names its target directly.

Two kinds of linkage

Assembler code reaches other code in two ways that matter here.

- **External linkage:** into another module, which the binder brings into the same load module or program object, or makes reachable by other means.
- **Intra-assembly linkage:** into another control section of the same assembly.

Both can be written with the same instruction, `BALR 14,15` or `BASR 14,15`. What distinguishes them is how register 15 got its value. That is why linkage is a separate layer from the raw CFG.

Level 2 — Engineering mechanics

Address constants are evidence, not execution

An **address constant** is a `DC` (define constant) whose value is an address. The assembler reserves storage for it, and the address is filled in when the module is assembled and bound. Two types concern this chapter.

PLATFORM / IBM SEMANTICS ([B5]). A **V-type** address constant, `DC V(name)` or the literal `=V(name)`, expresses the address of an external symbol. Writing it makes `name` an external reference, as an `EXTRN` would, and the binder resolves it. An **A-type** address constant, `DC A(expression)` or the literal `=A(expression)`, holds the value of an expression, usually a relocatable address within the assembly. An A-type constant can also refer to a symbol defined in another module if that symbol is declared with `EXTRN`.

So the tempting rule "A is local, V is external" is too simple. A V-type constant always expresses an external reference. An A-type constant usually expresses an address the assembly can resolve itself, and can express an external one when paired with `EXTRN`.

What both types share is that they are data. A V-type constant can sit in a module for years without any instruction loading it. An A-type constant can be an entry in a table that is never indexed. An address constant shows that an address is represented. It does not show that control ever goes there.

What the binder does, and what it does not

The **binder** resolves external references when the executable program is constructed. The statement below is the only binder behaviour this chapter relies on.

PLATFORM / IBM SEMANTICS ([B5]). The binder resolves the external symbol in a V-type constant. Link-time processing may make a branch through a V-type constant indirect, so the resolved address may lead to intermediate linkage code rather than directly to the referenced symbol. IBM's documentation recommends V-type constants for branching to external addresses for this reason.

Two consequences for evidence. First, the existence of a V-type constant shows that the module refers to `name` for binding. It is reference availability, not invocation. Second, even when a call through it does happen, the address in the register at the moment of the branch may not be `name`'s own entry point. Whatever ETC proves, it does not prove a physical address.

FIG-23-1 — V-type vs A-type constants · SOURCE-DERIVED + AUTHOR-CONCEPTUAL

```
PLATFORM / IBM SEMANTICS (declarations only; [B5])
  VQMALOG2 DC V(QMALOG)      external reference to QMALOG; the binder resolves it;
                             a branch through it may be made indirect at link time
  ADTAB     DC A(QMADTLP)     address of section QMADTLP in this assembly
                             neither line is executed; neither shows a call

CURRENT ETC INTERPRETATION (what layer 5 follows)
  L 15,=V(QMALOG) then BALR 14,15  -> proven external call (logical)
  L 15,VQMALOG2   then BALR 14,15  -> not followed: CURRENT ETC LIMITATION
                                   (the source still expresses linkage)
  L 15,=A(QMADTLP) then BASR 14,15 -> intra-assembly section call
                                   (QMADTLP is another executable section);
                                   not an application call
  ADTAB DC A(QMADTLP), never loaded -> nothing
```

Legend. `->` names the layer-5 outcome for the two-statement sequence on its left. The upper panel is platform meaning; the lower panel is ETC's current rule.

Caption. A V-type constant declares an external reference; an A-type constant holds an address. ETC follows only a literal V-type operand on a load adjacent to the branch, and only an A-type literal naming another executable section of the same assembly. Figure provenance: SOURCE-DERIVED ([B5]) for the upper panel; AUTHOR-CONCEPTUAL for the lower. The figure does not imply that the register holds `QMALOG`'s physical address, that any declaration proves a call or execution, or that a labelled V-type constant is not a call as a matter of platform truth.

ETC's external-call rule

CURRENT ETC INTERPRETATION. ETC's layer 5 proves an external call only for this shape:

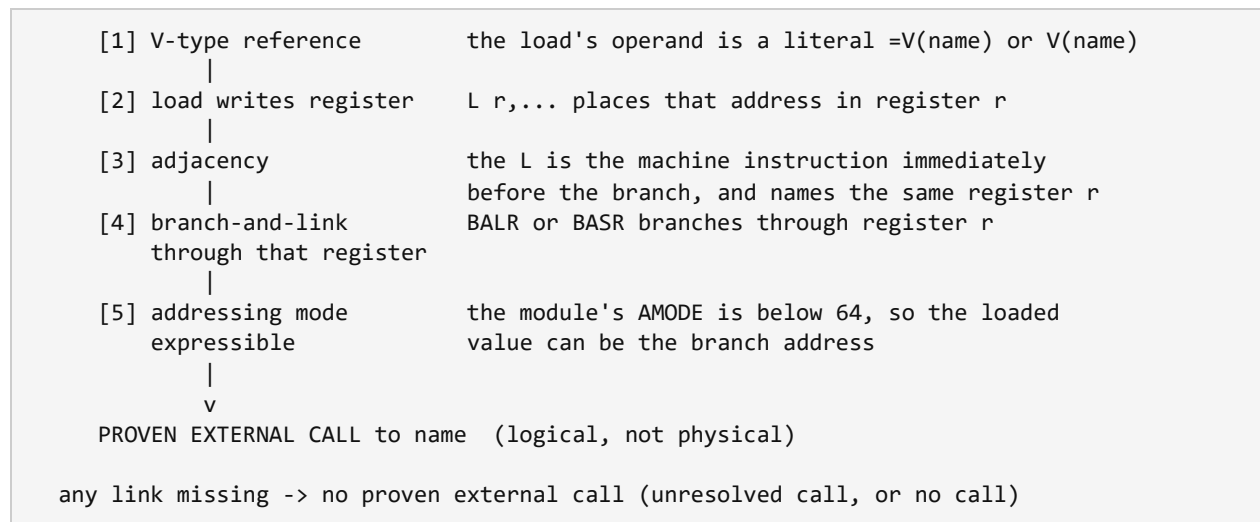
```
text L r,=V(name) (or L r,V(name)) BALR x,r (or BASR x,r), the next machine instruction
```

with the module's addressing mode below 64-bit.

In words: a `BALR` or `BASR` branches through register `r`; the machine instruction immediately before it is a load (`L`) into the same register; the load's operand is a V-type literal naming the target; and the addressing mode is one in which that four-byte address can be the whole branch address.

The rule is deliberately small. It covers the shape a programmer writes when calling an external routine directly, and it uses only facts the text fixes. It does not cover a `CALL` macro (unknown operation), a load from a labelled constant, a load from a parameter list, an address computed at run time, or a load separated from the branch by other instructions.

FIG-23-2 — The five facts of a proven external call · AUTHOR-CONCEPTUAL



Legend. Each box is one fact ETC checks; a vertical bar means "and". The outcome holds only when all five hold.

Footnote. A load from a labelled V-type constant (`L 15,VLAB` with `VLAB DC V(name)`) fails fact 1 under the current rule. That is a CURRENT ETC LIMITATION, not a statement that no call happens.

Caption. ETC proves an external call from five facts, each fixed by the text of two adjacent statements and the module's addressing mode. Figure provenance: AUTHOR-CONCEPTUAL (the five facts are cited, sourced claims). The figure does not imply that a register-zero branch-and-link is a call; that form is not shown because it does not branch.

The five facts are recorded in ETC as five cited claims, each with a public source. A test checks that every claim keeps a vendor, document, topic and retrieval date. The test checks that the claims stay sourced. It does not check that the rule is right. Two kinds of evidence sit behind the rule, and they should not be merged. IBM documentation establishes the platform semantics the rule uses: what branch-and-link instructions do, what V-type and A-type constants are, and what

the binder does with an external reference. The specific five-part proof policy, including the adjacency requirement and the accepted pattern, is ETC's deterministic implementation rule. It is established by the review of the code and tests, not by vendor documentation.

Logical, not physical

CURRENT ETC INTERPRETATION. An ETC external call is **logical**: it says the module transfers control towards the external symbol `name`. It is not **physical**: ETC does not model the runtime branch address, because link-time processing may make the transfer indirect. The call's evidence note says so.

This is not modesty. It is the honest scope of a static proof. In the case study, what a reviewer needs is the logical relation: the designed call in `QMAIDTCV` is expected to name `QMALOG`. That is an expected deterministic result (layer B), and layer C is UNKNOWN. The reviewer does not need, and ETC cannot supply, the address the processor will hold at that instant.

When a fact is missing

When the branch is a `BALR` or `BASR` through a register but the pattern fails, layer 5 records why:

Refusal	What failed
<code>NO_ADJACENT_DEFINITION</code>	The previous machine instruction does not load the branch register
<code>DIFFERENT_REGISTER</code>	The adjacent load writes a different register
<code>ADDRESSING_MODE_TOO_WIDE</code>	The addressing mode is 64-bit
<code>LOCAL_ADDRESS_CONSTANT</code>	The loaded address is a local A-type address, not another executable section

These first three leave the call unresolved. Layer 5 records a register-form branch-and-link instruction whose target the rule does not establish. A local address constant branched through is recorded as not a call at all, because the destination is not another executable section and not an external symbol. An unresolved call is never promoted to a named, proven call, although current handling can carry it onward as an unresolved call site (see "What reaches the EvidenceGraph" below).

The labelled V-type constant

A programmer may keep V-type constants in a table of named words and load them by label:

```
[AUTHOR EXAMPLE]
    L      15,VQMALOG2
    BALR   14,15
    ...
VQMALOG2 DC    V(QMALOG)
```

The source expresses external linkage as clearly as the literal form does: the constant is a V-type reference to `QMALOG`, and the branch goes through the register it was loaded into.

CURRENT ETC INTERPRETATION. The current rule does not follow a load from a labelled V-type constant. ETC does not record this sequence as a call to `QMALOG`. CURRENT ETC LIMITATION.

The limitation is ETC's, not the platform's. A reviewer reading an ETC report on a module that uses this style should expect external calls to be missing, and should not read their absence as evidence that the module calls nothing.

Intra-assembly section calls

One assembly can contain several executable control sections, and code in one can call another. The usual way is to load the other section's address from an A-type literal and branch through it.

CURRENT ETC INTERPRETATION. ETC's layer 5 proves an **intra-assembly section call** for this shape:

```
text L r,=A(SECTB) SECTB is another executable control section BALR x,r (or BASR x,r), the
next machine instruction
```

in the same assembly.

A section call is logical, intra-assembly linkage. ETC proves it, and then deliberately keeps it out of application `CALLS` and out of the EvidenceGraph. Its one job downstream is to give layer 6 a **link register**: the register named in the caller's first operand (`x` above), which layer 6 may use to prove that a `BR` in `SECTB` returns (Chapter 24).

The A-type constant is not the section call. It is one of the facts. The proof is the adjacency of the load and the branch-and-link, together with what the address names. An `ADTAB DC A(QMADTLP)` that is never loaded and branched through gives no section call.

Nor is a `BAS 14,LABEL` to a label a section call, even if the label begins a routine. Chapter 22 showed it is a static branch in the raw CFG; layer 5 reads only register forms.

Why section calls stay out of `CALLS`

A section call connects two parts of one assembly, which become one module and one `program` node. Turning it into a `calls` edge would make a module call itself, or would invent nodes for sections no other program can name. Chapter 10's graph is about relationships between programs, jobs and stores. Section calls are about the inside of one program.

FIG-23-3 — BALR/BASR vs BR · SOURCE-DERIVED

PLATFORM / IBM SEMANTICS – architecture ([B7]); PLATFORM CONVENTION in brackets

```
BALR r1,r2    (r2 not 0)    r1 <- link information; branch to address in r2
BASR r1,r2    (r2 not 0)    r1 <- next-instruction address; branch to address in r2
BALR r1,0 / BASR r1,0      r1 <- link / next-instruction address; NO branch
BR   r        (= BCR 15,r)  branch to address in r; nothing stored
BR   0                          NO branch
```

[convention: a caller uses BALR/BASR 14,15 with R15 = entry address;
the callee returns with BR 14]

CURRENT ETC INTERPRETATION

```
BALR/BASR r1,r2 (r2 not 0)  layer 4 unresolved; layer 5 may prove a call
                           (patterns above)
BALR/BASR r1,0              layer 4: fall-through only; handling at later
                           layers is a current implementation limitation
BR r                        layer 4 unresolved; layer 6 may prove a return
                           (Chapter 24)
```

Legend. <- means "receives". "No branch" means execution continues with the next instruction. Bracketed text is a software convention, not an architectural rule.

Caption. Branch-and-link stores a link and then branches, unless the second operand is register 0. BR only branches. The architecture does not say which of these is a call or a return; ETC uses its own patterns for that, not these semantics. Figure provenance: SOURCE-DERIVED ([B7]) for the upper panel. The figure does not imply that any BR 14 is a return, that a BAS or BAL to a label is a call between programs, or that ETC uses these semantics as proof.

What reaches the EvidenceGraph

CURRENT ETC INTERPRETATION.

Assembler-derived fact	In the EvidenceGraph?
The assembler unit itself	Yes, as a <code>program</code> node
A proven external call from the unit	Yes, as an ordinary <code>calls</code> edge to the called program, or to an estate boundary when the target is outside the bundle
A COBOL <code>CALL</code> to the unit, or a JCL <code>EXEC PGM=</code> naming it	Yes, as for any program
A register-form branch-and-link that layer 5 refuses for one of the first three reasons above (no adjacent load, a different register, 64-bit addressing)	Under current handling, some such refusals can appear as unresolved external or estate-boundary call evidence; they never appear as a named, proven call. A refusal classed as not a call at all (a local address constant branched through, or a load from a labelled V-type constant) produces no call evidence
Raw CFG edges (Chapter 22)	No; not projected merely because they are edges
Intra-assembly section calls	No; they do not become application-level <code>calls</code>
Returns (Chapter 24)	No
Location facts, <code>COPY</code> names	No

The internals of an assembler module never reach the graph. Its proven external calls can, as ordinary `calls` edges with the same shape as a COBOL static call, and current handling can also carry some unresolved call sites onward as external or estate-boundary call evidence. Chapter 10's statement that an assembler module is a `program` node with no internal edges remains true: an external call is not an internal edge. This chapter adds that outbound `calls` relationships from an assembler node are possible.

On the three estates, the incidence is zero: neither CardDemo assembler module contains an external call that matches the pattern. Both CardDemo assembler modules are themselves called, by COBOL programs, and those `calls` edges point into them.

The current implementation has a provenance-labelling limitation for some assembler-derived call evidence: part of the support record on such an edge does not describe its assembler origin accurately. For a proven call, the edge and its target are correct under the rule; the label on its support is the issue. A reviewer should read support records for assembler-derived edges with that in mind.

ENTRY and EXTRN again

Chapter 21 classed `ENTRY` and `EXTRN` as declarations. Here is why that matters for linkage. `EXTRN QMALOG` tells the binder the module refers to `QMALOG`; a V-type constant does the same. Neither is fact 3 or fact 4 of the external-call proof. A module could declare a dozen externals and call none of them. ETC records the declarations and proves calls only from the load and branch pattern.

Quillon's QMADTCV : the linkage

AUTHOR-DESIGNED SCENARIO – not an ETC result. (Layer C: no case-study run is recorded in this edition)

[CASE-STUDY ARTIFACT] The rows below are layer A, the content of the fictional module, and layer B, what ETC's rules, as reviewed, would give for that content. The rule application was checked on an author snippet of the same shape. That snippet is a design aid; it is not QMADTCV and not a Quillon run.

Element	Layer A — content	Layer B — EXPECTED DETERMINISTIC RESULT
External call	L 15,=V(QMALOG) then BALR 14,15	Proven external call to QMALOG : five facts present; logical, not physical
Labelled V-type constant	L 15,VQMALOG2 then BALR 14,15, with VQMALOG2 DC V(QMALOG)	Not followed. No call recorded; the branch stays unresolved at later layers. CURRENT ETC LIMITATION, never platform truth
Address table	ADTAB DC A(QMADTLP)	No call, no section call
Section call 1	L 15,=A(QMADTLP) then BASR 14,15	Intra-assembly section call, QMADTCV → QMADTLP
Section call 2	L 15,=A(QMADTRT) then BASR 14,15	Intra-assembly section call, QMADTCV → QMADTRT
Static branch	BAS 14,QMADTFX to a label in the main section	Static direct branch (Chapter 22); no call of any kind

Expected EvidenceGraph contribution (layer B): `program:QMARTE01 -calls→ program:QMADTCV`, from the COBOL `CALL 'QMADTCV'`; and `program:QMADTCV -calls→ estate_boundary:QMALOG`, because QMALOG is outside the bundle. The second edge's support is subject to the provenance-labelling gap above. No edge comes from the labelled V-type constant, the section calls, the static branch, the returns or the location tracker. NOT AN ETC RUN. LAYER C: UNKNOWN.

Notice what the design teaches. The same module calls QMALOG twice in the source, in two styles. Under the current rule, ETC is expected to prove one and not the other. A reader comparing the source with an ETC report would see one `calls` edge and might conclude the second call does not exist. It does, in the fiction; the rule does not follow it.

Pseudocode — proving an external call

Pseudocode written for this book. It is not an excerpt of ETC source.

```

[PSEUDOCODE] layer 5 for one control section
find_linkage(section):
  for each BALR / BASR whose branch register r is not 0:
    prev = the machine instruction immediately before it
    if prev is not L:                                unresolved (NO_ADJACENT_DEFINITION)
    elif prev writes a register other than r:          unresolved (DIFFERENT_REGISTER)
    elif AMODE is 64-bit:                               unresolved (ADDRESSING_MODE_TOO_WIDE)
    elif prev operand is =V(name) or V(name):          EXTERNAL_CALL to name
                                                         (logical; physical address not
modelled)
    elif prev operand is =A(S) and S is another
      executable section of this assembly:              INTRA_ASSEMBLY_SECTION_CALL to S
                                                         (link register = first operand)
    elif prev operand is a local A-type address:        not a call (LOCAL_ADDRESS_CONSTANT)
    else:                                                not followed (current limitation)
project:
  PROVEN_EXTERNAL_CALL      -> static call site -> ordinary `calls` edge
  UNRESOLVED_CALL_SITE      -> unresolved call site -> external / estate-boundary
    (the first three        unresolved call evidence, under current
    refusals)               handling; never a proven call
  SECTION_CALL              -> kept for layer 6; not an application-level `calls`
    not a call / not followed -> no call site
  RAW_CFG_EDGE              -> not an EvidenceGraph edge merely because it is an edge

```

Left out: the claim citations, the exact operand matching and the bookkeeping for warnings. The register-zero form is not part of this trace's input.

Code Trace Challenge — which are calls?

Use this author-created fragment: one assembly, two executable control sections `MAINS` and `HELPS`, addressing mode 31, no macros.

```

[AUTHOR EXAMPLE]
MAINS      CSECT
           L      15,=V(PRINTR)
           BALR   14,15
           L      15,=A(HELPS)
           LA     1,PARMS
           BASR   14,15
           L      15,=A(HELPS)
           BASR   14,15
           L      15,VTAB
           BALR   14,15
           EXTRN  AUDITR
VTAB       DC     V(LOGGER)
PARMS      DC     F'0'
HELPS      CSECT
           BR     14

```

a. Which sequences does ETC prove as external calls, and which as section calls? b. Why is the first `BASR 14,15` not a section call? c. Does the module call `AUDITR` or `LOGGER`?

Answer (by the rules, on an author-created fragment; not an ETC run). (a) External call: `L 15,=V(PRINTR) + BALR 14,15`. Section call: the second `L 15,=A(HELPS) + BASR 14,15`. (b) The machine instruction immediately before it is `LA 1,PARMS`, not the load, so adjacency fails (`NO_ADJACENT_DEFINITION`); the call is unresolved. (c) `AUDITR`: the module declares it, and nothing branches to it; there is no evidence of a call. `LOGGER`: the source does branch through a register loaded from a V-type constant for `LOGGER`, so it expresses a call, but the current rule does not follow a labelled constant. ETC records no call to `LOGGER`; that is a limitation, not evidence of absence.

Level 3 — Architecture reasoning

Narrow rules, honest gaps

ETC could widen its rule: follow labelled constants, look further back than one instruction, read parameter lists, recognise `CALL` macros. Each widening would find more calls and would also make more assumptions about what happens between the load and the branch. The current rule proves fewer calls, each on facts the text fixes. The cost is visible: some calls are missed, and the book names the main gap, the labelled V-type constant.

This book's methodology prefers that trade. A missed call is a known category of gap that a reviewer can look for. A false call, presented with the same confidence as a true one, is not.

Failure Case — the V-con inventory

A team builds an "assembler dependency map" by listing every V-type constant and `EXTRN` in each module and drawing an edge for each. One module has fourteen. Two are called; the others are left over from a retired release, used only on an error path that cannot be reached, or referenced so that the binder includes a routine another module calls. The team plans interface work for fourteen dependencies and misses one real call that goes through a parameter list. A reference was treated as an invocation, and the absence of a declaration was treated as the absence of a call.

Architect's Warning

Keep four things apart in every assembler report: a declaration (`EXTRN`, `ENTRY`, `DC V`, `DC A`), a recognised call pattern, a section call inside one assembly, and an application-level `calls` relationship. Only the second can become the fourth as a proven relationship, and only when ETC proves it. A section call never does.

Claim Check

"ETC maps QMADTCV's dependencies: it calls QMALOG twice, and the two section calls to QMADTLP and QMADTRT appear in the graph."

This fails four ways. ETC has not read QMADTCV; every outcome is layer B. Under the current rule only one of the two QMALOG calls is expected to be proven. Section calls never appear in the graph. And a proven external call is logical, so "maps" must not suggest a physical address.

Current Support

Capability	Status	Evidence
External call from adjacent <code>L r,=V(name) + BALR / BASR r,</code> AMODE below 64	IMPLEMENTED	linkage tests; direct probes
Intra-assembly section call from adjacent <code>L r,=A(S) + BALR / BASR r</code>	IMPLEMENTED	linkage tests
Refusal reasons for failed patterns	IMPLEMENTED	Reviewed at the baseline
Proven external calls as ordinary <code>calls</code> edges	IMPLEMENTED	linkage projection tests; graph probe
Section calls, returns, CFG and locations kept out of the graph	IMPLEMENTED	projection tests (passing)

Current Limitation

CURRENT IMPLEMENTATION LIMITATION.

1. **Labelled V-type constants are not followed.** The source may still express a call.
2. **Only one load-and-branch shape per kind is recognised.** `CALL` macros, parameter-list loads, computed addresses and non-adjacent loads are not followed.
3. **Calls are logical, not physical.** No runtime branch address is claimed.
4. **A provenance-labelling limitation** affects the support record of some assembler-derived call evidence.
5. **The register-zero branch-and-link form** is a current implementation limitation outside layer 4.
6. **A list of known external routines** is not used; a claim that depends on one is out of scope here.
7. **Quillon is a case study.** QMADTCV's linkage and graph contribution are layer B.

Quick Recall

1. Why is "A-type is local, V-type is external" too simple?
2. Name the five facts ETC needs to prove an external call.
3. What does "logical, not physical" rule out?
4. `L 15, =A(SECTB)` followed by `BASR 14,15` into another section of the same assembly: what does ETC prove, and where does that result go?
5. Why must a reviewer not read a missing `calls` edge from an assembler module as proof that the module makes no call?

Key Takeaways

- Address constants are data. A V-type constant is an external reference for the binder; an A-type constant holds an address. Neither is a call.
- ETC proves an external call only from an adjacent load of a literal V-type operand and a `BALR / BASR` through the same register, below 64-bit addressing: five facts, logical, not physical.
- A load from a labelled V-type constant is not followed. That is a current ETC limitation, not platform truth.
- An intra-assembly section call is proven from an adjacent `=A()` load into another executable section; it supplies a link register and stays out of `CALLS` and the graph.
- Proven external calls can become ordinary `calls` edges, and current handling can also carry some unresolved call sites onward as external or estate-boundary call evidence. Raw CFG edges, section calls, returns and locations are not projected as graph relationships.
- `QMACTCV` is expected to show one proven call to `QMALOG`, one unfollowed, and two section calls. Layer C is unknown.

Chapter 24 — Proving Returns and Refusing the Rest

SPECIAL REVIEW SECTION — Early Review Edition (see Chapter 21).

After this chapter you can:

- keep three statements about a return apart: what the architecture does, what the linkage convention says, and what ETC proves;
- explain the standard roles of R1, R13, R14 and R15 as a software convention, not a processor rule;

- state the chain ETC needs before it calls a `BR` a return, and why `LM` followed by `BR 14` is not enough;
- explain why both CardDemo assembler modules end in the textbook exit and are refused, and why that refusal is correct;
- describe the four validation-corpus returns exactly, without stretching them, and say why ETC stops at analysis.

Every assembler programmer knows how a routine returns: restore the caller's registers and `BR 14`. This chapter is about the distance between knowing that convention and proving, from source, that a particular `BR 14` is a return. ETC proves very few. The reasons are the most useful lesson in Part V.

Level 1 — Intuition

A key card in the right shape

A hotel key card has a standard shape, a magnetic strip and the hotel's logo. A card with all three looks like a key to some room. But a guard at the door does not let you in because the card looks right. The reader checks that the card was issued for this door, by this hotel, and has not been reissued since.

`LM 14,12,12(13)` followed by `BR 14` is a card in the right shape. It is the conventional exit, so a reader who knows the convention expects a return. ETC's layer 6 is the guard: it wants evidence of who issued the address in register 14, and that nothing changed it on the way.

Three truths about a return

Label	Statement about <code>BR 14</code>
PLATFORM / IBM SEMANTICS (architecture)	It branches to the address in register 14. The processor has no notion of caller or return
PLATFORM / IBM SEMANTICS (linkage convention)	By convention, register 14 holds the caller's return address, so in a program that follows the convention <code>BR 14</code> normally returns
CURRENT ETC INTERPRETATION	It is a <code>RETURN</code> only when layer 6 proves that register 14 holds the link value of a proven call into this section, either untouched or restored by a checked <code>LM</code> . Otherwise it is refused, with a reason

The book's method sits underneath all three. A recognisable pattern is evidence that a convention may have been followed. A proof needs the provenance of the value in the register. That second requirement is the book's and ETC's methodology. It is not something IBM's documentation states; IBM documents the convention, not how a static analyser should prove it.

Level 2 — Engineering mechanics

The standard linkage convention

PLATFORM / IBM SEMANTICS — STANDARD z/OS LINKAGE CONVENTION ([B6]). Under the standard linkage convention used between cooperating programs:

- **R1** holds the address of a parameter list;
- **R13** holds the address of a register save area provided by the caller;
- **R14** holds the address the called program should return to;
- **R15** holds the called program's entry address on entry, and a return code on return.

The called program typically stores the caller's registers in that save area with `STM 14,12,12(13)` (registers 14 through 12, starting at offset 12) and restores them with `LM 14,12,12(13)` before returning with `BR 14`. When it calls further routines, it chains its own save area to the caller's (offset 4 holds the pointer back).

This is a convention: an agreement among programs, compilers and system macros so that separately written code can call each other. The processor does not enforce it. Nothing stops a program using R14 as a loop counter, R15 as scratch, or R13 for something other than a save area, and private linkage conventions exist. The convention explains why `LM` and `BR 14` appear together so often. It does not, by itself, tell an analyser what a given register holds.

CURRENT ETC INTERPRETATION. ETC infers no register role from the convention. Register 14 is not assumed to hold a return address. Register 13 matters only as the base of a save slot that the layer-6 checks follow. Register 15 matters only as the destination of a layer-5 load. Nothing in ETC treats `SR 15,15` as setting a return code.

Where a link register comes from

ETC proves a return from one source of truth only: a proven call into the section that contains the `BR`.

CURRENT ETC INTERPRETATION. A **link register** for a section is the register named as the first operand of a proven intra-assembly section call into that section (Chapter 23). If no proven section call enters the section, there is no link register, and every `BR` in it is refused with `NO_PROVEN_CALL_CONTEXT`.

This has a sharp consequence. A module entered from outside its own assembly (by a COBOL `CALL`, by JCL, by another load module) has no proven call into its entry section from inside the assembly. Its main exit can therefore never be proven a return under the current rule, whatever it

looks like. The book records this as a design limitation of the current rule, not a defect: the rule does what it says.

The chain ETC needs

For a `BR R` in a section with link register `L`:

1. **Same register.** R is the link register `L`.
2. **Reaching definition.** The value R holds at the `BR` is traced back to the statement that last set it.
3. **That definition is the link.** Either R still holds the value placed by the proven call's branch-and-link, untouched, or R was restored by an `LM` from a save slot that held that link value.
4. **The restore passes the save/restore checks.** For an `LM` restore, ETC checks that a save slot is established; that a save of the link reaches the restore; that the base register addressing the slot is stable; that the link was not overwritten before the save; that the save dominates the restore (every path to the restore passes through the save); that the word is wide enough for the address; that the saved word is not overwritten in between; that each memory effect in between is known; and that there is no unresolved transfer in the section that could break any of this.

If every link holds, the outcome is `RETURN`. It says that control goes back along the link. It does not say where back is, and it is projected nowhere: no return reaches the EvidenceGraph.

When a link fails, the outcome is unresolved, with the reason. The reasons seen in this chapter are:

Refusal	What failed
<code>NO_PROVEN_CALL_CONTEXT</code>	No proven call enters the section, so there is no link register
<code>REACHING_DEFINITION_IS_NOT_A_LINK_INSTRUCTION</code>	The value in the register comes from somewhere other than the link, for example a plain load from storage
<code>SAVED_WORD_MAY_HAVE_BEEN_OVERWRITTEN</code>	Between save and restore, something may have stored over the saved link
<code>NOT_IN_SCOPE_FOR_THIS_PHASE</code>	The register branch is not one layer 6 examines (it examines <code>BR</code> and the branch-and-link spellings)

BOOK / EVIDENCE METHODOLOGY. A refusal is a correct outcome when the chain is incomplete. It is not a claim that the routine fails to return at run time. It says the evidence in the text does not establish where the branch goes.

FIG-24-1 — Convention vs proof · SOURCE-DERIVED + AUTHOR-CONCEPTUAL

PANEL 1 – PLATFORM CONVENTION ([B6])	PANEL 2 – ETC PROOF CHAIN (layer 6)
caller: R15 <- entry address R14 <- return address (BALR/BASR 14,15) R13 -> caller's save area callee: STM 14,12,12(13) save caller's registers operand ... LM 14,12,12(13) restore them the BR 14 branch to return address convention alone never proves a return. reason LM + BR 14 alone is not proof. ETC infers nothing from R13 / R14 / R15.	proven intra-assembly section call into this section (Chapter 23) link register = caller's first optional STM ... LM that passes save/restore checks BR of that same register RETURN (control goes back; not where; projected nowhere) any link missing -> refused, with

Legend. In panel 1, <- means "is loaded with" and -> means "points to", as the convention describes. In panel 2, a vertical bar means "and then requires".

Caption. The convention explains why LM and BR 14 mark the end of a routine. ETC's proof requires a proven call into the section and a link register traced to the BR. Figure provenance: SOURCE-DERIVED ([B6]) for panel 1; AUTHOR-CONCEPTUAL for panel 2. The figure does not imply that the convention is wrong, that ETC recognises the convention, or that a refused BR does not return at run time.

What the validation corpus shows

On the validation corpus (VALIDATION CORPUS – NOT AN ESTATE), at the book baseline, layer 4 leaves 96 transfers unresolved because their destination is a register (Chapter 22). Layers 5 and 6 resolve them as:

Outcome	Count
Proven external calls	11
Proven intra-assembly section calls	46
Proven returns	4
Unresolved: BR, NO_PROVEN_CALL_CONTEXT	25
Unresolved: BR, SAVED_WORD_MAY_HAVE_BEEN_OVERWRITTEN	10

So 61 of the 96 are proven as calls or returns, and 35 remain unresolved.

The four proven returns are in four (member, section) pairs from the zblas repository's z390 test drivers: `Z390_DASUM / DASUM`, `Z390_DDOT / DDOT`, `Z390_DNRM2 / DNRM2` and `Z390_IDAMAX / IDAMAX`. In each, a `BR` is reached by an `LM` that passes the save/restore checks, in a section entered by a proven section call.

Read those results exactly:

- they are **four test-driver sections**, not the library's computational kernels and not four programs proven in general;
- they are **validation-corpus results**, measured on public test repositories, not estate results and not production evidence;
- they show that the chain can be completed on real assembler, not how often it is completed, and not that ETC recognises returns generally;
- they say nothing about behaviour on z/OS.

The 43 HLASM validation tests behind these figures passed against the pinned validation corpus, using the corrected validation-corpus root recorded in the book's validation evidence. Tests are not programs; 43 passing tests is not "43 programs validated". The corpus members match their pinned SHA-256 digests, 51 of 51, by independent check. A digest match shows that the files are the pinned files. It is not behavioural validation, and it is not a pass from the project's own corpus verifier (which does not pass on the local layout).

The census also asserts that there are no proven register branches. That assertion compares a constant with zero and cannot fail, because no outcome named "register branch" exists. It is not a finding, and this chapter does not present it as one.

The CardDemo exits: the textbook shape, refused

The two CardDemo assembler program units end in the conventional exit. `COBDATFT` ends:

```
[ABSTRACT SEQUENCE – summarised from the CardDemo COBDATFT exit;
not a verbatim excerpt]
  reload R13 from the module's own save area
  LM  R14 through R12 from offset 12 of that area
  SR  R15,R15          (return code zero)
  BR  R14
```

`MVSWAIT` ends with `LM 14,12,12(13)`, `MVI 12(13),X'FF'`, `SR 15,15` and `BR 14`, with a comment saying it returns to COBOL.

Both sequences resemble the conventional return idiom, and anyone who knows the convention reads them that way. This analysis makes no claim about how either module behaves when run, and it cites no runtime observation. Without the required proof context there is no proved return. ETC's layer 6 refuses both `BR` instructions with `NO_PROVEN_CALL_CONTEXT`. Each module is

entered by a COBOL `CALL` (`CBACT01C` calls `COBDATFT`; `COBSWAIT` calls `MVSWAIT`), not by a proven section call inside its own assembly, so no link register exists for the section and the chain cannot begin.

This was observed by reading the product's functions directly on the estate members at the book baseline. No regression test pins it. No estate return is proven.

The refusal is the rule working as written. It is not a false negative to be explained away, and it is not a claim that either module fails to return. What it tells a reviewer is precise: the return behaviour of these modules rests on the convention, and nothing in the text proves that the convention holds here.

Quillon's `QMADTCV`: one proof path, three refusal paths

AUTHOR-DESIGNED SCENARIO – not an ETC result. (Layer C: no case-study run is recorded in this edition)

[CASE-STUDY ARTIFACT] The module was designed so that each exit exercises one rule. Layer A is the fiction's content; layer B is what the rules as reviewed would give.

Exit	Layer A — content	Layer B — EXPECTED DETERMINISTIC RESULT
<code>QMADTLP</code> exit (proof path)	Section entered only by <code>L 15,=A(QMADTLP) + BASR 14,15</code> ; body <code>STM 14,12,12(13) ... LM 14,12,12(13); BR 14</code> ; no macro, <code>COPY</code> , unknown operation or store into the save area between <code>STM</code> and <code>LM</code>	<code>RETURN</code> : <code>R14</code> is the link register of a proven section call into <code>QMADTLP</code> , restored by a checked <code>LM</code>
<code>QMADTRT</code> exit (refusal path)	Section entered by <code>L 15,=A(QMADTRT) + BASR 14,15</code> ; <code>ST 14,SAVE14 ... L 14,SAVE14; BR 14</code>	Unresolved, <code>REACHING_DEFINITION_IS_NOT_A_LINK_INSTRUCTION</code> : the value comes back from storage by a plain load, which the rule does not accept as the link
Main exit (refusal path)	<code>L 13,4(,13); LM 14,12,12(13); SR 15,15; BR 14</code> — the shape <code>COBDATFT</code> uses	Unresolved, <code>NO_PROVEN_CALL_CONTEXT</code> : <code>QMADTCV</code> is entered from <code>QMARTE01</code> , outside the assembly
Local exit <code>QMADTFX</code>	<code>BR 14</code> , reached by <code>BAS 14,QMADTFX</code>	Unresolved, <code>NO_PROVEN_CALL_CONTEXT</code> : a static <code>BAS</code> is not a proven section call

NOT AN ETC RUN. LAYER C: UNKNOWN. Whether any of these exits returns when run is not established by anything in this book.

FIG-24-2 — A proof-path pattern and a refusal-path pattern · AUTHOR-CONCEPTUAL (case study), with separate measured panels

CASE-STUDY PANEL — Expected under the rules as reviewed — not an ETC result (layer C UNKNOWN)		
QMA DT CV main section	QMA DT LP	QMA DT RT
L 15,=A(QMA DT LP) --proven-->	STM 14,12,12(13)	ST 14,SAVE14
BASR 14,15 section call	...	...
L 15,=A(QMA DT RT) --proven-----	LM 14,12,12(13) ---->	L 14,SAVE14
BASR 14,15 section call	BR 14	BR 14
...	=> RETURN	=> refused:
L 13,4(,13)	(proof path)	REACHING_DEFINITION_
LM 14,12,12(13)		
IS NOT A LINK INSTRUCTION		
BR 14 => refused: NO_PROVEN_CALL_CONTEXT (entered from outside the assembly)		
MEASURED PANEL A — validation corpus	MEASURED PANEL B — CardDemo estate	
4 proven returns, each a BR reached by LM,	COBDATFT	BR R14 refused
NO_PROVEN_CALL_CONTEXT		
in 4 zblas z390 test-driver sections:	MVSWAIT	BR 14 refused
NO_PROVEN_CALL_CONTEXT		
DASUM, DDOT, DNRM2, IDAMAX		(direct probe; no pinned test)
(not programs; not estate; not production)		

Legend. --proven--> marks an expected proven intra-assembly section call into the section it points at. => gives the expected (case-study panel) or measured (panels A and B) layer-6 outcome for the BR above it. In the QMA DT RT column, ----> only aligns the two exits for comparison.

Caption. The same final instruction, BR 14, is expected to be proven in one place and refused in two others, because what matters is the provenance of register 14, not its number. The measured panels are kept apart from the case study and are never summed with it. Figure provenance: AUTHOR-CONCEPTUAL for the case-study panel; CORPUS-DERIVED measurements for the measured panels. The figure does not imply that ETC proved or refused anything in QMA DT CV, that estate returns were proven, that four programs were proven, general or production return recognition, or a measured "zero register branches".

Pseudocode — proving and refusing returns

Pseudocode written for this book. It is not an excerpt of ETC source.

```

[PSEUDOCODE] layer 6 for one control section
classify_transfers(section, linkage):
    entry_links = { first operand of each proven section call INTO this section }
    for each BR (and branch-and-link spelling) with register R not 0:
        if R is a branch-and-link proven at layer 5:    proven call (Chapter 23)
        elif entry_links is empty:                    refuse NO_PROVEN_CALL_CONTEXT
        elif R not in entry_links:                    refuse (not the link register)
        else:
            d = reaching definition of R at this BR
            if d is the section's entry link value:    RETURN
            elif d is an LM restoring R and
                save/restore checks all pass:        RETURN
            elif d is an LM and a saved word
                may have been overwritten:            refuse
            SAVED_WORD_MAY_HAVE_BEEN_OVERWRITTEN
            else:                                     refuse
            REACHING_DEFINITION_IS_NOT_A_LINK_INSTRUCTION
            other register branches:                  NOT_IN_SCOPE_FOR_THIS_PHASE
            nothing from this layer is projected into the EvidenceGraph

```

Left out: the individual save/restore checks and their refusal names, the dominance computation and the reporting. The census is the evidence for the figures above.

Applied to `COBDATFT` and `MVSWAIT`, the trace stops at the second test: no proven section call enters either module's section, so `entry_links` is empty and each `BR` is refused with `NO_PROVEN_CALL_CONTEXT`. The save/restore checks are never reached, although both modules contain exactly the save and restore the convention describes.

Code Trace Challenge — which exits are proven?

Use this author-created fragment: one assembly, sections `TOPS` (entered from outside the assembly) and `SUBS`, addressing mode 31, no macros.

```

[AUTHOR EXAMPLE]
TOPS      CSECT
          STM 14,12,12(13)
          L   15,=A(SUBS)
          BASR 14,15
          LM  14,12,12(13)
          BR   14
SUBS      CSECT
          STM 14,12,12(13)
          LA  3,1
          LM  14,12,12(13)
          BR   14

```

a. Which `BR 14` does the rule prove a return, and why? b. Why is the other one refused, given that it has the same shape? c. If `SUBS` stored a value at `12(13)` between its `STM` and `LM`, what would change?

Answer (by the rules, on an author-created fragment; not an ETC run). (a) The `BR 14` in `SUBS`: a proven section call enters `SUBS` with link register 14, and the `LM` restores 14 from the slot the `STM` saved it to, with nothing in between that touches the slot. (b) `TOPS` is entered from outside the assembly, so no proven call gives it a link register: `NO_PROVEN_CALL_CONTEXT`. (c) The saved link word may have been overwritten, so the `LM` no longer restores a value the rule can trust. The `BR` in `SUBS` would be refused; the expected reason is `SAVED_WORD_MAY_HAVE_BEEN_OVERWRITTEN`.

Level 3 — Architecture reasoning

Why refusal matters

BOOK / EVIDENCE METHODOLOGY. This reasoning is the book's method. It is not an industry finding.

Suppose an assembler analysis is used to decide what to keep, wrap, rewrite or retire, or to scope testing. If the analysis says "this module returns normally to its caller" and the module in fact returns through a register a macro altered, the decision rests on a defect it cannot see. If the analysis says "return not proven: no proven call context", the decision has a work item: someone confirms the linkage, or the interface test covers it.

The book therefore prefers a bounded unknown to false certainty. The unknown is visible and can be assigned. The false certainty is invisible to the reader of the report. ETC's refusals are how the unknowns become visible.

Why ETC stops at analysis

CURRENT ETC INTERPRETATION. ETC v0.1 does not translate assembler. Current ETC has no HLASM-to-Java transformation path.

The measured reasons are in this section:

1. **No macro expansion.** On the validation corpus, 228 branch targets are refused because they need a macro expanded. Macros carry linkage, I/O and control flow that a translation would need.
2. **Most register-indirect transfers are not proven.** Of 96 register targets on the validation corpus, 35 remain unresolved, and only four returns are proven. On the estates, no return is proven at all.

3. **No target exists.** There is no assembler target adapter in the product.

None of this says that proving more would make conversion safe. A complete CFG, every call and every return proven would still leave data access, I/O, system services and timing unexamined. Proof of control structure is not proof of behaviour, and it is not a translation.

Failure Case — the return assumed

AUTHOR-DESIGNED SCENARIO – illustrative; not an estate incident and not an ETC result.

A team converts a COBOL caller and replaces its `CALL` to an assembler date routine with a Java stub that "returns the converted date", on the understanding that the routine is a simple call and return. The analysis they used had marked the routine's `LM` and `BR 14` as a return because it matched the convention. In the scenario, on one error path the routine branches through R14 after a macro that reloads it from a system control block, and control goes elsewhere. ETC would have refused the exit; the team's tool presented the idiom as a fact. The stub hides a behaviour nobody knew existed.

Architect's Warning

The convention is real, widely followed and the reason the textbook exit is recognisable. It is still a convention. In front of a decision-maker, say "follows the conventional exit pattern; return not proven" when that is what the evidence supports. Do not say "returns normally" on the strength of a register number.

Claim Check

"ETC proves that CardDemo's assembler modules return correctly to their COBOL callers, and on the validation corpus ETC proved four HLASM programs."

This fails twice. ETC refuses both CardDemo returns with `NO_PROVEN_CALL_CONTEXT`, and says nothing either way about how they behave when run. The four validation-corpus results are four bounded test-driver sections, not four programs, and not general return recognition.

Current Support

Capability	Status	Evidence
Link registers from proven intra-assembly section calls	IMPLEMENTED	transfer tests
<code>RETURN</code> for an untouched link or an <code>LM</code> restore that passes the checks	IMPLEMENTED	census (4 returns, validation corpus)
Refusal reasons for incomplete chains	IMPLEMENTED	census
CardDemo exits refused with <code>NO_PROVEN_CALL_CONTEXT</code>	OBSERVED (probe)	no pinned test
No return projected into the EvidenceGraph	IMPLEMENTED	projection tests (passing)

Current Limitation

CURRENT IMPLEMENTATION LIMITATION.

1. **A module entered from outside its assembly can never have its own return proven** under the current rule. Both CardDemo modules are in this position.
2. **Only one source of link registers:** proven section calls. A static `BAS` to a label, an external caller or a macro gives none.
3. **A return says that control goes back, not where back is**, and it is not projected.
4. **The validation-corpus results are bounded:** four test-driver sections.
5. **The census's "no register branches" is a constant**, not a finding.
6. **The CardDemo refusals are probe evidence**, with no pinned regression test.
7. **No HLASM translation.**
8. **Quillon is a case study.** `QMADTCV`'s return outcomes are layer B.

Quick Recall

1. Give the architectural, conventional and ETC statements about `BR 14`.
2. Where does ETC get a link register, and what happens to every `BR` in a section that has none?
3. `COBDATFT` ends with `LM` and `BR R14`. Why does ETC refuse it, and what does that refusal not claim?
4. What exactly are the four validation-corpus returns, and which three descriptions of them are prohibited?
5. In `QMADTCV`, why is the `QMADTLP` exit expected to be proven and the `QMADTRT` exit expected to be refused?

Key Takeaways

- Architecture branches; convention says R14 holds the return address; ETC proves a return only from the provenance of the value in the register.
 - R1, R13, R14 and R15 roles are a standard software convention. ETC infers nothing from register numbers.
 - A link register comes only from a proven intra-assembly section call. With none, every `BR` in the section is refused.
 - CardDemo's `COBDATFT` and `MVSWAIT` end in the textbook exit and are refused `NO_PROVEN_CALL_CONTEXT` (probe; no pinned test). No estate return is proven.
 - Four returns are proven in four zblas test-driver sections on the validation corpus: bounded results, not programs, not production.
 - ETC stops at analysis: no macro expansion, most register transfers unproven, no target adapter.
-

Closing Part V — what the special review section has shown

The book's method for assembler comprehension is to prove enough behaviour to know what remains unknown. Across these four chapters, ETC reads the text under stated assumptions, names the control flow syntax can name, proves two narrow kinds of call and one narrow kind of return, and refuses everything else with a reason. The proven facts are few and dependable. The refusals are many and specific, and each one is a question for a person who knows the system. An analysis that reports both is smaller than the one a pattern-matcher would produce, and it is honest about what it does not know.

Forthcoming Chapters

These chapters are planned. This early review edition contains no text for them, and the titles may change.

Part III — Evidence Graph and Dependency Reasoning (continued)

13. Execution Slices: Grouping by Technical Coupling
14. Technical-Boundary Confidence
15. What an EvidenceGraph Does and Does Not Say
16. Slices Are Not Applications

Part IV — COBOL Transformation and the Evidence of Correctness

- 17. Reading COBOL Logic and Refusing Well
- 18. Translation Eligibility: Whole Program or Nothing
- 19. Generating Java, the Runtime, and Compile Evidence
- 20. Equivalence Tiers

Part VI — LLM Interpretation and Agentic Architecture Boundaries

- 25. The Model Behind a Verifier
- 26. Interpretation Is Not Evidence

Part VII — Target Architecture

- 27. Four Statements About a Target
- 28. Batch to AWS Batch on Fargate
- 29. Data and Online Transformation
- 30. Target Architecture and Migration Planning

Part VIII — End-to-End Modernization Methodology

- 31. Working With Unknowns and Human Review
- 32. Production-Readiness Evidence and Cutover Thinking
- 33. Defending a Modernization Architecture

References

Public sources only. Each statement in the book about platform behaviour is the author's own explanation and cites one of these documents. Retrieval dates are 2026-10-04 and 2026-10-05. Document numbers are those printed on the documents. Vendor documents may be revised after the retrieval date.

Product and company names are used only for identification and remain the property of their respective owners.

IBM documentation

[B1] IBM. *z/OS MVS JCL Reference*, SA23-1385-70.

https://www.ibm.com/docs/en/SSLTBW_3.2.0/pdf/ieab600_v3r2.pdf

[B2] IBM. *z/OS MVS JCL User's Guide*, SA23-1386-70.

https://www.ibm.com/docs/en/SSLTBW_3.2.0/pdf/ieab500_v3r2.pdf

[B3] IBM. *Enterprise COBOL for z/OS Programming Guide*, SC27-8714-04.

https://www.ibm.com/docs/en/SS6SG3_6.5/pdf/pgmvs.pdf

[B4] IBM. *Enterprise COBOL for z/OS Language Reference*, SC27-8713-04.

https://www.ibm.com/docs/en/SS6SG3_6.5/pdf/lrmvs.pdf

[B5] IBM. *High Level Assembler for z/OS & z/VM & z/VSE Language Reference*, SC26-4940-09.

https://www.ibm.com/docs/en/SS6SG3_6.5/pdf/asmr1024.pdf

[B6] IBM. *High Level Assembler for z/OS & z/VM & z/VSE Programmer's Guide*, SC26-4941-07.

https://www.ibm.com/docs/en/SS6SG3_6.5/pdf/asmp1024.pdf

[B7] IBM. *z/Architecture Principles of Operation*, SA22-7832 (fourteenth edition, September 2022).

<https://publibfp.dhe.ibm.com/epubs/pdf/a227832c.pdf>

[B8] IBM. *z/OS DFSMS Using Data Sets*, SC23-6855-70.

https://www.ibm.com/docs/en/SSLTBW_3.2.0/pdf/dco0000_v3r2.pdf

[B9] IBM. *z/OS DFSMS Access Method Services Commands*, SC23-6846-70.

https://www.ibm.com/docs/en/SSLTBW_3.2.0/pdf/idai200_v3r2.pdf

Cloud and scheduler documentation

[B10] Amazon Web Services. *AWS Batch User Guide*, "Fargate job definitions".

<https://docs.aws.amazon.com/batch/latest/userguide/fargate-job-definitions.html>

[B11] BMC Software. Control-M documentation: job utility and in/out conditions (three pages).

https://documents.bmc.com/supportu/ctrlm9/help/Main_help/en-US/17227.htm

https://documents.bmc.com/supportu/ctrlm9/help/Main_help/en-US/JefJobInCond.htm

https://documents.bmc.com/supportu/ctrlm9/help/Main_help/en-US/Tools_Conditions.htm

[B12] Broadcom. CA Workload Automation CA 7 Edition documentation: `LJOB` command (12.0)

and schedule ID examples (12.1). <https://techdocs.broadcom.com/us/en/ca-mainframe-software/automation/ca-workload-automation-ca-7-edition/12-0/using/commands/command-descriptions/ljob-command.html> <https://techdocs.broadcom.com/us/en/ca-mainframe-software/automation/ca-workload-automation-ca-7-edition/12-1/using/primer/schedule-id-examples.html>

Language and compiler documentation

[B13] GnuCOBOL project. *GnuCOBOL 3.1.2 NEWS* and *GnuCOBOL manual*.

<https://sourceforge.net/p/gnucobol/code/HEAD/tree/tags/gnucobol-3.1.2/NEWS>

<https://gnucobol.sourceforge.io/doc/gnucobol.html>

[B14] Oracle. *The Java Language Specification, Java SE 21 Edition*.
<https://docs.oracle.com/javase/specs/jls/se21/html/index.html>

Public sample applications measured in this edition

See "Open-Source Attribution" for repositories, licences and pinned commits.